

1. Latar Belakang

Perkembangan transportasi perkotaan di Kota New York menuntut pengelolaan layanan taksi yang semakin efisien, adaptif, dan berbasis data. Tingginya mobilitas masyarakat, variasi waktu perjalanan, serta perbedaan karakteristik wilayah menyebabkan permintaan layanan taksi tidak merata baik dari sisi waktu, lokasi, maupun jenis perjalanan. Kondisi ini menuntut perusahaan dan vendor taksi untuk mampu mengoptimalkan operasional armada agar tetap kompetitif dan berkelanjutan.

Sebagai regulator dan penyedia data, NYC Taxi and Limousine Commission (NYC-TLC) memiliki peran strategis dalam menyediakan informasi yang dapat dimanfaatkan oleh vendor taksi untuk meningkatkan kinerja operasional dan pendapatan. Data perjalanan taksi yang tercatat oleh NYC-TLC menyimpan potensi besar untuk dianalisis guna memahami pola perjalanan, tren permintaan, distribusi wilayah, serta perilaku pembayaran penumpang.

Namun, dalam praktiknya, banyak vendor taksi masih menghadapi tantangan dalam menentukan waktu operasional optimal, penempatan armada yang tepat, serta strategi peningkatan pendapatan, khususnya terkait metode pembayaran dan jenis perjalanan yang paling menguntungkan. Tanpa analisis data yang komprehensif, pengambilan keputusan sering kali dilakukan secara intuitif dan kurang efisien.

Oleh karena itu, diperlukan suatu analisis berbasis data perjalanan taksi NYC-TLC untuk mengidentifikasi pola waktu, lokasi, dan pendapatan, serta menyusun rekomendasi strategis yang dapat digunakan oleh vendor taksi dalam mengoptimalkan operasional armada dan meningkatkan pendapatan.

2. Rumusan Masalah

Berdasarkan latar belakang tersebut, maka rumusan masalah dalam analisis ini adalah sebagai berikut:

- Bagaimana pola dan tren perjalanan taksi di Kota New York selama Januari 2023 berdasarkan waktu harian dan mingguan?
- Pada waktu dan jam berapa permintaan perjalanan serta pendapatan taksi mencapai tingkat tertinggi?
- Wilayah mana saja yang menjadi lokasi penjemputan dan pengantaran dengan volume dan pendapatan tertinggi?
- Bagaimana distribusi pendapatan taksi berdasarkan metode pembayaran, kode tarif, dan jenis perjalanan?
- Bagaimana hubungan antara metode pembayaran dengan besaran tip yang diterima pengemudi?
- Insight strategis apa yang dapat diambil dari pola perjalanan dan pendapatan tersebut untuk mendukung pengambilan keputusan operasional vendor taksi?
- Rekomendasi apa yang dapat diberikan kepada vendor taksi agar dapat meningkatkan efisiensi armada dan pendapatan berbasis data NYC-TLC?

3. Data

Untuk menjawab pertanyaan di atas, kita akan menganalisa data catatan perjalanan TLC yang sudah dikumpulkan oleh perusahaan. Dataset dapat diakses [di sini](#).

Taksi Hijau: Secara hukum hanya boleh mengambil penumpang di wilayah luar Manhattan (The Bronx, Brooklyn, Queens, Staten Island) dan Manhattan bagian utara (di atas 96th St)

Dataset ini berisi informasi terkait rute perjalanan taksi TLC, ada 20 kolom didalam dataset NYC TLC Trip Record, yaitu:

- VendorID : Id penyedia LPEP
 - 1 = Creative Mobile Technologies LLC.
 - 2 = VeriFone Inc.
- Waktu & Durasi:
 - lpep_pickup_datetime : Tanggal & waktu penjemputan
 - lpep_dropoff_datetime : Tanggal & waktu penumpang turun

- Durasi_perjalanan: lpep_dropoff_datetime - lpep_pickup_datetime
- Lokasi (Taxi Zones):
 - PULocationID : Zona taksi TLC ketika penjemputan
 - DOLocationID : Zona taksi TLC ketika penumpang turun
- Detail Perjalanan:
 - Passenger_count : Jumlah penumpang dalam kendaraan
 - Trip_distance : Jarak tempuh (dalam mill)
 - RateCodeID : Kode tarif akhir perjalanan
 - 1 = Tarif standar --> \$3.00
 - 2 = Bandara John F. Kennedy (JFK) --> \$70.00
 - 3 = Bandara Newark (EWR) --> \$20.00
 - 4 = Tarif luar kota Nassau / Westchester --> dua kali lipat tarif standar
 - 5 = Tarif negosiasi
 - 6 = Tarif perjalanan kelompok
 - Store_and_fwd_flag : Menunjukkan apakah catatan perjalanan disimpan dalam memori kendaraan sebelum dikirim ke vendor
 - Y = Tidak disimpan dan teruskan
 - N = Simpan dan teruskan
 - Payment_type : Kode yang menunjukkan jenis pembayaran yang dilakukan oleh penumpang
 - 1 = Kartu Kredit
 - 2 = Tunai
 - 3 = Gratis
 - 4 = Dispute
 - 5 = Tidak diketahui
 - 6 = Perjalanan dibatalkan
 - Trip_type : Kode yang menunjukkan jenis perjalanan
 - 1 = Street-hail
 - 2 = Dispatch
- Rincian Biaya:
 - Total_amount : Total biaya yang dibayar penumpang, selain tip tunai
 - Fare_amount : Tarif dasar perjalanan berdasarkan waktu/jarak (belum termasuk biaya tambahan)
 - \$0.50 = Waktu jam sibuk
 - \$1 = Waktu malam hari
 - MTA_tax : \$ 0.50 Pajak otomatis yang dikenakan perpejalanan
 - Improvement_surcharge : \$0.30 Biaya tambahan saat menurunkan penumpang diwaktu hujan, peraturan dimutai ditetapkan pada tahun 2015
 - Tip_amount : Jumlah tip yang tercatat saat menggunakan kartu kredit, tip tunai tidak termasuk
 - Tolls_amount : Jumlah total biaya tol yang digunakan dalam perjalanan
 - extra : Biaya tambahan lainnya, seperti jam sibuk atau tengah malam
 - \$5.00 untuk jam sibuk (jam 16.00 - 20.00 pada hari kerja, tidak termasuk libur nasional)

- `congestion_surcharge` :
 - biaya tambahan karena kemacetan, yang dimulai/berakhir/melewati Manhattan diselatan jalan ke-96
 - taksi kuning : \$2.50
 - taksi hijau : \$2.75 / FWV
 - semua perjalanan berbagi tumpangan : \$0.75
- `ehail_fee` : Biaya pemesanan

Berikut 5 baris teratas dan terbawah dari dataset NYC TLC Trip Record.

In [1155]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as ms
from scipy.stats import normaltest, chi2_contingency, mannwhitneyu, ttest_ind
import folium
```

setting agar bisa menampilkan semua kolom

```
pd.set_option("display.max_columns",None)
```

In [1156]:

```
df= pd.read_csv("C:/Users/HP/Downloads/Projek Modul 2/NYC TLC Trip Record.csv")
display(df.head(), df.tail())
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\4196828646.py:1: DtypeWarning: Columns (3) have mixed types. Specify dtype option on import or set low_memory=False.

```
df= pd.read_csv("C:/Users/HP/Downloads/Projek Modul 2/NYC TLC Trip Record.csv")
```

	VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	store_and_fwd_flag	RatecodeID	passenger_count
0	2	2023-01-01 00:26:10	2023-01-01 00:37:11	N	1.0	1
1	2	2023-01-01 00:51:03	2023-01-01 00:57:49	N	1.0	1
2	2	2023-01-01 00:35:12	2023-01-01 00:41:32	N	1.0	1
3	1	2023-01-01 00:13:14	2023-01-01 00:19:03	N	1.0	1
4	1	2023-01-01 00:33:04	2023-01-01 00:39:02	N	1.0	1

	VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	store_and_fwd_flag	RatecodeID	passenger_count
68206	2	2023-01-31 22:29:00	2023-01-31 22:42:00	NaN	N	1
68207	2	2023-01-31 22:40:00	2023-01-31 22:48:00	NaN	N	1
68208	2	2023-01-31 23:46:00	2023-02-01 00:02:00	NaN	N	1
68209	2	2023-01-31 23:01:00	2023-01-31 23:19:00	NaN	N	1
68210	2	2023-01-31 23:51:00	2023-02-01 00:07:00	NaN	N	1

4. Data Understanding and Cleaning

Sebelum masuk ke dalam analisis, kita perlu mengenal dataset kita lebih jauh dalam tahapan *data understanding*. Dari proses ini, kita akan tahu anomali-anomali apa saja yang terdapat di dalam dataset kita dan perlu ditangani dalam tahapan *data cleaning*. Setiap penanganan anomali yang dilakukan, akan disertai dengan justifikasi langkah yang diambil, baik secara *domain knowledge* maupun secara statistik.

In [1157]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 68211 entries, 0 to 68210
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   VendorID              68211 non-null  int64
1   lpep_pickup_datetime  68211 non-null  object
2   lpep_dropoff_datetime 68211 non-null  object
3   store_and_fwd_flag    63887 non-null  object
4   RatecodeID            63887 non-null  float64
5   PULocationID          68211 non-null  int64
6   DOLocationID          68211 non-null  int64
7   passenger_count       63887 non-null  float64
8   trip_distance         68211 non-null  float64
9   fare_amount           68211 non-null  float64
10  extra                 68211 non-null  float64
11  mta_tax               68211 non-null  float64
12  tip_amount            68211 non-null  float64
13  tolls_amount          68211 non-null  float64
14  ehail_fee             0 non-null      float64
15  improvement_surcharge 68211 non-null  float64
16  total_amount          68211 non-null  float64
17  payment_type          63887 non-null  float64
18  trip_type             63877 non-null  float64
19  congestion_surcharge  63887 non-null  float64
dtypes: float64(14), int64(3), object(3)
memory usage: 10.4+ MB
Jumlah baris dan kolom di dataset df adalah (68211, 20)
```

• cek data unik di setiap kolom

```
In [1158]:
pd.set_option('display.max_colwidth', 1)

# data unik di tiap kolom

listItem = []
for col in df.columns :
    listItem.append( [col, df[col].nunique(), df[col].unique()] )
```

```
tabel1Desc = pd.DataFrame(columns=['Column Name', 'Number of Unique', 'Unique Sample'],
                             data=listItem)
tabel1Desc
Out[1158]:
```

	Column Name	Number of Unique	Unique Sample
0	VendorID	2	[2, 1]
1	lpep_pickup_datetime	66575	[2023-01-01 00:26:10, 2023-01-01 00:51:03, 2023-01-01 00:35:12, 2023-01-01 00:13:14, 2023-01-01 00:33:04, 2023-01-01 00:53:31, 2023-01-01 00:09:14, 2023-01-01 00:11:58, 2023-01-01 00:41:29, 2023-01-01 00:50:32, 2023-01-01 00:16:12, 2023-01-01 00:08:43, 2023-01-01 00:26:32, 2023-01-01 00:18:35, 2023-01-01 00:39:32, 2023-01-01 00:49:34, 2023-01-01 00:10:45, 2023-01-01 00:35:11, 2023-01-01 00:31:06, 2023-01-01 00:14:37, 2023-01-01 00:26:21, 2023-01-01 00:41:56, 2023-01-01 00:32:02, 2023-01-01 00:55:13, 2023-01-01 00:01:31, 2023-01-01 00:22:31, 2023-01-01 00:57:28, 2023-01-01 00:35:21, 2023-01-01 00:52:52, 2023-01-01 00:32:56, 2023-01-01 00:24:01, 2023-01-01 00:46:48, 2023-01-01 00:40:58, 2023-01-01 00:20:23, 2023-01-01 01:49:24, 2023-01-01 00:41:53, 2023-01-01 00:56:26, 2023-01-01 00:33:10, 2023-01-01 00:58:28, 2023-01-01 00:42:49, 2023-01-01 00:48:21, 2023-01-01 00:08:05, 2023-01-01 00:13:47, 2023-01-01 00:16:10, 2023-01-01 00:04:25, 2023-01-01 00:33:38, 2023-01-01 00:46:08, 2023-01-01 01:00:08, 2023-01-01 00:28:04, 2023-01-01 00:23:04, 2023-01-01 00:34:07, 2023-01-01 00:52:03, 2023-01-01 00:25:31, 2023-01-01 00:53:09, 2023-01-01 00:42:37, 2023-01-01 00:39:52, 2023-01-01 00:15:32, 2023-01-01 00:06:48, 2023-01-01 00:57:15, 2023-01-01 00:14:13, 2023-01-

ID	Column Name	Number of Unique	Unique Sample
1	lpep_dropoff_datetime	66519	00:00:40, 2023-01-01 00:37:11, 2023-01-01 00:57:49, 2023-01-01 00:52:48, 2023-01-01 00:34:13, 2023-01-01 00:44:41, 2023-01-01 00:11:12, 2023-01-01 01:44:43, 2023-01-01 01:48:11, 2023-01-01 01:11:39, 2023-01-01 01:54:25, 2023-01-01 01:59:10, 2023-01-01 01:22:35, 2023-01-01 01:37:10, 2023-01-01 01:46:22, 2023-01-01 01:59:48, 2023-01-01 01:09:49, 2023-01-01 01:23:35, 2023-01-01 01:53:20, 2023-01-01 01:32:14, 2023-01-01 01:01:05, 2023-01-01 01:34:41, 2023-01-01 01:52:08, 2023-01-01 01:49:28, 2023-01-01 01:03:59, 2023-01-01 01:13:11, 2023-01-01 01:40:06, 2023-01-01 01:33:50, 2023-01-01 01:45:19, 2023-01-01 01:51:29, 2023-01-01 01:34:07, 2023-01-01 01:10:08, 2023-01-01 01:32:17, 2023-01-01 01:48:28, 2023-01-01 01:30:43, 2023-01-01 01:51:10, 2023-01-01 01:09:36, 2023-01-01 01:19:54, 2023-01-01 01:31:10, 2023-01-01 01:42:30, 2023-01-01 00:57:51, 2023-01-01 01:27:36, ...]
2	lpep_dropoff_datetime	66519	[2023-01-01 00:37:11, 2023-01-01 00:57:49, 2023-01-01 00:41:32, 2023-01-01 00:19:03, 2023-01-01 00:39:02, 2023-01-01 01:11:04, 2023-01-01 00:26:39, 2023-01-01 00:24:55, 2023-01-01 00:46:26, 2023-01-01 01:13:42, 2023-01-01 00:41:43, 2023-01-01 00:17:08, 2023-01-01 00:45:31, 2023-01-01 00:30:09, 2023-01-01 01:18:06, 2023-01-01 01:08:23, 2023-01-01 00:19:37, 2023-01-01 01:17:35, 2023-01-01 00:42:23, 2023-01-01 00:21:50, 2023-01-01 00:30:26, 2023-01-01 00:46:06, 2023-01-01 00:41:25, 2023-01-01 00:59:47, 2023-01-01 00:16:02, 2023-01-01 00:39:24, 2023-01-01 01:09:31, 2023-01-01 00:53:49, 2023-01-01 01:00:53, 2023-01-01 00:42:53, 2023-01-01 00:32:05, 2023-01-01 01:08:17, 2023-01-01 01:04:32, 2023-01-01 00:24:32, 2023-01-01 01:57:51, 2023-01-01 00:41:58, 2023-01-01 01:14:41, 2023-01-01 00:43:08, 2023-01-01 01:14:46, 2023-01-01 00:51:53, 2023-01-01 01:23:50, 2023-01-01 00:13:26, 2023-01-01 00:25:57, 2023-01-01 00:54:55, 2023-01-01 00:17:50, 2023-01-01 00:38:51, 2023-01-01 00:59:10, 2023-01-01 01:05:46, 2023-01-01 00:43:45, 2023-01-01 23:18:32, 2023-01-01 00:50:01, 2023-01-01 00:57:56, 2023-01-01 00:38:40, 2023-01-01 01:13:35, 2023-01-01 00:47:15, 2023-01-01 00:48:54, 2023-01-01 00:23:03, 2023-01-01 01:08:47, 2023-01-01 01:00:15, 2023-01-01 00:25:43, 2023-01-01 01:21:19, 2023-01-01 00:43:57, 2023-01-01 00:48:26, 2023-01-01 00:38:12, 2023-01-01 01:57:31, 2023-01-01 01:11:23, 2023-01-01 02:05:51, 2023-01-01 01:41:21, 2023-01-01 02:05:19, 2023-01-01 02:11:36, 2023-01-01 01:25:11, 2023-01-01 01:41:30, 2023-01-01 02:04:14, 2023-01-01 02:05:29, 2023-01-01 01:17:55, 2023-01-01 01:53:43, 2023-01-01 02:27:01, 2023-01-01 01:43:55, 2023-01-01 01:13:02, 2023-01-01 02:00:06, 2023-01-01 02:00:57, 2023-01-01 02:13:18, 2023-01-01 01:07:17, 2023-01-01 01:26:01, 2023-01-01 01:49:28, 2023-01-01 01:38:15, 2023-01-01 01:47:57, 2023-01-01 02:05:02, 2023-01-01 01:52:34, 2023-01-01 01:30:01, 2023-01-01 01:41:04, 2023-01-01 02:16:18, 2023-01-01 01:40:20, 2023-01-01 02:00:28, 2023-01-01 01:12:39, 2023-01-01 01:23:25, 2023-01-01 01:37:17, 2023-01-01 02:26:39, 2023-01-01 01:07:05, 2023-01-01 01:27:43, ...]
3	store_and_fwd_flag	2	[N, Y, nan]
4	RatecodeID	6	[1.0, 5.0, 4.0, 3.0, 2.0, 99.0, nan]
5	PULocationID	226	[166, 24, 223, 41, 181, 255, 75, 66, 195, 83, 97, 7, 42, 95, 260, 134, 130, 258, 129, 116, 92, 65, 74, 17, 152, 210, 112, 69, 243, 82, 179, 168, 80, 25, 145, 264, 33, 226, 52, 169, 36, 51, 157, 43, 256, 173, 70, 251, 40, 101, 61, 241, 146, 147, 228, 171, 138, 56, 244, 263, 196, 37, 236, 213, 28, 117, 14, 188, 177, 29, 262, 250, 72, 216, 22, 20, 139, 55, 257, 193, 127, 219, 3, 119, 265, 167, 150, 78, 135, 247, 93, 159, 48, 31, 91, 53, 160, 94, 47, 254, ...]
			[143, 43, 179, 238, 74, 262, 45, 75, 166, 140, 234, 148, 255, 186, 210, 129, 68, 260, 42, 24, 244, 146, 135, 173, 216, 142, ...]

	Column Name	Number of Unique	Unique Sample
6	location_id	249	265, 164, 7, 83, 95, 130, 223, 80, 48, 263, 196, 250, 226, 239, 189, 41, 168, 162, 141, 33, 145, 151, 116, 195, 249, 235, 160, 98, 79, 229, 181, 193, 237, 233, 264, 102, 10, 57, 258, 82, 170, 61, 112, 231, 97, 17, 169, 96, 219, 243, 236, 56, 36, 114, 157, 161, 54, 90, 40, 188, 70, 63, 251, 101, 92, 153, 53, 57, 256, 65, 257, 138, 85, 52, ...]
7	passenger_count	10	[1.0, 2.0, 4.0, 3.0, 6.0, 5.0, 0.0, 7.0, 9.0, 8.0, nan]
8	trip_distance	1870	[2.58, 1.81, 0.0, 1.3, 1.1, 2.78, 3.8, 1.88, 1.11, 4.22, 4.8, 1.99, 7.08, 3.23, 5.91, 11.3, 8.28, 2.02, 0.96, 0.53, 1.51, 0.88, 2.48, 3.53, 2.25, 4.29, 2.53, 1.7, 10.44, 6.78, 1.39, 1.31, 6.7, 3.52, 1.74, 2.31, 1.37, 10.35, 1.05, 6.19, 2.8, 1.13, 3.19, 1.26, 2.87, 8.55, 4.46, 1.18, 2.5, 9.01, 0.63, 2.08, 1.47, 7.83, 2.15, 12.73, 1.62, 7.3, 2.73, 4.9, 2.17, 0.6, 0.8, 2.39, 1.23, 10.45, 2.23, 2.21, 5.2, 5.25, 6.32, 3.82, 3.58, 0.7, 2.91, 5.13, 12.4, 1.45, 7.52, 2.45, 0.54, 0.47, 1.16, 15.39, 5.34, 2.56, 14.97, 0.62, 0.87, 0.01, 0.86, 2.38, 2.64, 11.34, 1.61, 2.0, 4.08, 10.96, 6.14, 3.16, ...]
9	fare_amount	2553	[14.9, 10.7, 7.2, 6.5, 6.0, 17.7, 19.1, 14.2, 24.7, 26.8, 11.4, 30.3, 35.9, 32.5, 15.0, 44.3, 12.8, 8.6, 5.8, 16.3, 19.8, 18.4, 13.5, 8.0, 47.1, 60.0, 7.9, 10.0, 3.0, 5.5, 17.0, 75.0, 15.6, 47.8, 21.2, 37.3, 52.0, 4.4, 51.3, 25.5, 5.1, 9.3, 29.6, 48.5, 28.2, 21.9, 5.0, 4.0, 25.4, 36.6, 66.0, 20.0, 50.0, 190.0, 20.5, 46.4, 12.1, 31.0, 33.8, 26.1, 22.6, 15.5, 24.0, 25.0, 10.5, 19.5, 100.0, 40.0, 31.7, 40.8, 80.0, 110.0, 12.0, 14.5, 43.6, 23.0, 90.0, 49.2, 16.0, 28.9, 30.0, 25.8, 35.0, 35.2, 7.0, 78.0, 27.5, 32.4, 70.0, 41.5, 21.0, 45.0, 9.0, 54.8, 58.3, 38.0, 8.5, 23.3, 28.65, 49.9, ...]
10	extra	16	[1.0, 0.5, 3.75, 0.0, 3.25, 6.0, 5.0, 2.75, 5.5, -1.0, 2.5, 5.25, 7.5, -2.5, -0.5, 12.5]
11	mta_tax	6	[0.5, 1.5, 0.0, 1.0, -0.5, 2.75]
12	tip_amount	1492	[4.03, 2.64, 1.94, 1.7, 0.0, 4.85, 1.0, 3.0, 3.33, 4.04, 5.0, 2.42, 3.76, 4.46, 4.32, 4.18, 9.92, 12.61, 2.5, 2.78, 1.36, 2.22, 2.8, 2.63, 6.26, 5.3, 7.11, 2.6, 2.25, 4.58, 1.38, 7.55, 2.08, 1.5, 6.41, 3.2, 4.88, 2.36, 8.71, 6.69, 4.45, 1.75, 7.66, 2.0, 1.52, 2.49, 9.0, 6.65, 1.55, 6.44, 10.33, 21.5, 4.38, 6.42, 7.81, 3.96, 6.06, 8.79, 4.87, 4.0, 7.52, 6.32, 5.02, 2.65, 3.11, 8.2, 3.62, 20.06, 3.75, 3.34, 8.75, 15.2, 9.24, 4.2, 3.82, 2.18, 3.3, 0.05, 6.09, 4.5, 4.35, 5.22, 8.66, 1.66, 15.66, 6.0, 1.8, 5.01, 4.17, 3.92, 3.9, 6.55, 3.54, 10.61, 5.5, 5.75, 10.26, 5.44, 2.95, 9.9, ...]
13	tolls_amount	26	[0.0, 6.55, 4.0, 13.75, 9.0, 3.0, 11.75, 2.45, 7.0, 13.1, 18.3, 20.5, 12.75, 22.75, 21.3, 14.75, 25.5, 36.05, 19.3, 6.25, 9.55, 8.55, 8.0, 5.09, 6.75, 20.0]
14	ehail_fee	0	[nan]
15	improvement_surcharge	5	[1.0, 0.0, 0.3, -1.0, -0.3]
16	total_amount	4670	[24.18, 15.84, 11.64, 10.2, 8.0, 22.95, 29.2, 16.7, 10.7, 32.95, 32.05, 19.98, 38.55, 24.24, 41.15, 34.5, 15.0, 54.55, 15.3, 11.1, 12.12, 8.3, 13.9, 9.7, 22.56, 26.76, 25.92, 25.08, 18.75, 13.75, 59.52, 75.66, 10.4, 5.5, 21.6, 16.68, 20.2, 13.2, 49.6, 8.16, 21.72, 78.05, 15.78, 27.11, 13.32, 18.1, 58.35, 30.81, 13.0, 39.8, 11.25, 16.0, 19.83, 54.5, 8.28, 53.8, 37.8, 22.93, 10.0, 38.46, 19.2, 7.6, 9.0, 24.38, 14.16, 43.56, 53.75, 40.14, 24.4, 34.85, 26.7, 8.75, 6.0, 38.31, 12.5, 47.65, 9.12, 10.79, 12.48, 68.5, 59.55, 21.0, 20.85, 9.3, 13.1, 191.0, 14.5, 17.4, 60.0, 13.35, 32.19, 61.98, 33.45, 26.3, 23.2, 18.98, 33.5, 38.52, 46.86, 28.6, ...]
17	payment_type	5	[1.0, 2.0, 3.0, 4.0, 5.0, nan]
18	trip_type	2	[1.0, 2.0, nan]
19	congestion_surcharge	4	[2.75, 0.0, 2.5, -2.75, nan]

- **Missing Value**

```
In [1159]:
```

```
# Cek duplicated
```

```
df.duplicated().sum() # tidak ada data yang duplikat
```

```
Out[1159]:
```

```
np.int64(0)
```

```
In [1160]:
```

```
# Cek ada berapa data yang missing value di tiap kolomnya
```

```
df.isna().sum()
```

```
Out[1160]:
```

```
VendorID          0
lpep_pickup_datetime  0
lpep_dropoff_datetime  0
store_and_fwd_flag    4324
RatecodeID         4324
PULocationID        0
DOLocationID        0
passenger_count     4324
trip_distance        0
fare_amount          0
extra               0
mta_tax             0
tip_amount          0
tolls_amount        0
ehail_fee          68211
improvement_surcharge  0
total_amount         0
payment_type        4324
trip_type           4334
congestion_surcharge  4324
dtype: int64
```

```
In [1161]:
```

```
# Persentase missing value di tiap kolo
```

```
df.isna().sum()/df.shape[0]*100
```

```
Out[1161]:
```

```
VendorID          0.000000
lpep_pickup_datetime  0.000000
lpep_dropoff_datetime  0.000000
store_and_fwd_flag    6.339154
RatecodeID          6.339154
PULocationID        0.000000
DOLocationID        0.000000
passenger_count     6.339154
trip_distance        0.000000
fare_amount          0.000000
extra              0.000000
mta_tax             0.000000
tip_amount          0.000000
tolls_amount        0.000000
ehail_fee          100.000000
improvement_surcharge  0.000000
total_amount         0.000000
payment_type        6.339154
trip_type           6.353814
congestion_surcharge  6.339154
dtype: float64
```

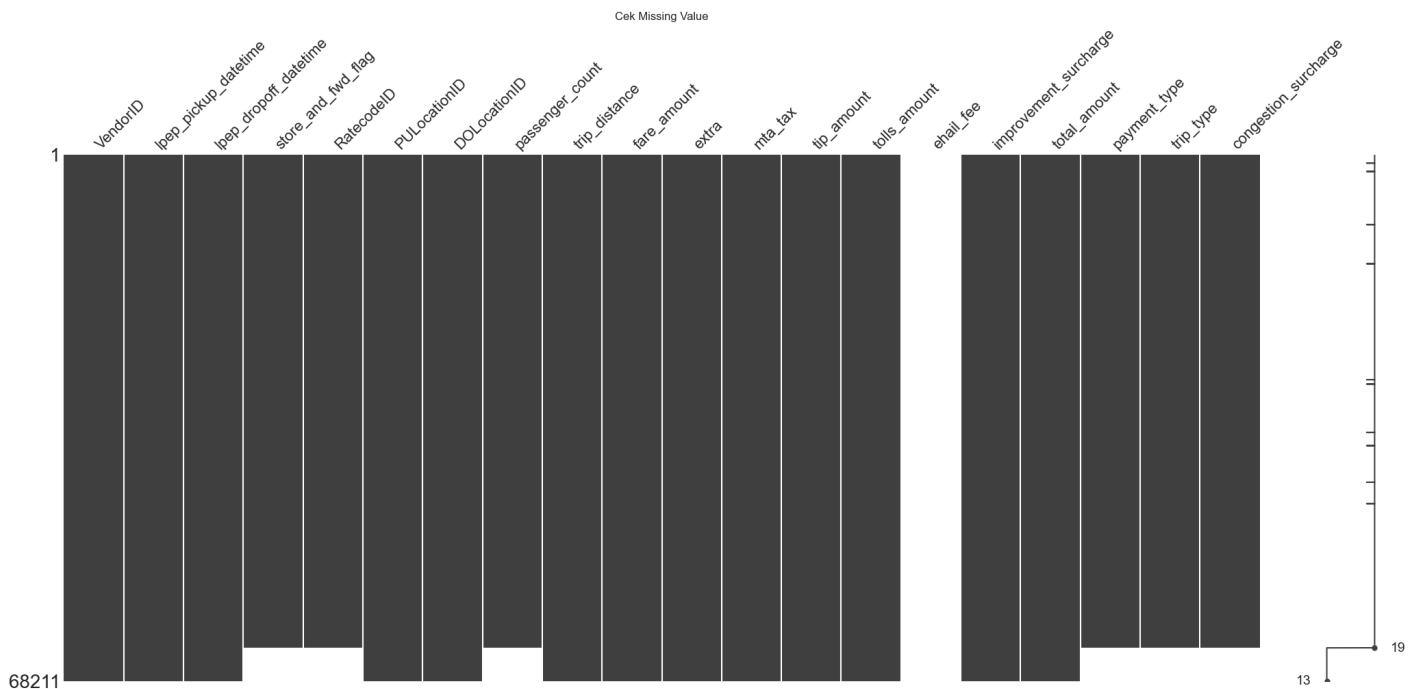
```
In [1162]:
```

```
# cek posisi missing value
```

```
ms.matrix(df)
```

```
plt.title("Cek Missing Value")
```

```
plt.show()
```



Secara garis besar:

- kolom `ehail_fee` tidak memiliki nilai sehingga boleh di hapus kolomnya.
- beberapa kolom seperti `store_and_fwd_flag` , `RatecodeID` , `passenger_count` , `payment_type` , `trip_type` , dan `congestion_surcharge` memiliki *missing value* yang cukup rendah <10% dan memiliki posisi yang cenderung sama.
- kolom `RatecodeID` memiliki data bernilai 99 yang bukan termasuk klasifikasi data dikolom tersebut.
- kolom `lpep_pickup_datetime` dan `lpep_dropoff_datetime` memiliki type data object yang seharusnya memiliki type data `datetime` sehingga harus dirubah terlebih dahulu.

Data Cleaning

• Kolom `ehail_fee`

In [1163]:

```
# Drop kolom ehail_fee karena tidak memiliki nilai
```

```
df.drop(columns= "ehail_fee", inplace= True)
```

• Kolom `lpep_pickup_datetime` & `lpep_dropoff_datetime`

In [1164]:

```
# Mengubah type data pada lpep_pickup_datetime & lpep_dropoff_datetime
```

```
df["lpep_pickup_datetime"] = pd.to_datetime(df["lpep_pickup_datetime"])
```

```
df["lpep_dropoff_datetime"] = pd.to_datetime(df["lpep_dropoff_datetime"])
```

In [1165]:

```
# Melihat rentang data pertahunnya
```

```
df.groupby(df["lpep_pickup_datetime"].dt.year).size()
```

Out[1165]:

```
lpep_pickup_datetime
```

```
2009    1
```

```
2022    2
```

```
2023  68208
```

```
dtype: int64
```

In [1166]:

```
# Melihat data perbulan
```

```
df.groupby(df["lpep_pickup_datetime"].dt.month).size()
```

Out[1166]:

```
lpep_pickup_datetime
```

```
1    68208
```

```
2      1
```

```
12     2
```

```
dtype: int64
```

```
In [1167]:
```

```
# Melihat data pertahunnya
```

```
df.groupby(df["lpep_dropoff_datetime"].dt.year).size()
```

```
Out[1167]:
```

```
lpep_dropoff_datetime
```

```
2009    1
```

```
2022    2
```

```
2023   68208
```

```
dtype: int64
```

```
In [1168]:
```

```
# Melihat data perbulan
```

```
df.groupby(df["lpep_dropoff_datetime"].dt.month).size()
```

```
Out[1168]:
```

```
lpep_dropoff_datetime
```

```
1    68193
```

```
2     16
```

```
12     2
```

```
dtype: int64
```

```
In [1169]:
```

```
df[df["lpep_dropoff_datetime"].dt.month == 2]
```

```
Out[1169]:
```

	VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	store_and_fwd_flag	Ratecode
25100	2	2023-02-01 03:10:05	2023-02-01 03:40:35	N	
61992	2	2023-01-31 08:46:26	2023-02-01 07:34:55	N	
62120	2	2023-01-31 09:02:52	2023-02-01 09:01:47	N	
62148	2	2023-01-31 09:53:33	2023-02-01 00:00:00	N	
62510	2	2023-01-31 12:41:35	2023-02-01 12:05:19	N	
62591	2	2023-01-31 13:58:22	2023-02-01 11:46:10	N	
62653	2	2023-01-31 13:57:30	2023-02-01 10:49:04	N	
62762	2	2023-01-31 15:21:47	2023-02-01 00:00:00	N	
63105	2	2023-01-31 17:45:46	2023-02-01 17:27:05	N	
63279	2	2023-01-31 18:22:24	2023-02-01 00:00:00	N	
63462	2	2023-01-31 19:40:22	2023-02-01 00:00:00	N	
63858	2	2023-01-31 23:56:16	2023-02-01 00:03:16	N	
63872	2	2023-01-31 23:58:23	2023-02-01 00:07:16	N	
63876	2	2023-01-31 23:53:41	2023-02-01 00:31:14	N	
68208	2	2023-01-31 23:46:00	2023-02-01 00:02:00	NaN	N
68210	2	2023-01-31 23:51:00	2023-02-01 00:07:00	NaN	N

- karena data lpep_pickup_datetime & lpep_dropoff_datetime pada tahun 2009 dan 2022 tidak lengkap, maka kita hapus dan hanya tahun 2023 yang akan kita gunakan
- karena data lpep_pickup_datetime perbulan hanya bulan januari yang lengkap, maka kita hapus sisa bulan yang lainnya
- untuk lpep_dropoff_datetime kita buang bulan ke 12 karena anomali, sedangkan bulan ke kedua lanjutan dari lpep_pickup_datetime bulan 1

```
In [1170]:
```

```
df = df[df["lpep_pickup_datetime"].dt.year == 2023]
```

```
In [1171]:
```

```
df = df[df["lpep_dropoff_datetime"].dt.year == 2023]
```

```
In [1172]:
```

```
df = df[df["lpep_pickup_datetime"].dt.month == 1]
```

```
In [1173]:
```

```
df = df[df["lpep_dropoff_datetime"].dt.month != 12]
```

```
In [1174]:
```

```
df["lpep_dropoff_datetime"].dt.month.unique()
```

```
Out[1174]:
```

```
array([1, 2], dtype=int32)
```

```
In [1175]:
```

```
df["lpep_pickup_datetime"].dt.month.unique()
```

```
Out[1175]:
```

```
array([1], dtype=int32)
```

- **store_and_fwd_flag**

```
In [1176]:
```

```
df["store_and_fwd_flag"].mode()
```

```
Out[1176]:
```

```
0    N
```

```
Name: store_and_fwd_flag, dtype: object
```

karena modusnya adalah N, nilai null pada store_and_fwd_flag bisa diisi dengan N

```
In [1177]:
```

```
df["store_and_fwd_flag"] = df["store_and_fwd_flag"].fillna(df["store_and_fwd_flag"].mode()[0])
```

- **passenger_count**

```
In [1178]:
```

```
df["passenger_count"].unique()
```

```
Out[1178]:
```

```
array([ 1.,  2.,  4.,  3.,  6.,  5.,  0.,  7.,  9.,  8., nan])
```

```
In [1179]:
```

```
df["passenger_count"].median()
```

```
Out[1179]:
```

```
np.float64(1.0)
```

```
In [1180]:
```

```
df.groupby("trip_type")["passenger_count"].median()
```

```
Out[1180]:
```

```
trip_type
```

```
1.0    1.0
```

```
2.0    1.0
```

```
Name: passenger_count, dtype: float64
```

```
In [1181]:
```

```
df.groupby("RatecodeID")["passenger_count"].median()
```

```
Out[1181]:
```

```
RatecodeID
```

```
1.0    1.0
```

```
2.0    1.0
```

```
3.0    1.0
```

```
4.0    1.0
```

```
5.0    1.0
```

```
99.0   1.0
```

```
Name: passenger_count, dtype: float64
```

kita ambil nilai median untuk jumlah penumpang

```
In [1182]:
```

```
df["passenger_count"] = df["passenger_count"].fillna(df["passenger_count"].median())
```

- **payment_type**

```
In [1183]:
```

```
# kalo ada data tip artinya dia membayar menggunakan kartu kredit
```

```
df[["payment_type", "tip_amount"]]
```

Out[1183]:

	payment_type	tip_amount
0	1.0	4.03
1	1.0	2.64
2	1.0	1.94
3	1.0	1.70
4	1.0	0.00
...	...	...
68206	NaN	0.00
68207	NaN	0.00
68208	NaN	3.51
68209	NaN	3.20
68210	NaN	2.00

68207 rows × 2 columns

In [1184]:

```
df.loc[(df["payment_type"].isnull()) & (df["tip_amount"] != 0), "payment_type"] = 1
```

In [1185]:

```
df["payment_type"] = df["payment_type"].fillna(5)
```

- **trip_type**

In [1186]:

```
df["trip_type"].mode()[0]
```

Out[1186]:

```
np.float64(1.0)
```

In [1187]:

```
df["trip_type"] = df["trip_type"].fillna(df["trip_type"].mode()[0])
```

- **congestion_surcharge**

In [1188]:

```
df["congestion_surcharge"].unique()
```

Out[1188]:

```
array([ 2.75,  0. ,  2.5 , -2.75,  nan])
```

karena ini data taksi hijau, maka congestion_surcharge berniali \$2.75 jika melewati/turun/naik(hanyak Manhattan bagian utara (di atas 96th St)) di Manhattan

In [1189]:

```
# congestion_surcharge bernilai $2.5 untuk taksi kuning sehingga kita hapus saja
```

```
congestion_surcharge_kuning = df[df["congestion_surcharge"] == 2.5]
```

```
congestion_surcharge_kuning
```

Out[1189]:

	VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	store_and_fwd_flag	Ratecode
2497	2	2023-01-02 19:16:36	2023-01-02 19:30:21	N	
2498	2	2023-01-02 19:49:45	2023-01-02 19:57:56	N	
2691	2	2023-01-02 22:15:33	2023-01-02 22:25:11	N	
2902	2	2023-01-03 07:49:18	2023-01-03 08:10:44	N	
3016	2	2023-01-03 08:43:00	2023-01-03 08:59:42	N	
8698	2	2023-01-05 18:55:20	2023-01-05 19:08:33	N	
9633	2	2023-01-06 09:32:58	2023-01-06 09:57:17	N	
9995	2	2023-01-06 12:38:54	2023-01-06 13:06:32	N	
14200	2	2023-01-08 12:29:11	2023-01-08 12:41:26	N	
15352	2	2023-01-09 06:53:22	2023-01-09 07:01:01	N	
17502	2	2023-01-10 07:33:17	2023-01-10 07:39:01	N	
17952	2	2023-01-10 11:43:03	2023-01-10 12:00:38	N	
18066	2	2023-01-10 12:14:36	2023-01-10 12:53:14	N	
21677	2	2023-01-11 22:10:29	2023-01-11 22:37:25	N	
23011	2	2023-01-12 15:54:15	2023-01-12 16:09:09	N	
27482	2	2023-01-14 15:19:46	2023-01-14 16:01:22	N	

In [1190]:

```
df = df[df["congestion_surcharge"] != 2.5]
```

Karena kebijakan congestion_surcharge bernilai \$2.5 hanya untuk taksi kuning, terdapat 16 baris kolom congestion_surcharge == 2.5 yang kita hapus.

In [1191]:

```
# Cek data yang bernilai negatif
```

```
df[(df["total_amount"]<0) | (df["fare_amount"]<0) | (df["mta_tax"]<0) | (df["improvement_surcharge"]<0) | (df["tip"]<0)]
```

Out[1191]:

	VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	store_and_fwd_flag	Ratecode
662	2	2023-01-01 14:58:13	2023-01-01 14:58:39	N	
898	2	2023-01-01 17:24:52	2023-01-01 17:25:12	N	
1317	2	2023-01-02 02:05:08	2023-01-02 02:12:57	N	
1686	2	2023-01-02 12:56:53	2023-01-02 13:00:10	N	
1864	2	2023-01-02 13:51:33	2023-01-02 13:52:42	N	
...	...	...	...	...	
64376	2	2023-01-04 15:11:00	2023-01-04 15:12:00	N	N
64487	2	2023-01-05 10:51:00	2023-01-05 11:20:00	N	N
65108	2	2023-01-10 10:00:00	2023-01-10 10:26:00	N	N
66459	2	2023-01-19 17:48:00	2023-01-19 17:52:00	N	N
67461	2	2023-01-26 10:10:00	2023-01-26 10:20:00	N	N

179 rows × 19 columns

Jika diperhatikan dari beberapa baris data terdapat data yang bernilai negatif, diketahui bahwa

```
total_amount = fare_amount + mta_tax + improvement_surcharge + tip_amount + tolls_amount + extra + congestion_surcharge
```

maka nilai minus akan di absolutkan, karena nilai tersebut merupakan biaya yang dikeluarkan oleh penumpang.

```
In [1192]:  
df.iloc[:, 9:19] = (df.iloc[:, 9:19]).abs()
```

- **RatecodeID**

Karena posisi missing value RatecodeID dan congestion_surcharge cenderung sama dan persentase missing valuenya adalah $6.3\% < 10\%$ dari data maka boleh dihapus langsung.

```
In [1193]:  
# menghapus baris yang missing  
df.dropna(inplace=True)  
In [1194]:  
df["RatecodeID"].mode()
```

```
Out[1194]:  
0    1.0  
Name: RatecodeID, dtype: float64
```

```
In [1195]:  
# data yang bernilai 99 pada RatecodeID diganti menjadi 1 = Standard rate  
df['RatecodeID'] = df['RatecodeID'].replace(99.0, df['RatecodeID'].mode()[0])
```

```
In [1196]:  
# Cek data unik di tiap kolom, sudah tidak ada data yang bernilai negatif dan NaN
```

```
pd.set_option('display.max_colwidth', 1)
```

```
listItem = []  
for col in df.columns :  
    listItem.append( [col, df[col].nunique(), df[col].unique()] )
```

```
tabel1Desc = pd.DataFrame(columns=['Column Name', 'Number of Unique', 'Unique Sample'],  
                           data=listItem)
```

```
tabel1Desc
```

```
Out[1196]:
```

	Column Name	Number of Unique	Unique Sample
0	VendorID	2	[2, 1] [2023-01-01 00:26:10, 2023-01-01 00:51:03, 2023-01-01 00:35:12, 2023-01-01 00:13:14, 2023-01-01 00:33:04, 2023-01-01 00:53:31, 2023-01-01 00:09:14, 2023-01-01 00:11:58, 2023-01-01 00:41:29, 2023-01-01 00:50:32, 2023-01-01 00:16:12, 2023-01-01 00:08:43, 2023-01-01 00:26:32, 2023-01-01 00:18:35, 2023-01-01 00:39:32, 2023-01-01 00:49:34, 2023-01-01 00:10:45, 2023-01-01 00:35:11, 2023-01-01 00:31:06, 2023-01-01 00:14:37, 2023-01-01 00:26:21, 2023-01-01 00:41:56, 2023-01-01 00:32:02, 2023-01-01 00:55:13, 2023-01-01 00:01:31, 2023-01-01 00:22:31, 2023-01-01 00:57:28, 2023-01-01 00:35:21, 2023-01-01 00:52:52, 2023-01-01 00:32:56, 2023-01-01 00:24:01, 2023-01-01 00:46:48, 2023-01-01 00:40:58, 2023-01-01 00:20:23, 2023-01-01 01:49:24, 2023-01-01 00:41:53, 2023-01-01 00:56:26, 2023-01-01 00:33:10, 2023-01-01 00:58:28, 2023-01-01 00:42:49, 2023-01-01 00:48:21, 2023-01-01 00:08:05, 2023-01-01 00:13:47, 2023-01-01 00:16:10, 2023-01-01 00:04:25, 2023-01-01 00:33:38, 2023-01-01 00:46:08, 2023-01-01 01:00:08, 2023-01-01 00:28:04, 2023-01-01 00:23:04, 2023-01-01 00:34:07, 2023-01-01 00:52:03, 2023-01-01 00:25:31, 2023-01-01 00:53:09, 2023-01-01 00:42:37, 2023-01-01 00:39:52, 2023-01-01 00:15:32, 2023-01-01 00:06:48, 2023-01-01 00:57:15, 2023-01-01 00:14:13, 2023-01-01 00:52:48, 2023-01-01 00:34:13, 2023-01-01 00:44:41, 2023-01-01 00:11:12, 2023-01-01 01:44:43, 2023-01-01 01:04:44, 2023-01-01 01:48:11, 2023-01-01 01:11:39, 2023-01-01
1	lpep_pickup_datetime	62657	

ID	Column Name	Number of Unique	Sample Data	
			Column 1	Column 2
1	DropoffDatetime	62593	[2023-01-01 01:54:25, 2023-01-01 01:59:10, 2023-01-01 01:22:35, 2023-01-01 01:37:10, 2023-01-01 01:46:22, 2023-01-01 01:50:48, 2023-01-01 01:09:49, 2023-01-01 01:23:35, 2023-01-01 01:53:20, 2023-01-01 01:32:14, 2023-01-01 01:01:05, 2023-01-01 01:34:41, 2023-01-01 01:52:08, 2023-01-01 01:49:28, 2023-01-01 01:03:59, 2023-01-01 01:13:11, 2023-01-01 01:40:06, 2023-01-01 01:33:50, 2023-01-01 01:45:19, 2023-01-01 01:51:29, 2023-01-01 01:34:07, 2023-01-01 01:10:08, 2023-01-01 01:32:17, 2023-01-01 01:48:28, 2023-01-01 01:30:43, 2023-01-01 01:51:10, 2023-01-01 01:09:36, 2023-01-01 01:19:54, 2023-01-01 01:31:10, 2023-01-01 01:42:30, 2023-01-01 00:57:51, 2023-01-01 01:27:36, ...]	
			[2023-01-01 00:37:11, 2023-01-01 00:57:49, 2023-01-01 00:41:32, 2023-01-01 00:19:03, 2023-01-01 00:39:02, 2023-01-01 01:11:04, 2023-01-01 00:26:39, 2023-01-01 00:24:55, 2023-01-01 00:46:26, 2023-01-01 01:13:42, 2023-01-01 00:41:43, 2023-01-01 00:17:08, 2023-01-01 00:45:31, 2023-01-01 00:30:09, 2023-01-01 01:18:06, 2023-01-01 01:08:23, 2023-01-01 00:19:37, 2023-01-01 01:17:35, 2023-01-01 00:42:23, 2023-01-01 00:21:50, 2023-01-01 00:30:26, 2023-01-01 00:46:06, 2023-01-01 00:41:25, 2023-01-01 00:59:47, 2023-01-01 00:16:02, 2023-01-01 00:39:24, 2023-01-01 01:09:31, 2023-01-01 00:53:49, 2023-01-01 01:00:53, 2023-01-01 00:42:53, 2023-01-01 00:32:05, 2023-01-01 01:08:17, 2023-01-01 01:04:32, 2023-01-01 00:24:32, 2023-01-01 01:57:51, 2023-01-01 00:41:58, 2023-01-01 01:14:41, 2023-01-01 00:43:08, 2023-01-01 01:14:46, 2023-01-01 00:51:53, 2023-01-01 01:23:50, 2023-01-01 00:13:26, 2023-01-01 00:25:57, 2023-01-01 00:54:55, 2023-01-01 00:17:50, 2023-01-01 00:38:51, 2023-01-01 00:59:10, 2023-01-01 01:05:46, 2023-01-01 00:43:45, 2023-01-01 23:18:32, 2023-01-01 00:50:01, 2023-01-01 00:57:56, 2023-01-01 00:38:40, 2023-01-01 01:13:35, 2023-01-01 00:47:15, 2023-01-01 00:48:54, 2023-01-01 00:23:03, 2023-01-01 01:08:47, 2023-01-01 01:00:15, 2023-01-01 00:25:43, 2023-01-01 01:21:19, 2023-01-01 00:43:57, 2023-01-01 00:48:26, 2023-01-01 00:38:12, 2023-01-01 01:57:31, 2023-01-01 01:11:23, 2023-01-01 02:05:51, 2023-01-01 01:41:21, 2023-01-01 02:05:19, 2023-01-01 02:11:36, 2023-01-01 01:25:11, 2023-01-01 01:41:30, 2023-01-01 02:04:14, 2023-01-01 02:05:29, 2023-01-01 01:17:55, 2023-01-01 01:53:43, 2023-01-01 02:27:01, 2023-01-01 01:43:55, 2023-01-01 01:13:02, 2023-01-01 02:00:06, 2023-01-01 02:00:57, 2023-01-01 02:13:18, 2023-01-01 01:07:17, 2023-01-01 01:26:01, 2023-01-01 01:49:28, 2023-01-01 01:38:15, 2023-01-01 01:47:57, 2023-01-01 02:05:02, 2023-01-01 01:52:34, 2023-01-01 01:30:01, 2023-01-01 01:41:04, 2023-01-01 02:16:18, 2023-01-01 01:40:20, 2023-01-01 02:00:28, 2023-01-01 01:12:39, 2023-01-01 01:23:25, 2023-01-01 01:37:17, 2023-01-01 02:26:39, 2023-01-01 01:07:05, 2023-01-01 01:27:43, ...]	
3	store_and_fwd_flag	2	[N, Y]	
4	RatecodeID	5	[1.0, 5.0, 4.0, 3.0, 2.0]	
5	PULocationID	221	[166, 24, 223, 41, 181, 255, 75, 66, 195, 83, 97, 7, 42, 95, 260, 134, 130, 258, 129, 116, 92, 65, 74, 17, 152, 210, 112, 69, 243, 82, 179, 168, 80, 25, 145, 264, 33, 226, 52, 169, 36, 51, 157, 43, 256, 173, 70, 251, 40, 101, 61, 241, 146, 147, 228, 171, 138, 56, 244, 263, 196, 37, 236, 213, 28, 117, 14, 188, 177, 29, 262, 250, 72, 216, 22, 20, 139, 55, 257, 193, 127, 219, 3, 119, 265, 167, 150, 78, 135, 247, 93, 159, 48, 31, 91, 53, 160, 94, 47, 254, ...]	
6	DOLocationID	248	[143, 43, 179, 238, 74, 262, 45, 75, 166, 140, 234, 148, 255, 186, 210, 129, 68, 260, 42, 24, 244, 146, 135, 173, 216, 142, 265, 164, 7, 83, 95, 130, 223, 80, 48, 263, 196, 250, 226, 239, 189, 41, 168, 162, 141, 33, 145, 151, 116, 195, 249, 235, 160, 98, 79, 229, 181, 193, 237, 233, 264, 102, 10, 37, 258, 82, 170, 21, 112, 221, 27, 17, 160, 22, 232, 212, 222, 52, 22, 114, 157, ...]	

	Column Name	Number of Unique	Unique Sample
7	passenger_count	10	[1.0, 2.0, 4.0, 3.0, 6.0, 5.0, 0.0, 7.0, 9.0, 8.0]
8	trip_distance	1728	[2.58, 1.81, 0.0, 1.3, 1.1, 2.78, 3.8, 1.88, 1.11, 4.22, 4.8, 1.99, 7.08, 3.23, 5.91, 11.3, 8.28, 2.02, 0.96, 0.53, 1.51, 0.88, 2.48, 3.53, 2.25, 4.29, 2.53, 1.7, 10.44, 6.78, 1.39, 1.31, 6.7, 3.52, 1.74, 2.31, 1.37, 10.35, 1.05, 6.19, 2.8, 1.13, 3.19, 1.26, 2.87, 8.55, 4.46, 1.18, 2.5, 9.01, 0.63, 2.08, 1.47, 7.83, 2.15, 12.73, 1.62, 7.3, 2.73, 4.9, 2.17, 0.6, 0.8, 2.39, 1.23, 10.45, 2.23, 2.21, 5.2, 5.25, 6.32, 3.82, 3.58, 0.7, 2.91, 5.13, 12.4, 1.45, 7.52, 2.45, 0.54, 0.47, 1.16, 15.39, 5.34, 2.56, 14.97, 0.62, 0.87, 0.01, 0.86, 2.38, 2.64, 11.34, 1.61, 2.0, 4.08, 10.96, 6.14, 3.16, ...]
9	fare_amount	440	[14.9, 10.7, 7.2, 6.5, 6.0, 17.7, 19.1, 14.2, 24.7, 26.8, 11.4, 30.3, 35.9, 32.5, 15.0, 44.3, 12.8, 8.6, 5.8, 16.3, 19.8, 18.4, 13.5, 8.0, 47.1, 60.0, 7.9, 10.0, 3.0, 5.5, 17.0, 75.0, 15.6, 47.8, 21.2, 37.3, 52.0, 4.4, 51.3, 25.5, 5.1, 9.3, 29.6, 48.5, 28.2, 21.9, 5.0, 4.0, 25.4, 36.6, 66.0, 20.0, 50.0, 190.0, 20.5, 46.4, 12.1, 31.0, 33.8, 26.1, 22.6, 15.5, 24.0, 25.0, 10.5, 19.5, 100.0, 40.0, 31.7, 40.8, 80.0, 110.0, 12.0, 14.5, 43.6, 23.0, 90.0, 49.2, 16.0, 28.9, 30.0, 25.8, 35.0, 35.2, 7.0, 78.0, 27.5, 32.4, 70.0, 41.5, 21.0, 45.0, 9.0, 54.8, 58.3, 38.0, 8.5, 23.3, 28.65, 49.9, ...]
10	extra	13	[1.0, 0.5, 3.75, 0.0, 3.25, 6.0, 5.0, 2.75, 5.5, 2.5, 5.25, 7.5, 12.5]
11	mta_tax	5	[0.5, 1.5, 0.0, 1.0, 2.75]
12	tip_amount	1318	[4.03, 2.64, 1.94, 1.7, 0.0, 4.85, 1.0, 3.0, 3.33, 4.04, 5.0, 2.42, 3.76, 4.46, 4.32, 4.18, 9.92, 12.61, 2.5, 2.78, 1.36, 2.22, 2.8, 2.63, 6.26, 5.3, 7.11, 2.6, 2.25, 4.58, 1.38, 7.55, 2.08, 1.5, 6.41, 3.2, 4.88, 2.36, 8.71, 6.69, 4.45, 1.75, 7.66, 2.0, 1.52, 2.49, 9.0, 6.65, 1.55, 6.44, 10.33, 21.5, 4.38, 6.42, 7.81, 3.96, 6.06, 8.79, 4.87, 4.0, 7.52, 6.32, 5.02, 2.65, 3.11, 8.2, 3.62, 20.06, 3.75, 3.34, 8.75, 15.2, 9.24, 4.2, 3.82, 2.18, 3.3, 0.05, 6.09, 4.5, 4.35, 5.22, 8.66, 1.66, 15.66, 6.0, 1.8, 5.01, 4.17, 3.92, 3.9, 6.55, 3.54, 10.61, 5.5, 5.75, 10.26, 5.44, 2.95, 9.9, ...]
13	tolls_amount	25	[0.0, 6.55, 4.0, 13.75, 9.0, 3.0, 11.75, 2.45, 7.0, 13.1, 18.3, 20.5, 12.75, 22.75, 21.3, 14.75, 25.5, 36.05, 19.3, 6.25, 9.55, 8.55, 8.0, 5.09, 6.75]
14	improvement_surcharge	3	[1.0, 0.0, 0.3]
15	total_amount	3434	[24.18, 15.84, 11.64, 10.2, 8.0, 22.95, 29.2, 16.7, 10.7, 32.95, 32.05, 19.98, 38.55, 24.24, 41.15, 34.5, 15.0, 54.55, 15.3, 11.1, 12.12, 8.3, 13.9, 9.7, 22.56, 26.76, 25.92, 25.08, 18.75, 13.75, 59.52, 75.66, 10.4, 5.5, 21.6, 16.68, 20.2, 13.2, 49.6, 8.16, 21.72, 78.05, 15.78, 27.11, 13.32, 18.1, 58.35, 30.81, 13.0, 39.8, 11.25, 16.0, 19.83, 54.5, 8.28, 53.8, 37.8, 22.93, 10.0, 38.46, 19.2, 7.6, 9.0, 24.38, 14.16, 43.56, 53.75, 40.14, 24.4, 34.85, 26.7, 8.75, 6.0, 38.31, 12.5, 47.65, 9.12, 10.79, 12.48, 68.5, 59.55, 21.0, 20.85, 9.3, 13.1, 191.0, 14.5, 17.4, 60.0, 13.35, 32.19, 61.98, 33.45, 26.3, 23.2, 18.98, 33.5, 38.52, 46.86, 28.6, ...]
16	payment_type	5	[1.0, 2.0, 3.0, 4.0, 5.0]
17	trip_type	2	[1.0, 2.0]
18	congestion_surcharge	2	[2.75, 0.0]

Sebelum dibersihkan, kita memiliki 68,210 baris data, sekarang kita memiliki 63,886 baris. Sekitar 4,324 baris data yang kosong dihapus, dan sisa data kosong diisi dengan data yang dirasa relevan.

• Membuat kolom baru

`trip_durasi = lpep_dropoff_datetime - lpep_pickup_datetime`

In [1197]:

```
# trip_durasi = df["lpep_dropoff_datetime"]- df["lpep_pickup_datetime"]
```

Dataset baru untuk map PULocation & DOLocation

data didapat dari <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

In [1198]:

```
df_map = pd.read_csv("C:/Users/HP/Downloads/Projek Modul 2/taxi_zone_lookup.csv")
df_map.head()
```

Out[1198]:

	LocationID	Borough	Zone	service_zone
0	1	EWB	Newark Airport	EWB
1	2	Queens	Jamaica Bay	Boro Zone
2	3	Bronx	Allerton/Pelham Gardens	Boro Zone
3	4	Manhattan	Alphabet City	Yellow Zone
4	5	Staten Island	Arden Heights	Boro Zone

In [1199]:

```
df_map["service_zone"].unique()
```

Out[1199]:

```
array(['EWB', 'Boro Zone', 'Yellow Zone', 'Airports', nan], dtype=object)
```

In [1200]:

```
df_map["Borough"].unique()
```

Out[1200]:

```
array(['EWB', 'Queens', 'Bronx', 'Manhattan', 'Staten Island', 'Brooklyn',
      'Unknown', nan], dtype=object)
```

In [1201]:

```
df_map.isna().sum()
```

Out[1201]:

```
LocationID    0
Borough       1
Zone          1
service_zone  2
dtype: int64
```

In [1202]:

```
# menghapus data yang mengandung missing
```

```
df_map.dropna(inplace= True)
```

• Marge dataset

In [1203]:

```
# Marge untuk lokasi DOLocationID
```

```
df = pd.merge(df, df_map, left_on= "DOLocationID", right_on= "LocationID")
```

In [1204]:

```
# Rename kolom untuk DOLocationID
```

```
df = df.rename(columns= {"Borough" : "DO_Borough",
                        "Zone" : "DO_Zone",
                        "service_zone" : "DO_service_zone"})
```

```
# Hapus kolom LocationID
```

```
df = df.drop(columns= "LocationID")
```

In [1205]:

```
# Marge untuk lokasi PULocationID
```

```
df = pd.merge(df, df_map, left_on= "PULocationID", right_on= "LocationID")
```

In [1206]:

```
# Rename kolom untuk PULocationID
```

```
df = df.rename(columns= {"Borough" : "PU_Borough",
                        "Zone" : "PU_Zone",
                        "service_zone" : "PU_service_zone"})
```

```
# Hapus kolom LocationID
```

```
df = df.drop(columns= "LocationID")
```

Descriptive Statistic

In [1207]:

```
display(df.describe(), df.describe(include='object'))
```

	VendorID	lpep_pickup_datetime	lpep_dropoff_datetime	RatecodeID	PULocationID
count	63175.000000	63175	63175	63175.000000	63175.000000
mean	1.856478	2023-01-16 22:38:24.929307392	2023-01-16 22:56:20.517736448	1.083831	97.09016
min	1.000000	2023-01-01 00:01:31	2023-01-01 00:13:26	1.000000	3.000000
25%	2.000000	2023-01-09 12:37:23	2023-01-09 12:53:02	1.000000	74.000000
50%	2.000000	2023-01-17 08:51:12	2023-01-17 09:06:24	1.000000	75.000000
75%	2.000000	2023-01-24 16:22:38.500000	2023-01-24 16:38:42	1.000000	116.000000
max	2.000000	2023-01-31 23:58:23	2023-02-01 17:27:05	5.000000	263.000000
std	0.350607	NaN	NaN	0.565046	59.43725

	store_and_fwd_flag	DO_Borough	DO_Zone	DO_service_zone	PU_Borough	PU_Zone
count	63175	63175	63175	63175	63175	63175
unique	2	6	245	4	5	216
top	N	Manhattan	East Harlem South	Boro Zone	Manhattan	East Harlem North
freq	62893	37356	3415	41026	37341	12918

Pada deskriptif data terlihat bahwa:

- Top lokasi penjemputan adalah PU_Zone 216 East Harlem North, Manhattan dengan total 12918 perjalanan
- Top lokasi pengantaran adalah DO_Zone 245 East Harlem North, Manhattan dengan total 3415 perjalanan
- Rata-rata total biaya yang dibayarkan penumpang adalah \$21.04
- Rata-rata jarak tempuh adalah 2.5 mill

In [1208]:

```
# Set style untuk tampilan yang lebih menarik
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
```

```
# Membuat figure dan subplots
```

```
fig, axs = plt.subplots(ncols=3, nrows=6, figsize=(24, 20))
```

```
fig.suptitle('Analisis Outlier Dataset NYC-TLC',
             fontsize=28, fontweight='bold', y=1.02)
```

```
# Flatten axes untuk iterasi yang lebih mudah
```

```
axs = axs.flatten()
```

```
# Warna gradasi untuk setiap boxplot
```

```
colors = plt.cm.viridis(np.linspace(0.2, 0.9, len(df.select_dtypes(include=[np.number]).columns)))
```

```
# Membuat boxplot untuk setiap kolom numerik
```

```
for index, (k, v) in enumerate(df.select_dtypes(include=[np.number]).items()):
```

```
    # Boxplot horizontal dengan styling
```

```
    box = axs[index].boxplot(df[k].dropna(),
```

```
                             vert=False, # Horizontal boxplot
```

```
                             patch_artist=True,
```

```
                             widths=0.7,
```

```
                             showfliers=True,
```

```
                             flierprops=dict(marker='o',
```

```
                             markerfacecolor='red',
```

```

        markersize=4,
        alpha=0.5,
        markeredgewidth=1,
        medianprops=dict(linewidth=2, color='black'),
        boxprops=dict(facecolor=colors[index],
            alpha=0.7,
            edgecolor='navy',
            linewidth=1.5),
        whiskerprops=dict(linewidth=1.5, color='navy'),
        capprops=dict(linewidth=1.5, color='navy'))

# Menambahkan judul untuk setiap subplot
axs[index].set_title(k, fontsize=12, fontweight='bold', pad=10)

# Menambahkan grid untuk memudahkan pembacaan
axs[index].grid(True, axis='x', alpha=0.3, linestyle='--')
axs[index].set_facecolor('#f8f9fa')

# Menambahkan label nilai statistik
quartiles = df[k].dropna().quantile([0.25, 0.5, 0.75])
stats_text = f'Q1: {quartiles[0.25]:.2f}\nMedian: {quartiles[0.5]:.2f}\nQ3: {quartiles[0.75]:.2f}'

# Posisikan text di pojok kanan atas
axs[index].text(0.98, 0.95, stats_text,
    transform=axs[index].transAxes,
    fontsize=9,
    verticalalignment='top',
    horizontalalignment='right',
    bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

# Format x-axis untuk readability yang lebih baik
axs[index].xaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'{x:,.0f}'))

# Menghilangkan y-axis label karena tidak diperlukan untuk boxplot tunggal
axs[index].set_yticklabels([])

# Menyembunyikan subplot yang tidak terpakai
for j in range(index + 1, len(axs)):
    axs[j].set_visible(False)

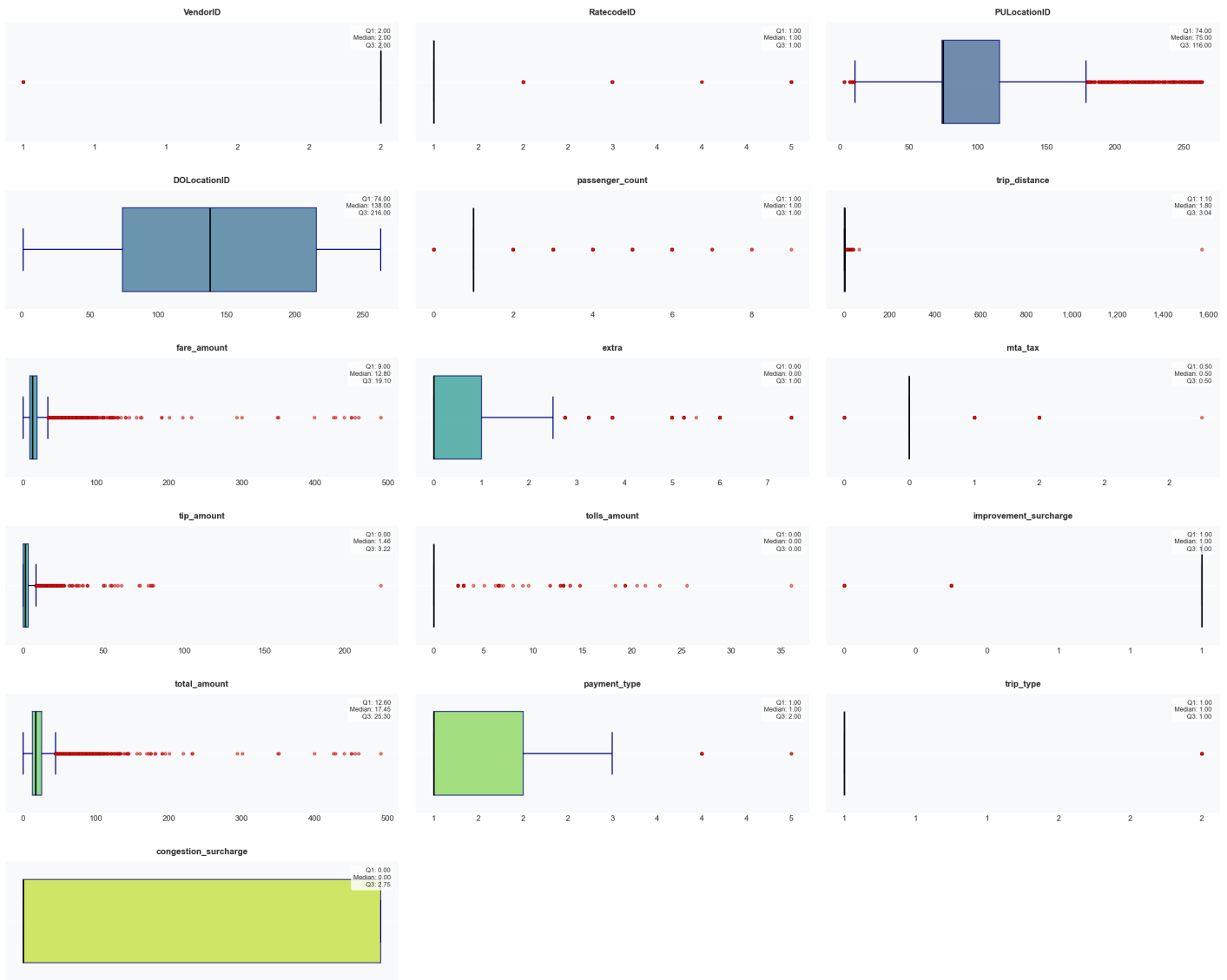
# Adjust layout
plt.tight_layout(w_pad=2, h_pad=3)

# Menambahkan watermark atau catatan
fig.text(0.99, 0.01, 'Data Source: NYC TLC January 2023',
    ha='right', va='bottom', fontsize=10, alpha=0.7)

plt.show()

```

Analisis Outlier Dataset NYC-TLC



Data Source: NYC TLC January 2023

Hampir setiap boxplot data diatas menunjukkan tidak berdistribusi normal, cenderung skewness kanan

Cek korelasi antar kolom

```
In [1209]:
df.columns
```

```
Out[1209]:
```

```
Index(['VendorID', 'lpep_pickup_datetime', 'lpep_dropoff_datetime',
      'store_and_fwd_flag', 'RatecodeID', 'PULocationID', 'DOLocationID',
      'passenger_count', 'trip_distance', 'fare_amount', 'extra', 'mta_tax',
      'tip_amount', 'tolls_amount', 'improvement_surcharge', 'total_amount',
      'payment_type', 'trip_type', 'congestion_surcharge', 'DO_Borough',
      'DO_Zone', 'DO_service_zone', 'PU_Borough', 'PU_Zone',
      'PU_service_zone'],
      dtype='object')
```

```
In [1210]:
```

```
korelasi = df[['VendorID', 'lpep_pickup_datetime', 'lpep_dropoff_datetime',
      'RatecodeID', 'PULocationID', 'DOLocationID',
      'passenger_count', 'trip_distance', 'fare_amount', 'extra', 'mta_tax',
      'tip_amount', 'tolls_amount', 'improvement_surcharge', 'total_amount',
      'payment_type', 'trip_type', 'congestion_surcharge']].corr()
```

```
In [1211]:
```

```
# plot berbentuk heatmap
```

```
# 1. Hitung matriks korelasi terlebih dahulu agar kode lebih bersih
```

```
corr_matrix = korelasi.corr(method="spearman")
```

```
# 2. Buat mask untuk menyembunyikan bagian segitiga atas (opsional tapi bagus)
```

```
mask_for_corr = np.triu(np.ones_like(corr_matrix, dtype=bool))
```

```
# 3. Setting ukuran figure (diperbesar sedikit agar angka tidak bertumpuk)
```

```
plt.figure(figsize=(12, 8))
```

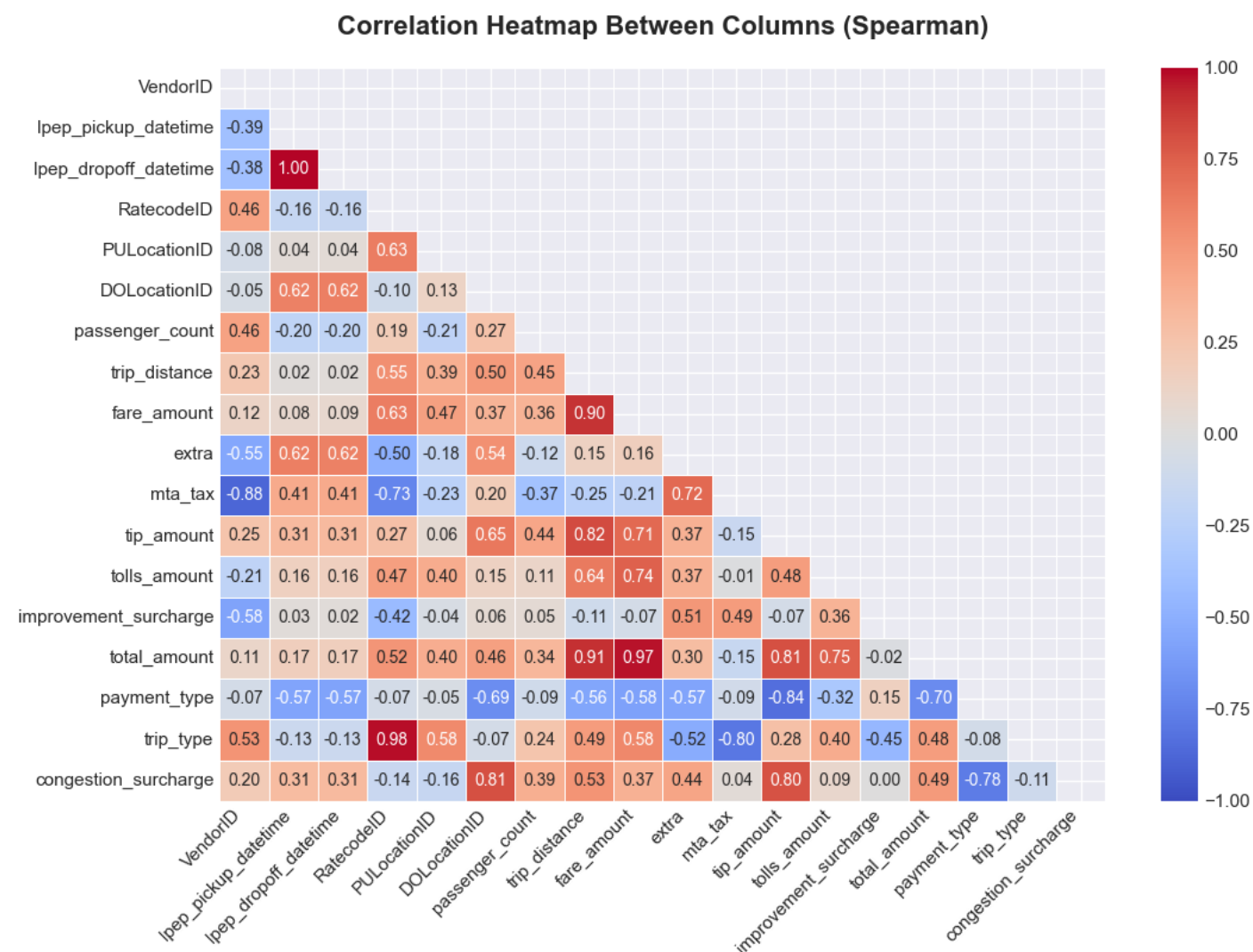
```
# 4. Gambar Heatmap
```

```
sns.heatmap(
    corr_matrix,
    annot=True,          # MENAMPILKAN ANGKA KORELASI
    fmt=".2f",           # FORMAT 2 ANGKA DI BELAKANG KOMA
    mask=mask_for_corr,
    vmin=-1, vmax=1,
    center=0,
    cmap='coolwarm',
    linewidths=.5,      # MEMBERI GARIS PEMISAH ANTAR KOTAK
    annot_kws={"size": 10} # MENGATUR UKURAN FONT ANGKA
)
```

```
plt.title("Correlation Heatmap Between Columns (Spearman)", fontsize=16, fontweight='bold', pad=20)
```

```
plt.xticks(rotation=45, ha='right') # Rotasi label X agar lebih rapi
```

```
plt.show()
```



Korelasi satu kolom dengan kolom yang lain:

- korelasi `lpep_pickup_datetime` dengan `lpep_dropoff_datetime` sebesar **1**, hubungannya sangat kuat secara positif karena saling berkaitan antara tempat penjemputan dan pengantaran
- korelasi `fare_amount` dengan `total_amount` sebesar **0.97**, hubungannya cukup kuat secara positif karena `total_amount` adalah total biaya yang dibayarkan penumpang termasuk `fare_amount` (jumlah tarif) yang didapat
- korelasi `trip_type` dengan `RatecodeID` sebesar **0.98**, hubungannya cukup kuat secara positif
- korelasi `trip_distance` dengan `total_amount` sebesar **0.91**, hubungannya cukup kuat secara positif bisa jadi karena total biaya yang dibayarkan penumpang bergantung dengan `trip_distance` (jarak yang ditempuh)
- korelasi `fare_amount` dengan `trip_distance` sebesar **0.90**, hubungannya cukup kuat secara positif bisa jadi karena `fare_amount` (jumlah tarif) yang didapat penumpang bergantung dengan `trip_distance` (jarak yang ditempuh)
- korelasi `tip_amount` dengan `trip_distance` sebesar **0.82**, hubungannya cukup kuat secara positif
- korelasi `congestion_surcharge` dengan `DOLocationID` sebesar **0.81**, hubungannya cukup kuat secara positif
- korelasi `tip_amount` dengan `total_amount` sebesar **0.81**, hubungannya cukup kuat secara positif
- korelasi `tip_amount` dengan `congestion_surcharge` sebesar **0.80**, hubungannya cukup kuat secara positif

- **Mengganti nilai dataframe ke data string sesuai keterangan**

In [1212]:

```
# kolom VendorID
```

```
df['VendorID'].replace([1, 2], ['Creative Mobile Technologies, LLC', 'VeriFone Inc'], inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\1186379579.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method({col: value}, inplace=True)` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
df['VendorID'].replace([1, 2], ['Creative Mobile Technologies, LLC', 'VeriFone Inc'], inplace=True)
```

In [1213]:

```
# kolom `RatecodeID`
```

```
df['RatecodeID'].replace([1, 2, 3, 4, 5, 6], ['Standard rate', 'JFK', 'Newark', 'Nassau or Westchester', 'Negotiated
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\3878098412.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method({col: value}, inplace=True)` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
df['RatecodeID'].replace([1, 2, 3, 4, 5, 6], ['Standard rate', 'JFK', 'Newark', 'Nassau or Westchester', 'Negotiated fare', 'Group ride'], inplace=True)
```

In [1214]:

```
# kolom `payment_type`
```

```
df['payment_type'].replace([1, 2, 3, 4, 5, 6], ['Credit card', 'Cash', 'No charge', 'Dispute', 'Unknown', 'Voided tri
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\629409971.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method({col: value}, inplace=True)` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
df['payment_type'].replace([1, 2, 3, 4, 5, 6], ['Credit card', 'Cash', 'No charge', 'Dispute', 'Unknown', 'Voided trip'], inplace=True)
```

In [1215]:

```
# kolom `trip_type`
```

```
df['trip_type'].replace([1, 2], ['Street-hail', 'Dispatch'], inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\3684183604.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['trip_type'].replace([1, 2], ['Street-hail', 'Dispatch'], inplace=True)
```

5. Data Analysis

Kita sudah melakukan tahap data cleaning. Sekarang, kita bisa mulai melakukan analisis untuk mencari tahu ---.

Analisis akan dilakukan dengan membandingkan data

a. Berdasarkan Korelasi

- **Tarif pajak by vendor taksi**

In [1216]:

```
pd.pivot_table(df, values='lpep_pickup_datetime', index=['VendorID', 'mta_tax'], aggfunc="count")
```

Out[1216]:

		lpep_pickup_datetime
	VendorID mta_tax	
Creative Mobile Technologies, LLC	0.00	62
	0.50	45
	1.00	25
	1.50	8934
	2.75	1
VeriFone Inc	0.00	1243
	0.50	52865

- **jumlah transaksi tiap RatecodeID by type trip**

In [1217]:

```
pd.pivot_table(data = df, values='lpep_pickup_datetime', index=['trip_type', 'RatecodeID'], aggfunc="count")
```

Out[1217]:

		lpep_pickup_datetime
trip_type	RatecodeID	
Dispatch	Negotiated fare	1140
	Standard rate	6
Street-hail	JFK	155
	Nassau or Westchester	3
	Negotiated fare	131
	Newark	24
	Standard rate	61716

Dapat dilihat penumpang yang memesan taksi dengan cara Street-hail atau dipanggil di jalan biasanya menggunakan tarif standar, sedangkan penumpang yang memesan via Dispatch atau palikasi biasanya menggunakan tarif negosiasi

- **Jumlah keseluruhan tip yang didapat by type pembayaran**

In [1218]:

```
pd.pivot_table(df, values='tip_amount', index=['payment_type'], aggfunc="sum").sort_values(by=['tip_amo
```

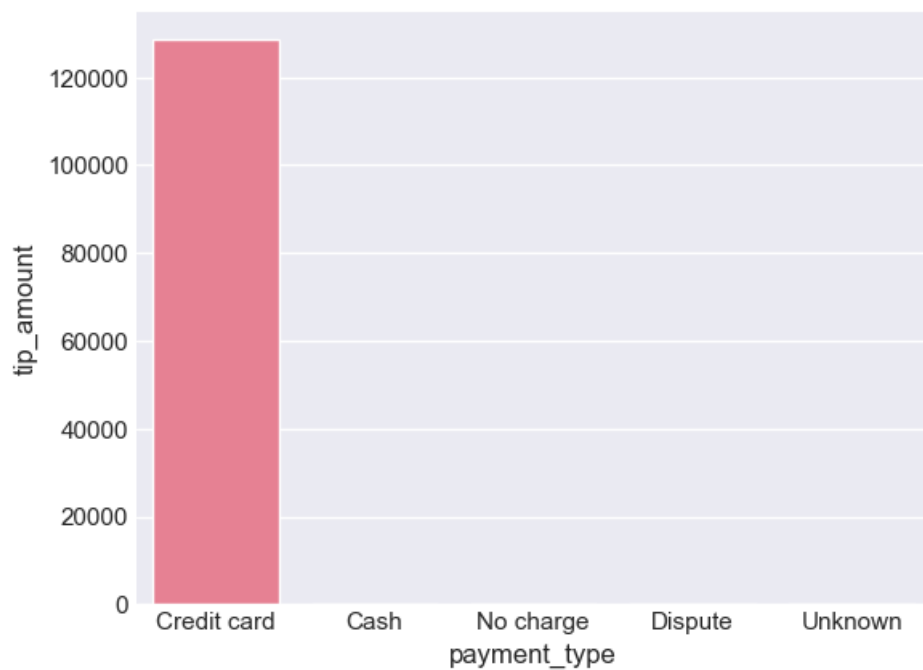
Out[1218]:

	tip_amount
payment_type	
Credit card	128559.33
Dispute	21.00
No charge	9.30
Cash	5.00
Unknown	0.00

In [1219]:

```
sns.barplot(data =df,
            x= "payment_type",
            y = "tip_amount",
            hue= "payment_type",
            estimator= "sum",
            errorbar= None)
```

plt.show()



- **Banyaknya tip yang didapat by type pembayaran**

In [1220]:

```
pd.pivot_table(df, values='tip_amount', index=['payment_type'], aggfunc="count").sort_values(by=['tip_amo
```

Out[1220]:

	tip_amount
payment_type	
Credit card	40247
Cash	22374
No charge	460
Dispute	92
Unknown	2

Data diatas menunjukkan banyaknya tip yang didapat berdasarkan type pembayaran adalah kartu kredit, untuk kesejahteraan supir bisa diadakan promosi yang menarik bagi pengguna kartu kredit agar mayoritas penumpang menggunakan kartu kredit.

b. Berdasarkan data perjalanan

- Daerah macet

In [1221]:

Daerah macet adalah daerah yang memiliki biaya tambahan congestion_surcharge == 2.75

```
zona_macet = pd.pivot_table(df, values='lpep_pickup_datetime', index=['congestion_surcharge', 'DO_Zone'],  
zona_macet
```

Out[1221]:

		lpep_pickup_datetime
congestion_surcharge	DO_Zone	
0.00	Allerton/Pelham Gardens	11
	Alphabet City	2
	Astoria	1102
	Astoria Park	3
	Auburndale	26
...	...	...
2.75	Williamsburg (South Side)	8
	Woodside	2
	World Trade Center	34
	Yorkville East	848
	Yorkville West	1410

364 rows × 1 columns

In [1222]:

Top 10 daerah termacet berdasarkan jumlah transaksi

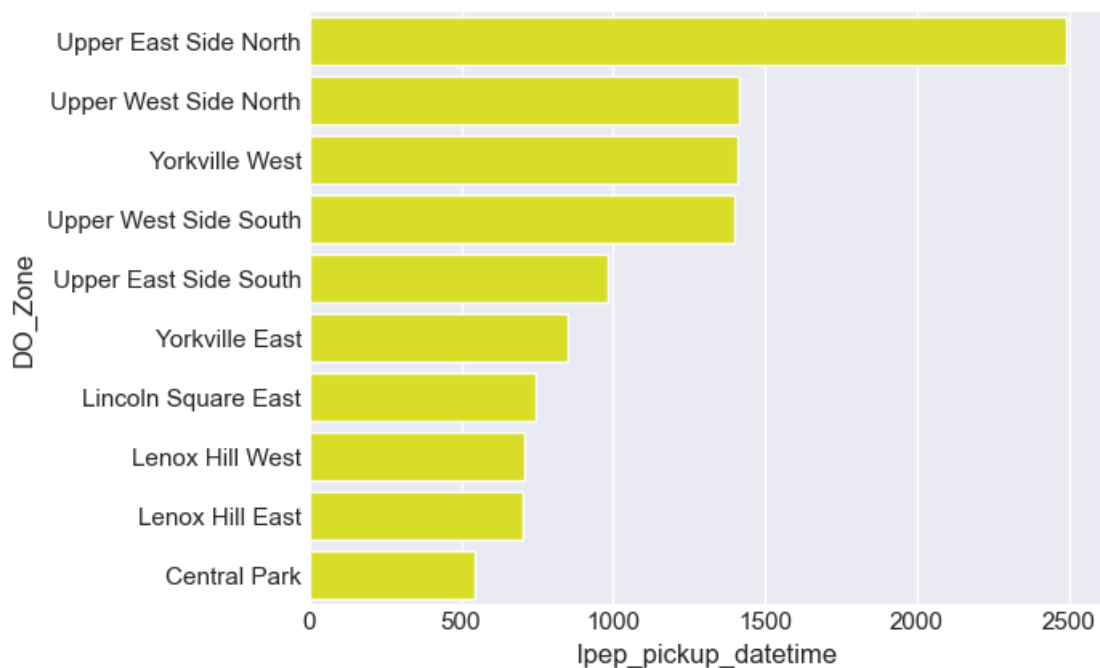
```
macet = zona_macet.loc[2.75].reset_index().sort_values(by=['lpep_pickup_datetime'], ascending=False).head  
macet
```

Out[1222]:

	DO_Zone	lpep_pickup_datetime
112	Upper East Side North	2491
114	Upper West Side North	1416
127	Yorkville West	1410
115	Upper West Side South	1397
113	Upper East Side South	980
126	Yorkville East	848
64	Lincoln Square East	743
63	Lenox Hill West	706
62	Lenox Hill East	701
20	Central Park	546

In []:

```
sns.barplot(data = macet,  
            x= "lpep_pickup_datetime",  
            y = "DO_Zone",  
            errorbar= None,  
            color= "#F5FD08DD" )  
plt.show()
```



Data diatas adalah 10 daerah drop off yang sering macet, sehingga penumpang diharuskan membayar lebih sebesar \$2.75

Dataset baru df_kordinat

In [1224]:

```
# pip install geopandas
```

In [1225]:

```
import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
```

```
# 1. Load Shapefile Zona Taxi NYC (Format GeoJSON atau Shapefile)
# Ini adalah file resmi yang memuat bentuk peta setiap LocationID
url_shapefile = "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zones.zip"
```

```
print("Sedang mendownload dan membaca data peta...")
gdf_zones = gpd.read_file(url_shapefile)
```

```
# Lihat isi datanya
# Kolom 'geometry' berisi bentuk poligon daerah tersebut
print(gdf_zones.head())
```

```
# 2. Mengubah Poligon menjadi Titik Koordinat (Centroid)
# Kita cari titik tengah dari setiap wilayah agar dapat Latitude & Longitude
gdf_zones['centroid'] = gdf_zones.geometry.centroid
gdf_zones['longitude'] = gdf_zones['centroid'].x
gdf_zones['latitude'] = gdf_zones['centroid'].y
```

```
# 3. Merapikan DataFrame
# Kita hanya butuh ID, Nama Zona, dan Koordinatnya
df_koordinat = gdf_zones[['LocationID', 'zone', 'borough', 'latitude', 'longitude']].copy()
```

```
print("\nData Koordinat Berhasil Dibuat:")
print(df_koordinat.head())
```

```
# (Opsional) Visualisasi Peta untuk memastikan data benar
gdf_zones.plot(column='borough', legend=True, figsize=(10, 10))
plt.title("Peta Zona Taxi NYC")
plt.show()
```

Sedang mendownload dan membaca data peta...

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID \
0	1	0.116357	0.000782	Newark Airport	1
1	2	0.433470	0.004866	Jamaica Bay	2
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3
3	4	0.043567	0.000112	Alphabet City	4
4	5	0.092146	0.000498	Arden Heights	5

	borough \
0	EWB
1	Queens
2	Bronx
3	Manhattan
4	Staten Island

geometry

```
0 POLYGON ((933100.918 192536.086, 933091.011 192572.175, 933088.585 192604.97, 933121.56 192857.382, 933149.268 19297
8.645, 933222.304 193293.273, 933252.582 193450.6, 933297.723 193555.433, 933330.571 193748.823, 933350.632 193794.687,
933355.706 193834.024, 933365.798 193886.466, 933431.739 194391.291, 933465.038 194801.093, 933463.434 195227.363, 933
471.105 195315.879, 933483.754 195397.827, 933511.436 195509.255, 933554.091 195620.651, 933609.194 195718.905, 933669.
247 195797.475, 933731.775 195866.204, 933779.276 195908.732, 933841.762 195957.786, 933906.723 195996.999, 933976.674
196036.202, 934119.044 196101.488, 934491.267 196304.019, 934656.105 196375.819, 934810.948 196441.083, 934963.289 196
503.074, 935150.582 196574.832, 935178.064 196591.171, 935205.551 196610.79, 935273.09 196692.628, 935335.676
196794.15, 935373.251 196866.212, 935430.8 196944.791, 935455.812 196974.252, 935483.312 197000.429, 935548.274 197042.
925, 935625.692 197075.56, 935688.106 197091.83, 935797.956 197121.121, 935960.198 197147.03, 936040.092 197169.824, 93
6154.885 197176.154, 936354.543 197195.433, 936594.162 197234.31, 936664.06 197250.568, 936746.409 197253.685, 936783.8
31 197250.333, 937133.175 197256.211, 937427.587 197242.526, 938001.432 197211.915, 938662.625 197187.712, 938777.393 1
97180.939, 939241.47 197160.401, 939663.11 197130.112, 939730.458 197116.873, 939797.805 197103.633, 939872.608 197073.
986, 940059.603 196991.671, 940154.345 196948.872, 940189.255 196935.692, 940368.759 196850.114, 941388.536 196402.352,
941525.645 196326.694, 941548.063 196303.702, 941585.458 196283.962, 941617.949 196313.416, 941765.07 196254.137, 9417
80.088 196280.342, 941810.009 196267.174, 941664.699 195936.252, 941589.498 195742.924, 941233.68 194910.693, 941148.53
1 194740.337, 941063.411 194586.376, 940993.274 194438.947, 940895.621 194255.499, 940590.076 193649.436, 940565.02 193
593.739, 940552.459 193547.856, 940542.388 193498.69, 940354.577 193144.899, 940016.459 192483.16, 939996.41
192437.291, 939841.237 192198.21, 939720.853 191877.091, 939149.831 190809.2, 939102.232 190714.199, 939072.178 190658.
512, 939067.186 190658.521, 939059.698 190658.535, 939027.264 190665.153, 939019.801 190678.283, 938984.895 190698.022,
938959.942 190701.348, 938932.493 190704.678, 938900.053 190708.017, 938822.704 190721.278, 938740.364 190734.548, 938
720.397 190734.585, 938705.403 190724.776, 938667.953 190718.289, 938627.981 190698.69, 938582.987 190662.706, 938547.9
58 190616.866, 938515.394 190554.627, 938492.795 190482.532, 938412.598 190308.898, 938372.558 190253.231, 938357.495 1
90207.354, 938322.409 190132.004, 938277.209 189987.815, 938247.152 189932.129, 938259.427 189823.9, 938261.861 189791.
```

106, 938266.623 189669.775, 938261.55 189627.158, 938241.446 189555.059, 938198.815 189450.212, 938101.102 189256.939, 938071.056 189207.811, 938005.884 189063.661, 937945.797 188968.685, 937825.509 188719.713, 937707.772 188500.248, 937 680.208 188444.558, 937642.634 188375.772, 937610.096 188329.929, 937592.535 188284.057, 937519.9 188156.317, 937414.63 8 187936.829, 937331.986 187792.714, 937294.39 187714.091, 937244.308 187632.214, 937149.07 187435.66, 937111.525 18738 3.27, 937083.92 187307.907, 936983.707 187121.201, 936956.133 187062.233, 936910.957 186937.72, 936838.309 186806.703, 9 36695.436 186508.597, 936675.401 186475.846, 936650.348 186429.99, 936560.171 186272.776, 936540.136 186240.026, 93630 9.63 185814.214, 936221.99 185679.95, 936199.438 185637.368, 936181.918 185614.45, 936099.321 185506.407, 935919.011 18 5221.496, 935881.479 185178.944, 935748.806 184998.865, 935706.255 184943.208, 935598.604 184792.591, 935415.838 18453 3.919, 935343.234 184432.417, 935313.17 184380.015, 935253.088 184298.162, 935210.488 184219.552, 935157.881 184131.127, 935125.313 184075.45, 935055.243 183993.618, 934997.721 183944.55, 934932.73 183905.334, 934865.242 183866. 124, 934697.843 183804.164, 934627.911 183791.191, 934560.496 183788.05, 934503.087 183794.725, 934375.868 183847.449, 934323.48 183867.231, 934288.568 183886.976, 934233.67 183900.205, 933946.867 184048.35, 933724.916 184166.854, 933251. 043 184394.092, 933106.444 184489.486, 933046.617 184532.238, 932954.432 184620.965, 932922.046 184653.823, 932894.661 184689.949, 932800.14 184854.097, 932670.907 185129.804, 932462.077 185540.119, 932288.071 185888.061, 932193.636 1860 88.28, 932116.502 186206.488, 932049.34 186318.117, 931959.909 186521.606, 931890.308 186659.472, 931813.298 186833.423, 931728.757 186987.717, 931691.575 187109.12, 931634.715 187365.002, 931615.015 187489.646, 931612.568 1875 12.604, 931597.683 187555.262, 931575.395 187637.285, 931560.624 187732.407, 931553.491 187896.371, 931553.79 188034.08 7, 931561.534 188152.113, 931556.72 188234.098, 931566.974 188358.676, 931572.116 188427.523, 931589.88 188561.923, 931615.424 188830.743, 931620.587 188909.427, 931658.873 189299.541, 931679.465 189588.046, 931689.691 189699.509, 931 702.398 189804.408, 931725.053 189892.892, 931725.131 189928.96, 931720.224 189968.318, 931722.84 190024.055, 931737.95 1 190086.323, 931773.178 190217.406, 931838.405 190371.377, 931918.585 190515.478, 932171.257 190787.09, 932326.317 190 934.312, 932566.381 191150.212, 932693.872 191245.031, 932771.396 191317.004, 933100.918 192536.086))

1 MULTIPOLYGON (((1033269.244 172126.008, 1033439.643 170883.946, 1033473.265 170808.208, 1033504.66 170731.521, 1033 533.803 170653.949, 1033560.667 170575.56, 1033585.23 170496.419, 1033607.472 170416.594, 1033627.372 170336.154, 1033 644.914 170255.167, 1033660.084 170173.702, 1033672.867 170091.829, 1033683.254 170009.618, 1033691.235 169927.138, 10 33696.804 169844.46, 1033699.956 169761.655, 1033700.687 169678.793, 1033698.999 169595.945, 1033914.712 168756.095, 1 033987.383 168631.157, 1034049.614 168461.429, 1034152.071 168286.528, 1034190.853 168220.321, 1034215.168 168143.684, 1034239.479 168067.044, 1034288.511 168020.384, 1034359.472 167990.046, 1034780.204 167800.349, 1034768.641 167571.082, 1034854.96 167383.735, 1035019.891 167173.945, 1035295.456 166890.275, 1035303.561 166784.912, 1035149.634 166644.979, 1035125.254 166622.815, 1034785.622 166702.902, 1034518.71 166336.924, 1034525.495 166168.944, 1034703.802 165974.427, 1034833.479 165917.694, 1034833.479 165804.225, 1035060.415 165755.596, 1035076.625 165893.379, 1035181.98 8 165942.008, 1035408.924 165755.596, 1035448.851 165480.719, 1035488.522 165255.535, 1035222.512 165074.789, 1035102.359 165016.103, 1035003.681 164945.111, 1034930.738 164815.434, 1034930.738 164523.66, 1034875.409 164341.763, 1034840.183 164301.313, 1034845.058 164230.654, 1034888.135 164112.951, 1034912.404 163898.988, 1035009.638 163697.64, 1035023.814 163493.28, 1035188.463 163371.508, 1034866.459 163365.731, 1034013.272 163473.152, 1033830.974 163407.96, 1033788.164 163354.655, 1033769.142 163340.744, 1033751.724 163323.795, 1033733.304 163302.65, 1033572.364 163117.891, 1033171.666 162340.845, 1033164.852 162111.03, 1033123.915 162264.997, 1033105.796 162341.098, 1033087.676 162409.952, 1033047.814 162478.805, 1033026.071 162507.796, 1033007.952 162493.301, 1033015.199 162453.438, 1032991.116 162417.612, 1032953.593 162438.943, 1032928.226 162446.191, 1032910.107 162493.301, 1032946.346 162623.76, 1032924.603 162685.366, 1032862.997 162754.219, 1032779.648 162837.568, 1032765.152 162935.413, 1032725.29 162978.899, 1032681.803 163069.496, 1032627.445 163091.239, 1032576.711 163080.367, 1032533.225 163051.376, 1032504.234 163022.385, 1032504.23 4 162982.523, 1032460.965 162943.019, 1032384.647 162928.165, 1032356.198 162898.655, 1032271.221 162800.135, 1032239.692 162794.082, 1032210.701 162794.082, 1032178.086 162808.577, 1032150.286 162857.228, 1032149.095 162859.311, 1032120.104 162899.174, 1032095.116 162883.272, 1032080.242 162873.807, 1032038.644 162909.932, 1032087.48 9 162975.275, 1032120.104 162997.018, 1032192.582 163018.761, 1032214.325 163044.129, 1032141.848 163094.863, 1032076.618 163105.734, 1031982.397 163094.863, 1031877.305 163098.486, 1031725.103 163149.722, 1031634.506 163207.202, 1031547.533 163254.313, 1031420.698 163312.295, 1031296.338 163358.586, 1031192.395 163384.772, 1031036.569 163384.772, 1030974.963 163366.652, 1030884.366 163344.91, 1030764.778 163276.056, 1030528.901 163196.901, 1030332.46 163155.67, 1030322.467 163148.628, 1030319.66 163146.65, 1030308.34 163147.71, 1030288.16 163148.6, 1030242.85 163149.7 7, 1030210.13 163159.55, 1030170.2 163167.29, 1030136.67 163180.57, 1030103.74 163186.85, 1030087.26 163186.9, 1030070.5 8 163188.6, 1030056.6 163195.65, 1030048.38 163201.85, 1030041.59 163205.78, 1030026.37 163211.39, 1030012.38 163215.35, 1030002.7 163211.45, 1029989.15 163216.44, 1029970.31 163224.03, 1029951.94 163233.44, 1029942.5 163238.47, 1029933.71 163246.47, 1029929.51 163255.5, 1029926.93 163268.66, 1029932.64 163276.12, 1029937.7 163282.26, 1029940.76 163291.02, 1 029941.86 163297.46, 1029941.03 163298.27, 1029930.35 163334.27, 1029916.28 163361.67, 1029896.11 163390.84, 1029876.95 163416.22, 1029851.88 163439.58, 1029839.29 163447.38, 1029821.92 163456.31, 1029808.37 163460.48, 1029785.76 163464.58 , 1029768.96 163463.78, 1029753.63 163464.31, 1029739.83 163459.59, 1029725.87 163453.71, 1029720.32 163447.4, 1029720.5 2 163443.61, 1029723.19 163439.02, 1029739.24 163430.75, 1029742.25 163425.99, 1029740.79 163423.01, 1029729.11 163420. 94, 1029717.53 163425.79, 1029711.07 163430.19, 1029706.13 163430.15, 1029699.56 163427.62, 1029693.16 163424.06, 102968 6.41 163423.39, 1029686.69 163429, 1029698.15 163438.15, 1029710.94 163445.84, 1029710.75 163449.63, 10297120.5 163454. 07, 1029702.88 163464.56, 1029692.27 163473.04, 1029675.72 163481.97, 1029664.98 163485.17, 1029651.12 163487.86, 10296 40.74 163487.61, 1029629.55 163484.88, 1029619.38 163479.2, 1029613.54 163468.11, 1029614.47 163455.76, 1029617.52 1634 45.24, 1029613.48 163436.14, 1029605.26 163432.78, 1029594.86 163435.82, 1029576.52 163441.94, 1029562.24 163455.5, 1029 546.29 163470.69, 1029542.11 163478.07, 1029534.81 163484.11, 1029526.39 163486.18, 1029510.08 163486.38, 1029496.06 16 3487.91, 1029484.51 163489.46, 1029470.16 163491.98, 1029454.97 163495.32, 1029444.37 163502.15, 1029435.93 163506.69, 1 029430.17 163505.82, 1029431.17 163503.69, 1029426.93 163499.21, 1029421.63 163502.62, 1029411.69 163509.13, 1029401.99 163526.19, 1029396.03 163529.93, 1029386.79 163531.17, 1029374.27 163530.74, 1029363.02 163536.25, 1029356.71 163542.63 , 1029344.75 163553.4, 1029334.34 163558.1, 1029308.1 163562.33, 1029298.2 163564.72, 1029291.07 163569.44, 1029285.91 1 63575.66, 1029284.2 163583.06, 1029285.24 163597.73, 1029294.17 163613.46, 1029302.59 163631.49, 1029306.35 163635.81, 1 029315.21 163640.49, 1029331.13 163647.87, 1029366.17 163654.92, 1029386.42 163656.89, 1029420.65 163661.46, 1029434.45 163667.01, 1029443.12 163674.66, 1029455.69 163689.43, 1029490.02 163741.94, 1029499.94 163758.83, 1029516.04 163785, 1 029528.61 163798.94, 1029536.33 163802.14, 1029553.49 163799.48, 1029573.65 163792.06, 1029611.77 163785.29, 1029625.61 163785.07, 1029633.992 163788.374, 1029652.01 163801.27, 1029658.97 163806.27, 1029687.26 163833.03, 1029722.71 163869. 41, 1029739.46 163887.26, 1029797.16 163943.1, 1029851.6 164006.08, 1029894.86 164065.5, 1029907.519 164084.152, 1029938.33 164129.55, 1029982.09 164199.19, 1030002.45 164237.42, 1030014.45 164261.7, 1030028.48 164290.1, 1030039.12 164311.63, 1030049.29 164336.811, 1030062.81 164379.14, 1030068.22 164393.57, 1030070.42 164413, 1030073.53 164431.98, 1030071.42 164443.85, 1030068.64 164452.06, 1030067.048 164458.676, 1030066.77 164459.83, 1030063.56 164462.55, 103005

3.98 164459.31, 1030043.48 164457.66, 1030034.78 164459.68, 1030026.77 164463.07, 1030016.68 164471.48, 1030011.84 164478.99, 1030010.66 164486.99, 1030013.36 164497.05, 1030016.31 164500.72, 1030021.78 164503.72, 1030027.27 164502.14, 1030034.13 164502.18, 1030042.57 164504.27, 1030048.7 164513.21, 1030052.31 164522.83, 1030056.36 164536.78, 1030056.29 164551.64, 1030060.32 164571.31, 1030066.18 164587.79, 1030064.7 164612.46, 1030067.98 164644.46, 1030068.1 164667.77, 1030063.88 164689.23, 1030047.79 164709.72, 1030032.62 164729.76, 1030023.21 164739.54, 1030015.42 164742.25, 1030010.63 164741.77, 1030000 164731.207, 1029997.99 164731.4, 1029985.47 164728.59, 1029969.65 164727.43, 1029961.74 164729.74, 1029958.12 164733.2, 1029956.63 164740.12, 1029962.56 164747.04, 1029965.36 164755.94, 1029963.87 164762.86, 1029958.1 164775.21, 1029949.04 164785.92, 1029941.95 164791.52, 1029936.18 164797.29, 1029928.93 164810.79, 1029920.35 164839.79, 1029917.04 164877.69, 1029919.66 164911.8, 1029919.32 164937.5, 1029919.32 164947.39, 1029917.01 164952.66, 1029912.06 164975.72, 1029903.98 164993.35, 1029897.744 165000, 1029887.58 165009.96, 1029873.44 165021.54, 1029858.18 165033.28, 1029854.22 165038.1, 1029849.64 165047.22, 1029849.08 165060.32, 1029854.94 165078.96, 1029858.05 165092.2, 1029860.03 165103.2, 1029871.11 165114, 1029876.12 165124.66, 1029879.03 165131.19, 1029888.02 165139.44, 1029900.06 165149.75, 1029904.44 165162.34, 1029907.2 165170.15, 1029908.21 165179.24, 1029914 165186.71, 1029919.12 165187.32, 1029949.9 165180.91, 1029968.25 165179.21, 1029982.01 165185.04, 1029992.56 165186.89, 1029998.29 165183.98, 1030001.79 165181.41, 1030015.62 165165.65, 1030023.62 165162.33, 1030033.4 165162.67, 1030042.51 165161.64, 1030052.77 165159.49, 1030058.52 165153.87, 1030067.938 165143.711, 1030068.49 165137.01, 1030068.59 165135.77, 1030068.69 165134.5, 1030068.8 165133.19, 1030068.91 165131.82, 1030069.02 165130.43, 1030069.14 165129.01, 1030069.26 165127.58, 1030069.37 165126.18, 1030069.48 165124.83, 1030069.494 165124.728, 1030069.39 165123.49, 1030069.29 165122.17, 1030069.2 165120.89, 1030069.183 165120.683, 1030068.95 165119.61, 1030068.6 165117.95, 1030068.57 165117.81, 1030068.25 165116.326, 1030068.15 165116.07, 1030068.09 165115.91, 1030067.47 165114.3, 1030067.4 165114.12, 1030066.79 165112.53, 1030066.694 165112.28, 1030066.06 165111.01, 1030065.32 165109.52, 1030064.58 165108.04, 1030064.515 165107.924, 1030064.49 165107.88, 1030063.48 165106.47, 1030063.37 165106.33, 1030062.36 165104.92, 1030062.26 165104.78, 1030061.23 165103.36, 1030061.14 165103.23, 1030060.11 165101.8, 1030060.02 165101.68, 1030059.92 165101.55, 1030059.83 165101.42, 1030058.914 165100.144, 1030058.81 165100.02, 1030058.71 165099.9, 1030057.59 165098.53, 1030057.5 165098.42, 1030056.38 165097.05, 1030056.3 165096.94, 1030055.17 165095.56, 1030055.09 165095.46, 1030053.95 165094.06, 1030053.67 165093.72, 1030053.6 165093.64, 1030052.47 165092.25, 1030052.35 165092.1, 1030051.22 165090.72, 1030048 165086.77, 1030047.28 165085.9, 1030046.54 165084.99, 1030045.65 165083.89, 1030044.75 165082.79, 1030043.83 165081.67, 1030043.665 165081.473, 1030042.93 165080.49, 1030042.04 165079.29, 1030041.798 165078.983, 1030040.99 165078.26, 1030039.8 165077.2, 1030038.997 165076.493, 1030038.49 165076.03, 1030037.03 165074.72, 1030035.32 165073.18, 1030035.2 165073.07, 1030033.63 165071.66, 1030032 165070.19, 1030030.41 165068.76, 1030028.74 165067.26, 1030027.12 165065.8, 1030026.55 165065.29, 1030025.69 165064.33, 1030024.37 165062.87, 1030023.749 165062.178, 1030023.2 165061.39, 1030022.22 165060, 1030021.571 165059.066, 1030021.36 165058.59, 1030020.72 165057.08, 1030020.07 165055.57, 1030019.43 165054.08, 1030018.79 165052.58, 1030017.837 165050.353, 1030016.46 165046.5, 1030016.28 165045.996, 1030016.11 165044.96, 1030015.86 165043.47, 1030015.658 165042.262, 1030015.61 165041.97, 1030015.36 165040.47, 1030015.11 165038.97, 1030015.035 165038.528, 1030015.13 165037.52, 1030015.26 165036.07, 1030015.34 165035.105, 1030015.45 165034.59, 1030015.75 165033.09, 1030015.969 165031.993, 1030016.08 165031.53, 1030016.28 165030.748, 1030016.62 165029.9, 1030016.903 165029.192, 1030017.48 165028.18, 1030018.147 165027.014, 1030018.62 165026.35, 1030019.2 165025.54, 1030019.703 165024.835, 1030020.15 165024.32, 1030021.571 165022.657, 1030021.85 165022.44, 1030023.126 165021.412, 1030024 165020.83, 1030024.994 165020.167, 1030025.616 165019.856, 1030026.45 165019.57, 1030026.55 165019.545, 1030027.483 165019.234, 1030028.106 165019.234, 1030029.039 165019.234, 1030029.973 165019.234, 1030031.53 165019.88, 1030033.69 165020.78, 1030033.707 165020.79, 1030035.3 165021.46, 1030036.55 165021.98, 1030037.441 165022.346, 1030037.52 165022.4, 1030038.14 165022.8, 1030038.69 165023.14, 1030039.17 165023.45, 1030039.59 165023.72, 1030040.864 165024.524, 1030041.29 165024.9, 1030041.59 165025.16, 1030041.87 165025.4, 1030042.12 165025.63, 1030042.36 165025.83, 1030042.58 165026.02, 1030043.354 165026.702, 1030047.088 165031.059, 1030051.49 165036.8, 1030051.68 165037.04, 1030051.86 165037.28, 1030052.03 165037.5, 1030052.35 165037.92, 1030052.51 165038.121, 1030052.66 165038.32, 1030052.81 165038.51, 1030052.95 165038.7, 1030053.25 165039.08, 1030053.4 165039.28, 1030054.246 165040.395, 1030054.3 165040.45, 1030055.23 165041.57, 1030057.23 165043.98, 1030057.358 165044.129, 1030058.3 165044.98, 1030059.37 165045.96, 1030060.44 165046.93, 1030061.49 165047.89, 1030062.54 165048.84, 1030063.72 165049.91, 1030064.2 165050.353, 1030064.94 165050.91, 1030066.21 165051.88, 1030067.5 165052.85, 1030068.8 165053.84, 1030070.11 165054.83, 1030071.43 165055.84, 1030072.66 165056.77, 1030073.229 165057.199, 1030073.851 165057.511, 1030074.07 165057.58, 1030074.785 165057.822, 1030075.407 165058.133, 1030076.04 165058.13, 1030076.34 165058.133, 1030079.66 165057.97, 1030082.876 165057.822, 1030083.39 165057.99, 1030083.809 165058.133, 1030083.83 165058.13, 1030083.85 165058.13, 1030083.8 165058.13, 1030083.95 165058.13, 1030084.743 165058.133, 1030085.09 165058.24, 1030085.676 165058.444, 1030085.988 165058.444, 1030086.299 165058.755, 1030086.47 165058.76, 1030086.61 165058.755, 1030087.232 165059.066, 1030087.232 165059.378, 1030087.543 165059.689, 1030087.62 165059.82, 1030087.89 165060.25, 1030087.96 165060.36, 1030087.97 165060.38, 1030087.98 165060.39, 1030087.99 165060.41, 1030088.97 165061.96, 1030089.099 165062.178, 1030089.79 165063.56, 1030089.79 165063.57, 1030090.345 165064.668, 1030090.55 165065.2, 1030090.56 165065.22, 1030091.22 165066.87, 1030091.23 165066.87, 1030091.589 165067.78, 1030091.62 165067.93, 1030091.67 165068.18, 1030092.12 165070.42, 1030092.12 165070.45, 1030092.212 165070.892, 1030092.4 165072.27, 1030093.145 165077.738, 1030093.07 165079.43, 1030093.07 165079.53, 1030092.99 165081.38, 1030092.98 165081.5, 1030092.91 165083.34, 1030092.9 165083.48, 1030092.834 165085.207, 1030092.82 165085.3, 1030092.81 165085.46, 1030092.66 165087.28, 1030092.64 165087.47, 1030092.522 165088.941, 1030092.44 165089.3, 1030092.01 165091.16, 1030091.65 165092.74, 1030091.58 165093.01, 1030091.23 165094.56, 1030091.16 165094.87, 1030090.81 165096.39, 1030090.73 165096.74, 1030090.655 165097.032, 1030090.58 165098.17, 1030090.55 165098.53, 1030090.45 165099.94, 1030090.42 165100.32, 1030090.39 165100.72, 1030090.36 165101.16, 1030090.345 165101.389, 1030090.34 165101.61, 1030090.34 165103.31, 1030090.34 165103.81, 1030090.345 165104.501, 1030090.44 165105.41, 1030090.5 165105.92, 1030090.655 165107.301, 1030090.7 165107.43, 1030091.278 165109.48, 1030091.9 165110.413, 1030092.522 165111.347, 1030094.39 165113.525, 1030096.879 165115.704, 1030097.11 165115.85, 1030097.57 165116.16, 1030098.05 165116.48, 1030098.66 165116.89, 1030099.28 165117.3, 1030099.89 165117.71, 1030100.4 165118.05, 1030100.9 165118.38, 1030101.4 165118.71, 1030101.88 165119.03, 1030102.98 165119.77, 1030103.65 165120.21, 1030104.34 165120.68, 1030105.06 165121.16, 1030105.82 165121.66, 1030106.215 165121.928, 1030106.55 165122.2, 1030107.24 165122.76, 1030109.638 165124.728, 1030110.49 165125.58, 1030111.43 165126.52, 1030111.816 165126.907, 1030112.28 165127.52, 1030113.684 165129.397, 1030116.85 165135.47, 1030117.418 165136.554, 1030117.76 165137.32, 1030118.62 165139.27, 1030118.663 165139.354, 1030118.99 165140.98, 1030119.285 165142.466, 1030119.31 165142.71, 1030119.596 165145.578, 1030119.596 165149.313, 1030118.68 165156.95, 1030118.663 165157.092, 1030118.28 165158.58, 1030116.88 165163.97, 1030116.59 165165.13, 1030116.28 165166.3, 1030116.173 165166.739, 1030114.67 165170.64, 1030114.617 165170.785, 1030114.22 165171.64, 1030113.74 165172.68, 1030113.24 165173.77, 1030112.75 165174.83, 1030112.7 165174.9, 1030111.94 165176.04, 1030111.16 165177.2, 1030110.38 165178.39, 1030110.261 165178.565, 1030109.52 165179.55, 1030108.6 165180.77, 1030107.65 165182.04, 1030107.46 165182.299, 1030106.51 165183.25, 1030105.33 165184.43, 1030104.14 165185.621, 1030104.037 165185.722, 1030102.88 165186.69, 1030101.62 1

65187.74, 1030100.38 165188.76, 1030100.302 165188.834, 1030099.16 165189.78, 1030097.96 165190.78, 1030096.78 165191.7
7, 1030096.568 165191.946, 1030095.61 165192.54, 1030094.48 165193.23, 1030093.41 165193.9, 1030092.522 165194.436, 103
0089.78 165196, 1030088.29 165196.85, 1030086.83 165197.69, 1030086.7 165197.76, 1030086.56 165197.84, 1030085.12 16519
8.66, 1030083.809 165199.415, 1030082.35 165200.18, 1030081 165200.88, 1030079.66 165201.59, 1030078.32 165202.29, 1030
077.274 165202.838, 1030076.98 165202.94, 1030075.62 165203.42, 1030071.48 165204.871, 1030071.05 165205.016, 1030069.9
8 165205.26, 1030069.7 165205.32, 1030068.19 165205.65, 1030065.449 165206.261, 1030063.7 165206.42, 1030062.06
165206.57, 1030060.44 165206.71, 1030058.87 165206.86, 1030057.33 165207, 1030046.06 165213.26, 1030038.99 165214.99, 1
030025.24 165223.93, 1030014.66 165233.59, 1030003.6 165245.74, 1029997.08 165249.45, 1029987.51 165251.1, 1029977.45 1
65251, 1029973.14 165251.99, 1029969.49 165255.04, 1029971.42 165258.06, 1029984.73 165267.73, 1029992.76 165274.86, 10
30006.23 165284.84, 1030025.19 165297.12, 1030039.37 165310.02, 1030050.58 165323.8, 1030062.42 165342.36, 1030071.76 1
65360.44, 1030091.521 165474.622, 1030059.08 165570.69, 1030062.23 165573.68, 1030074.24 165577.46, 1030087.63 165579.4
4, 1030099.31 165591.86, 1030110.45 165612.46, 1030112.52 165629.54, 1030118.93 165645.08, 1030124.42 165662.2, 1030118.
6 165673.51, 1030109.14 165685.45, 1030094.4 165700.52, 1030089.75 165709.56, 1030091.72 165715.95, 1030094.06 165730.0
8, 1030096 165738.75, 1030093.55 165753.5, 1030093.72 165758.51, 1030099.56 165764.72, 1030107.25 165768.9, 1030115.38 1
65774, 1030124.6 165782.52, 1030123.31 165796.16, 1030117.92 165809.51, 1030109.56 165823.74, 1030109.39 165838.53, 103
0112.45 165849.48, 1030123.63 165865.53, 1030136.45 165877.96, 1030147.16 165895.37, 1030152.4 165914.31, 1030153.35 16
5930.25, 1030147.67 165948.83, 1030140.25 165961.48, 1030130.8 165972.29, 1030121.3 165986.51, 1030121.41 165997.2, 1030
121.04 166009.25, 1030115.04 166016.68, 1030104.29 166021.33, 1030094.19 166029.63, 1030088.37 166040.93, 1030087.14 16
6048.88, 1030094.53 166058.29, 1030098.05 166069.02, 1030096.25 166086.29, 1030082.46 166098.64, 1030061.91 166125.01, 1
030062.44 166137.75, 1030063.02 166147.08, 1030059.9 166160.46, 1030053.67 166168.58, 1030048.34 166176.25, 1030048.76
166179.44, 1030052.13 166182.89, 1030055.99 166183.85, 1029957.253 166548.527, 1030012.594 167201.252, 1030489.499 167
673.921, 1030149.996 167908.304, 1030594.315 168213.066, 1030642.989 168266.744, 1030642.989 168295.836, 1030639.55 16
8333.673, 1030637.171 168359.839, 1030634.262 168447.116, 1030692.446 168487.845, 1030698.557 168490.999, 1030782.632
168534.393, 1030829.18 168621.67, 1030893.183 168720.584, 1030913.548 168857.318, 1030928.094 168964.959, 1030910.345
169055.676, 1030961.423 169306.833, 1030974.343 169367.053, 1030989.732 169426.69, 1031007.563 169485.642, 1031027.807
169543.811, 1031050.429 169601.096, 1031075.39 169657.402, 1031102.648 169712.632, 1031109.989 169746.312, 1031115.108
169780.401, 1031117.983 169814.752, 1031118.603 169849.218, 1031116.963 169883.65, 103113.071 169917.9, 1031106.945 1
69951.823, 1031098.61 169985.271, 1031088.102 170018.101, 1031077.779 170091.957, 1031065.184 170165.459, 1031003.734
170247.931, 1030966.996 170296.583, 1030896.092 170390.483, 1030903.38 170514.366, 1030904.82 170538.854, 1030890.274
170661.041, 1030844.963 170714.349, 1030791.36 170777.411, 1030741.903 170879.234, 1030750.631 170957.783, 1030701.174
171152.702, 1030625.534 171370.894, 1030573.168 171557.085, 1030485.884 171943.604, 1030612.36 172061.209, 1030626.159
172075.514, 1030650.313 172082.574, 1030815.713 172099.851, 1031604.267 171426.069, 1031802.266 171258.789, 1031964.36
3 170991.329, 1032248.033 170967.014, 1032361.501 170967.014, 1032401.76 170957.309, 1032442.551 170950.162,
1032483.711 170945.604, 1032525.077 170943.651, 1032566.484 170944.312, 1032607.767 170947.585, 1032648.761
170953.456, 1032689.303 170961.901, 1032729.231 170972.888, 1032768.387 170986.372, 1032806.614 171002.299, 1032843.76
171020.607, 1032879.676 171041.222, 1032914.221 171064.062, 1032947.255 171089.036, 1032978.648 171116.044, 1033058.51
8 172053.064, 1032936.538 172258.984, 1033041.708 172477.574, 1033269.244 172126.008), ((1026656.633 169555.499,
1026277.077 168886.773, 1023509.947 169001.028, 1024179.161 170686.497, 1026801.981 172020.802, 1027680.184
171210.624, 1027258.098 169997.083, 1026656.633 169555.499)), ((1040934.005 169654.832, 1040308.069 169302.811,
1040327.363 169268.269, 1040285.663 169244.929, 1040241.785 169323.038, 1040284.107 169347, 1040303.09 169313.08, 1040
923.121 169661.809, 1041524.212 169999.884, 1041524.419 170000, 1042204.796 170382.746, 1042311.166 170442.071, 104231
6.261 170431.768, 1041547.758 170000, 1041547.552 169999.884, 1040934.005 169654.832)), ((1029705.147 169249.571,
1028359.189 166859.136, 1027424.42 168785.244, 1027987.723 169400.053, 1029311.382 169878.713, 1029705.147 169249.571)
), ((1036136.414 168588.526, 1036012.277 167876.027, 1036272.224 168264.862, 1036633.522 166929.234, 1036617.837 166285.
702, 1036480.806 165979.41, 1036412.562 165616.022, 1036520.015 165471.606, 1036519.179 165249.605, 1036578.768
165141.374, 1036694.326 164454.299, 1036932.495 163936.517, 1036749.684 163838.056, 1036441.522 164187.475, 1036411.43
6 164459.262, 1036197.263 164817.802, 1035967.31 165719.322, 1035746.697 166086.169, 1035832.484 166559.862,
1035554.289 166941.649, 1035291.501 167161.452, 1035106.501 167525.362, 1035927.887 167373.098, 1035928.806
167449.188, 1035250.369 167815.991, 1034988.201 168529.723, 1034960.288 169081.848, 1035218.952 169355.91, 1035581.038
169280.647, 1035900.692 169140.087, 1036136.414 168588.526)), ((1026623.716 168509.477, 1025930.795 167462.57,
1026643.366 167608.505, 1027262.734 167168.535, 1027672.192 166355.758, 1026960.084 166006.727, 1026313.067
166076.707, 1024044.473 165189.346, 1023261.86 164704.095, 1022819.79 164356.136, 1022103.827 164415.742, 1021994.477
164899.698, 1022161.741 164803.258, 1022694.942 164843.094, 1022707.074 165640.999, 1022827.871 166697.644, 1022148.19
4 166423.192, 1022182.633 167347.288, 1022730.348 167367.912, 1022998.832 167072.417, 1023089.285 167257.495,
1023496.141 168091.548, 1023755.832 168099.888, 1023704.123 168267.811, 1024080.569 168506.131, 1024183.958
168208.848, 1024786.532 167965.758, 1024673.694 168405.216, 1025688.007 168110.111, 1026311.05 168708.801, 1026623.716
168509.477)), ((1020106.509 164080.687, 1018052.359 163522.942, 1019001.488 166490.84, 1021211.605 168039.593,
1021729.652 167250.33, 1021431.234 165731.583, 1021378.527 164625.05, 1020106.509 164080.687)), ((1040367.373 167923.04
8, 1040366.832 167779.046, 1040474.965 167814.639, 1040971.594 167452.751, 1039987.197 166711.148, 1040605.981
166996.741, 1041423.126 167279.365, 1041536.832 167105.222, 1040300.073 166339.61, 1039907.476 165651.071, 1039795.898
164703.458, 1039293.812 165209.376, 1039507.188 164410.293, 1039420.587 163720.596, 1039027.422 162882.051, 1038834.86
162732.773, 1038365.199 162826.615, 1038480.927 163143.57, 1038452.991 163689.694, 1038178.221 164014.743, 1038203.467
164344.66, 1037650.417 165654.798, 1037749.386 165825.077, 1037636.227 166047.514, 1037830.423 166628.802, 1038221.931
167029.336, 1038335.717 166915.635, 1039014.505 167382.97, 1039069.052 167526.766, 1039735.875 167746.257, 1039508.694
167963.121, 1040367.373 167923.048)), ((1047128.451 167093.04, 1047038.639 167034.69, 1047032.419 167041.688,
1047128.58 167103.253, 1047128.451 167093.04)), ((1047618.037 166547.787, 1047601.508 166518.253, 1047629.204 166501.44
8, 1047626.092 166494.602, 1047597.463 166512.34, 1047596.218 166510.161, 1047550.161 166433.608, 1047578.791
166416.181, 1047575.679 166408.712, 1047546.427 166426.762, 1047502.549 166351.142, 1047534.29 166331.225, 1047522.776
166312.242, 1047501.622 166325.862, 1047490.723 166332.781, 1047342.595 166081.959, 1047242.702 165909.869, 1047135.52
6 165730.075, 1047090.218 165654.068, 1046958.271 165430.32, 1046835.039 165222.132, 1046704.373 165000, 1046704.338 1
64999.94, 1046680.687 164960.107, 1046663.572 164971.31, 1046669.795 164981.891, 1046680.687 164974.733, 1046695.624 1
64999.94, 1046695.66 165000, 1046726.432 165052.22, 1046711.806 165061.245, 1046714.918 165067.468, 1046730.167
165058.755, 1046778.09 165138.421, 1046763.153 165147.445, 1046766.265 165154.291, 1046780.891 165144.956, 1046828.815
165225.244, 1046814.5 165233.335, 1046817.923 165239.247, 1046831.927 165230.845, 1046879.851 165311.444, 1046865.225
165319.535, 1046868.648 165325.137, 1046883.274 165316.423, 1046930.575 165397.645, 1046915.327 165405.425, 1046918.75
165410.715, 1046933.687 165402.935, 1046981.922 165483.223, 1046966.673 165491.314, 1046970.097 165497.849, 1046985.34
5 165489.447, 1047032.647 165568.801, 1047018.332 165577.826, 1047021.444 165583.739, 1047036.07 165574.403,

1047083.682 165654.379, 1047068.123 165663.404, 1047072.168 165670.25, 1047087.105 165661.226, 1047127.948 165730.013, 1047134.407 165740.891, 1047119.47 165749.916, 1047122.27 165756.451, 1047137.519 165747.426, 1047185.443 165826.47, 1 047169.572 165836.116, 1047173.306 165842.652, 1047187.932 165833.627, 1047236.167 165912.981, 1047221.23 165922.006, 1047224.342 165927.918, 1047239.279 165918.894, 1047286.892 165999.182, 1047271.332 166007.895, 1047276 166014.741, 10 47290.315 166005.717, 1047336.994 166086.316, 1047323.301 166095.029, 1047326.724 166100.319, 1047341.351 166091.917, 1047388.652 166171.272, 1047374.337 166180.296, 1047377.76 166186.52, 1047391.764 166177.807, 1047439.688 166257.472, 1047425.062 166266.497, 1047428.796 166272.41, 1047442.799 166263.696, 1047484.811 166336.204, 1047430.974 166368.879, 1047442.177 166387.862, 1047496.014 166354.565, 1047500 166361.004, 1047500.059 166361.1, 1047541.448 166429.873, 1047 499.736 166453.397, 1047491.346 166458.192, 1047495.391 166465.349, 1047500 166462.891, 1047543.626 166437.031, 104759 2.172 166515.452, 1047592.795 166516.385, 1047591.239 166517.319, 1047542.693 166544.704, 1047545.805 166552.173, 1047582.525 166529.767, 1047594.351 166522.298, 1047614.86 166555.319, 1047618.037 166547.787)), ((1042096.89 166070.71 6, 1042150.377 166057.842, 1042336.282 166076.761, 1042380.134 166120.992, 1042413.166 166147.07, 1042461.845 166169.671, 1042496.616 166167.933, 1042526.172 166162.717, 1042532.55 166162.717, 1042578.328 166162.717, 1042599.191 166166.194, 1042635.7 166181.841, 1042686.156 166229.989, 1044159.578 165558.12, 1044140.634 165536.233, 1044119.853 1 65516.081, 1044097.395 165497.817, 1044073.43 165481.581, 1044048.141 165467.496, 1044021.72 165455.669, 1043998.049 1 65440.25, 1043975.893 165422.722, 1043955.441 165403.234, 1043936.865 165381.951, 1043920.322 165359.051, 1043905.951 165334.729, 1043893.875 165309.19, 1043884.194 165282.65, 1043876.992 165255.334, 1043915.273 165220.132, 1043902.787 165164.835, 1043888.517 165105.079, 1043897.436 165036.405, 1043892.976 164975.757, 1043879.598 164954.353, 1043886.73 3 164920.462, 1043894.76 164900.84, 1043890.301 164841.085, 1043892.084 164815.22, 1043896.543 164779.545, 1043893.868 164739.411, 1043920.624 164687.682, 1043970.569 164652.899, 1043999.109 164626.142, 1044002.677 164593.143, 1044000.89 3 164552.117, 1043998.806 164546.236, 1043991.082 164524.469, 1043991.082 164509.307, 1043998.217 164496.82, 1044015.163 164494.145, 1044028.187 164496.41, 1044035.676 164497.712, 1044052.622 164497.712, 1044072.243 164497.712, 1044076.702 164488.794, 1044078.486 164473.632, 1044063.324 164462.038, 1044054.861 164451.174, 1044066.892 164441.524, 1044102.567 164433.497, 1044117.729 164418.335, 1044120.404 164396.931, 1044108.81 164382.66, 1044090.08 1 64360.364, 1044090.973 164342.526, 1044105.242 164330.04, 1044127.539 164322.013, 1044134.674 164294.365, 1044133.783 164268.501, 1044123.08 164252.447, 1044122.501 164250.868, 1044113.269 164225.691, 1044098.107 164218.556, 1044092.756 164188.232, 1044091.246 164172.003, 1044089.189 164149.881, 1044111.486 164111.53, 1044142.701 164087.45, 1044165.89 1 64063.369, 1044176.592 164018.775, 1044175.187 164007.062, 1044173.917 163996.478, 1044183.728 163983.992, 1044199.053 163973.775, 1044202.457 163971.506, 1044226.538 163948.317, 1044245.267 163933.155, 1044244.584 163910.272, 1044243.48 3 163873.4, 1044268.455 163827.022, 1044298.779 163821.671, 1044311.039 163817.585, 1044312.158 163817.212, 1044312.349 163815.757, 1044316.617 163783.32, 1044341.795 163774.688, 1044347.832 163772.618, 1044371.913 163761.024, 1044403.129 163712.862, 1044418.722 163667.395, 1044421.858 163637.053, 1044433.452 163622.783, 1044468.236 163628.134, 1044499.223 163645.721, 1044501.235 163646.863, 1044563.666 163642.404, 1044616.287 163627.242, 1044637.69 2 163619.215, 1044667.124 163599.594, 1044687.637 163575.514, 1044688.559 163571.619, 1044695.664 163541.622, 1044685.853 163500.596, 1044692.096 163473.84, 1044715.285 163472.948, 1044737.582 163483.65, 1044759.879 163478.299, 1044775.041 163468.488, 1044780.392 163451.543, 1044804.473 163430.138, 1044810.716 163445.3, 1044807.66 163472.926, 1 044817.851 163485.434, 1044826.77 163487.218, 1044852.634 163477.407, 1044874.039 163451.543, 1044890.093 163442.624, 1044914.173 163430.138, 1044919.649 163416.931, 1044929.335 163393.571, 1044936.47 163373.95, 1044974.821 163372.166, 1044990.824 163370.079, 1044995.334 163369.49, 1045009.604 163358.788, 1045013.171 163351.653, 1045010.48 163342.233, 1045007.82 163332.923, 1045015.847 163325.788, 1045021.348 163326.836, 1045034.577 163329.356, 1045055.981 163322.221, 1045091.656 163276.735, 1045109.494 163263.357, 1045105.035 163206.277, 1045106.818 163179.521, 1045117.521 163155.44, 1045129.115 163125.116, 1045119.58 163114.114, 1045117.521 163111.738, 1045121.088 163092.117, 1045133.575 163059.118, 1045094.332 163031.47, 1045060.441 163035.929, 1045011.388 163048.415, 1045012.28 163042.172, 1045019.414 163003.821, 1045041.739 162981.173, 1045052.414 162970.822, 1045074.711 162951.201, 1045085.413 162918.201, 1045079.17 162904.823, 1045055.981 162897.688, 1045037.252 162903.931, 1045008.712 162903.931, 1044999.793 162887.878, 1044998.901 162855.77, 1044991.767 162833.473, 1044975.713 162829.014, 1044981.064 162800.474, 1044948.064 162774.609, 1044886.525 162761.231, 1044837.472 162725.556, 1044783.068 162641.72, 1044687.637 162556.992, 1044667.124 162562.343, 1044668.016 162554.316, 1044605.585 162500.804, 1044574.369 162505.263, 1044558.315 162514.182, 1044532.451 162497.236, 1044522.64 162466.02, 1044520.856 162447.291, 1044510.154 162415.184, 1044492.316 162391.995, 1044454.858 162391.995, 1044436.128 162382.184, 1044414.723 162355.428, 1044400.453 162348.293, 1044371.913 162341.158, 1044353.184 162309.051, 1044346.04 9 162291.213, 1044301.455 162272.484, 1044288.196 162294.951, 1044269.348 162326.888, 1044237.24 162333.131, 1044159.871 162296.557, 1044119.127 162277.296, 1044090.08 162263.565, 1044057.973 162301.024, 1044058.465 162303.317, 1044060.649 162313.51, 1044043.703 162325.104, 1044019.622 162324.212, 1044010.704 162314.02, 1044007.136 162292.997, 1043983.947 162292.105, 1043950.948 162292.105, 1043941.137 162281.402, 1043924.192 162263.565, 1043918.84 162244.835, 1043927.759 162228.782, 1043925.433 162202.611, 1043924.192 162188.647, 1043899.219 162146.729, 1043885.841 162129.783, 1043882.273 162104.811, 1043819.842 162030.785, 1043802.897 162027.218, 1043794.87 162012.056, 1043728.871 161950.517, 1043606.684 161920.193, 1043537.118 161835.465, 1043438.531 161875.068, 1043432.768 161877.383, 1043361.41 8 161834.573, 1043334.662 161826.546, 1043330.203 161815.843, 1043335.554 161796.222, 1043326.635 161772.142, 1043326.21 161760.248, 1043325.744 161747.169, 1043334.662 161726.656, 1043317.716 161704.359, 1043274.906 161682.954, 1043230.313 161676.711, 1043224.817 161689.992, 1043219.61 161702.575, 1043193.746 161720.413, 1043157.179 161705.251, 1043136.666 161690.981, 1043138.682 161687.957, 1043147.368 161674.927, 1043151.828 161657.981, 1043045.694 161633.009, 1043031.424 161650.846, 1043022.506 161647.279, 1043012.695 161627.658, 1043010.02 161607.145, 1042963.642 161595.55, 1042947.588 161632.117, 1042929.751 161619.631, 1042914.589 161583.956, 1042916.373 161563.443, 1042916.373 161536.686, 1042899.427 161510.822, 1042892.292 161490.309, 1042902.994 161466.228, 1042902.103 161454.634, 1042889.61 6 161456.418, 1042879.806 161455.526, 1042872.671 161444.824, 1042853.049 161422.527, 1042842.442 161423.774, 1042837.888 161424.311, 1042812.915 161423.418, 1042808.456 161399.338, 1042804.64 161372.631, 1042803.104 161361.879, 1042807.564 161336.907, 1042826.293 161315.501, 1042830.753 161303.015, 1042788.835 161279.827, 1042756.727 161271.8, 1042728.196 161269.206, 1042727.295 161269.124, 1042702.322 161275.367, 1042686.269 161260.205, 1042667.575 161247.559, 1042655.945 161239.692, 1042600.649 161229.882, 1042571.217 161214.72, 1042562.298 161202.233, 1042558.731 161190.639, 1042540.001 161180.828, 1042527.515 161170.126, 1042531.974 161149.613, 1042555.163 161138.91, 1042555.163 161106.803, 1042537.326 161086.29, 1042510.569 161061.317, 1042478.462 161045.263, 1042476.437 161044.965, 1042448.138 161040.804, 1042427.625 161022.966, 1042425.291 161005.854, 1042424.949 161003.345, 1042428.517 160965.886, 1042453.48 9 160946.265, 1042408.004 160897.212, 1042360.374 160904.14, 1042358.95 160904.347, 1042343.789 160892.753, 1042321.492 160874.023, 1042318.816 160857.077, 1042321.492 160830.321, 1042326.185 160811.494, 1042311.681 160791.079, 1042312.573 160777.701, 1042316.14 160745.593, 1042306.33 160746.485, 1042285.817 160740.242, 1042284.925 160756.296, 1042268.871 160764.322, 1042268.715 160764.33, 1042229.629 160766.106, 1042187.71 160760.755, 1042173.44 1 60737.566. 1042173.081 160732.893. 1042171.657 160714.378. 1042150.252 160730.431. 1042148.728 160732.893. 1042125.296

160770.744, 1042115.469 160786.619, 1042139.549 160912.374, 1042139.937 160912.737, 1042168.089 160939.13, 1042201.98
160942.697, 1042237.655 160948.049, 1042249.25 160971.238, 1042260.844 160994.426, 1042233.196 161029.21, 1042207.332
161051.506, 1042188.603 161055.966, 1042164.522 161075.587, 1042149.36 161112.154, 1042119.036 161120.181, 1042069.983
161131.775, 1042009.666 161121.542, 1041963.039 161128.715, 1041795.285 161090.749, 1041747.714 161101.128, 1041746.23
2 161101.451, 1041744.129 161103.459, 1041726.611 161120.181, 1041709.665 161134.451, 1041693.943 161136.089,
1041666.855 161138.91, 1041649.91 161156.748, 1041646.298 161167.066, 1041643.667 161174.585, 1041640.099 161190.639,
1041614.235 161211.152, 1041581.236 161209.369, 1041570.533 161211.152, 1041544.668 161213.828, 1041523.263
161237.017, 1041523.263 161252.178, 1041509.885 161273.583, 1041499.183 161276.259, 1041488.48 161286.069, 1041461.724
161311.934, 1041463.508 161326.204, 1041480.453 161330.663, 1041480.826 161331.122, 1041492.048 161344.933, 1041490.26
4 161370.798, 1041475.994 161389.527, 1041453.697 161394.878, 1041429.616 161427.878, 1041404.364 161434.523,
1041395.725 161436.796, 1041378.78 161428.77, 1041353.807 161440.364, 1041327.051 161461.769, 1041296.727 161491.201,
1041295.835 161501.903, 1041317.404 161513.517, 1041319.024 161514.39, 1041317.24 161528.66, 1041301.187 161550.065, 1
041302.078 161563.443, 1041293.16 161588.415, 1041259.121 161621.582, 1041258.376 161622.306, 1041236.08 161653.522, 1
041221.809 161673.144, 1041229.211 161678.078, 1041235.188 161682.062, 1041235.188 161696.332, 1041222.702 161720.413,
1041203.972 161728.44, 1041183.459 161738.25, 1041153.135 161760.547, 1041145.097 161760.547, 1041129.947 161760.547,
1041102.298 161759.655, 1041086.244 161773.033, 1041064.84 161777.493, 1041014.003 161781.061, 1040985.463 161781.061,
1040948.896 161802.465, 1040882.005 161847.059, 1040823.141 161860.437, 1040785.682 161884.518, 1040737.521
161920.193, 1040668.847 161948.733, 1040642.091 161948.733, 1040634.064 161954.976, 1040635.848 161968.354, 1040635.99
6 161971.184, 1040636.739 161985.3, 1040622.47 161997.786, 1040609.983 161999.57, 1040579.659 161999.57, 1040567.872 16
1999.57, 1040551.119 161999.57, 1040525.255 161999.57, 1040510.985 162001.353, 1040510.093 162010.272, 1040511.896 162
022.893, 1040513.661 162035.245, 1040486.012 162061.109, 1040466.391 162052.191, 1040415.554 162046.839, 1040351.339 1
62056.65, 1040312.097 162059.326, 1040266.611 162054.866, 1040234.276 162058.021, 1040230.044 162058.434, 1040226.73 1
62059.842, 1040194.369 162073.595, 1040200.613 162091.433, 1040199.72 162113.73, 1040189.018 162132.459, 1040156.91 16
2143.162, 1040123.019 162133.351, 1040093.587 162128.892, 1040061.48 162119.973, 1040046.318 162128.892, 1040021.345 1
62165.458, 1040022.237 162200.242, 1040022.237 162251.97, 1040025.805 162288.537, 1040008.859 162317.97, 1039967.833 1
62317.97, 1039951.779 162309.051, 1039950.813 162303.98, 1039948.212 162290.321, 1039932.158 162282.294, 1039887.564 1
62262.673, 1039860.808 162264.457, 1039855.436 162260.253, 1039840.295 162248.403, 1039811.755 162229.673, 1039811.755
162214.512, 1039775.188 162176.161, 1039758.242 162170.81, 1039731.486 162170.81, 1039708.297 162160.999, 1039694.919
162180.621, 1039684.217 162189.539, 1039669.055 162189.539, 1039658.352 162184.188, 1039642.256 162177.571, 1039633.38
162177.945, 1039629.812 162180.621, 1039620.893 162198.458, 1039620.893 162204.033, 1039620.893 162211.836, 1039623.56
9 162226.106, 1039611.975 162236.809, 1039597.705 162248.403, 1039585.218 162260.889, 1039573.624 162274.267,
1039556.906 162283.295, 1039574.516 162292.997, 1039581.651 162301.915, 1039590.57 162312.618, 1039606.624 162316.186,
1039623.569 162317.077, 1039646.758 162309.942, 1039662.812 162322.429, 1039681.541 162331.347, 1039712.757 162337.59,
1039723.829 162357.363, 1039725.243 162359.887, 1039719.892 162377.725, 1039726.135 162399.13, 1039759.134 162472.264,
1039759.134 162482.966, 1039762.702 162495.452, 1039766.266 162499.526, 1039781.431 162516.858, 1039774.296
162530.236, 1039761.962 162542.57, 1039759.134 162545.397, 1039753.783 162556.1, 1039753.783 162573.046, 1039757.351 1
62592.667, 1039759.636 162599.905, 1039762.702 162609.613, 1039774.296 162622.099, 1039775.188 162640.828, 1039776.08
162665.801, 1039780.539 162692.557, 1039796.593 162724.664, 1039814.431 162732.691, 1039839.403 162735.367, 1039855.45
7 162741.61, 1039865.267 162760.34, 1039888.456 162760.34, 1039905.402 162763.907, 1039922.891 162777.595, 1039925.915
162779.961, 1039951.779 162787.096, 1039967.833 162803.15, 1039971.4 162829.014, 1039958.022 162862.905, 1039946.835 1
62872.058, 1039938.401 162878.959, 1039924.724 162880.098, 1039906.293 162881.634, 1039891.132 162885.202, 1039875.078
162896.796, 1039872.402 162936.931, 1039865.267 162955.66, 1039878.645 162969.93, 1039889.107 162980.392, 1039911.645
163002.929, 1039923.239 163100.144, 1039914.32 163129.576, 1039922.347 163139.387, 1039932.158 163147.413, 1039914.32
163154.548, 1039912.537 163166.143, 1039913.323 163174.397, 1039914.32 163184.872, 1039915.394 163185.639, 1039926.807
163193.791, 1039927.699 163206.277, 1039922.461 163224.004, 1039916.104 163245.52, 1039927.699 163272.276, 1039932.487
163278.433, 1039952.671 163304.384, 1039949.996 163330.248, 1039945.85 163350.974, 1039942.861 163365.923, 1039972.292
163416.759, 1039975.222 163419.013, 1040007.075 163443.516, 1040020.454 163471.164, 1040027.589 163501.488, 1040036.50
7 163522.893, 1040041.347 163524.487, 1040045.863 163526.847, 1040049.935 163529.91, 1040053.453 163533.595,
1040053.563 163542.75, 1040052.381 163551.829, 1040049.931 163560.651, 1040046.876 163567.842, 1040042.977 163574.613,
1040038.291 163580.865, 1040038.778 163588.154, 1040040.414 163595.274, 1040043.158 163602.045, 1040046.942
163608.294, 1040051.669 163613.864, 1040061.788 163609.13, 1040070.982 163602.783, 1040078.996 163594.999, 1040081.101
163592.459, 1040086.737 163591.444, 1040092.464 163591.368, 1040098.125 163592.234, 1040103.567 163594.018, 1040108.64
2 163596.672, 1040113.212 163600.123, 1040117.154 163604.278, 1040120.359 163609.023, 1040122.742 163614.231,
1040124.236 163619.76, 1040124.803 163625.458, 1040120.641 163644.217, 1040116.776 163662.025, 1040130.296 163667.734,
1040142.684 163675.602, 1040153.6 163685.412, 1040162.741 163696.893, 1040169.856 163709.728, 1040174.748 163723.565,
1040177.346 163724.94, 1040180.129 163725.886, 1040183.027 163726.378, 1040185.967 163726.405, 1040188.873 163725.965,
1040191.673 163725.069, 1040194.296 163723.741, 1040196.674 163722.014, 1040198.749 163719.931, 1040200.467
163717.546, 1040201.785 163714.918, 1040202.68 163712.069, 1040203.104 163709.112, 1040203.047 163706.126, 1040202.509
163703.189, 1040201.504 163700.376, 1040208.961 163690.35, 1040217.834 163681.553, 1040227.925 163674.183, 1040239.005
163668.407, 1040250.825 163664.355, 1040263.118 163662.119, 1040273.908 163661.687, 1040284.663 163662.657, 1040286.23
3 163662.917, 1040295.514 163673.957, 1040303.028 163686.269, 1040308.387 163698.926, 1040311.874 163712.222,
1040313.419 163725.88, 1040312.989 163739.619, 1040308.073 163738.312, 1040303.008 163737.832, 1040297.935 163738.194,
1040292.99 163739.387, 1040288.309 163741.378, 1040284.02 163744.113, 1040280.24 163747.517, 1040277.073 163751.497, 1
040274.604 163755.945, 1040272.903 163760.739, 1040272.014 163765.747, 1040271.963 163770.834, 1040272.621 163788.315,
1040275.697 163805.535, 1040281.13 163822.163, 1040288.816 163837.877, 1040298.607 163852.374, 1040310.313 163865.373,
1040309.564 163868.838, 1040309.372 163872.378, 1040309.742 163875.903, 1040310.666 163879.326, 1040312.12 163882.559,
1040314.066 163885.522, 1040316.457 163888.139, 1040319.232 163890.345, 1040332.061 163895.467, 1040359.835
163908.209, 1040386.66 163922.844, 1040412.408 163939.3, 1040436.955 163957.499, 1040455.689 163973.29, 1040473.309 16
3975.031, 1040491.004 163974.389, 1040501.181 163978.028, 1040510.596 163983.336, 1040518.979 163990.159, 1040526.087
163998.3, 1040531.717 164007.526, 1040535.707 164017.571, 1040537.941 164028.146, 1040538.355 164038.946, 1040536.937
164049.661, 1040533.729 164059.982, 1040528.822 164069.612, 1040523.935 164068.6, 1040518.949 164068.358, 1040513.986
164068.891, 1040509.166 164070.187, 1040504.604 164072.214, 1040500.412 164074.923, 1040496.69 164078.25, 1040493.529
164082.112, 1040491.004 164086.418, 1040489.1 164091.32, 1040488.021 164096.466, 1040487.796 164101.72, 1040491.004 16
4119.268, 1040496.124 164137.535, 1040502.958 164155.233, 1040500.126 164157.233, 1040497.628 164159.639, 1040495.523
164162.394, 1040493.857 164165.435, 1040492.544 164169.163, 1040491.891 164173.061, 1040491.919 164177.013, 1040492.62
6 164180.902, 1040493.991 164184.611, 1040495.974 164188.031, 1040498.515 164191.058, 1040501.539 164193.603,
1040503.95 164195.085, 1040506.526 164196.259, 1040523.839 164215.544, 1040542.607 164233.418, 1040562.714 164249.771

1040567.366 164259.778, 1040570.741 164270.285, 1040562.708 164289.005, 1040553.479 164307.165, 1040543.092
164324.689, 1040541.341 164327.756, 1040540.016 164331.03, 1040539.029 164335.12, 1040538.707 164339.315, 1040539.059
164343.508, 1040540.076 164347.591, 1040541.732 164351.459, 1040543.984 164355.012, 1040546.216 164352.729, 1040549.57
164350.169, 1040553.301 164348.196, 1040557.305 164346.865, 1040561.474 164346.21, 1040565.693 164346.251, 1040569.849
164346.986, 1040576.554 164355.675, 1040584.458 164363.291, 1040593.389 164369.67, 1040603.156 164374.676, 1040613.551
164378.201, 1040620.833 164379.715, 1040628.265 164380.007, 1040635.643 164379.067, 1040642.764 164376.922, 1040649.43
4 164373.63, 1040655.469 164369.283, 1040658.759 164371.431, 1040661.641 164374.101, 1040664.033 164377.218,
1040665.867 164380.693, 1040667.092 164384.426, 1040667.672 164388.312, 1040667.59 164392.24, 1040666.85 164396.099, 1
040665.472 164399.778, 1040663.496 164403.174, 1040660.95 164404.5, 1040658.658 164406.227, 1040656.681 164408.308, 10
40655.072 164410.685, 1040653.876 164413.294, 1040653.126 164416.065, 1040652.84 164418.921, 1040653.029 164421.785, 1
040653.685 164424.579, 1040659.717 164435.376, 1040663.813 164447.046, 1040665.852 164459.245, 1040665.775 164471.613,
1040664.327 164480.78, 1040661.712 164489.686, 1040658.499 164510.03, 1040652.777 164529.815, 1040647.252 164543.327,
1040640.554 164556.298, 1040634.517 164566.022, 1040627.821 164575.306, 1040613.551 164578.873, 1040609.983
164593.143, 1040605.524 164616.332, 1040605.524 164634.169, 1040614.442 164636.845, 1040634.956 164636.845, 1040646.55
164635.953, 1040656.361 164632.386, 1040665.476 164630.467, 1040673.306 164628.818, 1040687.576 164630.602, 1040702.73
8 164637.737, 1040715.225 164647.548, 1040729.494 164652.007, 1040741.981 164652.899, 1040745.548 164662.709,
1040758.035 164664.493, 1040770.521 164660.034, 1040785.682 164661.817, 1040791.034 164679.655, 1040792.817
164710.871, 1040786.575 164726.924, 1040789.25 164742.087, 1040802.628 164755.465, 1040797.277 164770.626, 1040804.412
164781.329, 1040819.574 164785.788, 1040821.358 164798.275, 1040805.304 164813.436, 1040805.304 164825.923, 1040823.14
1 164852.679, 1040828.405 164866.278, 1040831.769 164880.468, 1040833.171 164894.983, 1040832.585 164909.553,
1040830.022 164923.909, 1040825.529 164937.782, 1040819.191 164950.915, 1040814.223 164958.812, 1040802.628
164980.217, 1040818.682 165005.189, 1040840.979 165031.054, 1040849.006 165047.999, 1040840.979 165064.053, 1040831.16
8 165082.783, 1040839.195 165105.079, 1040875.762 165141.647, 1040891.533 165145.151, 1040891.816 165145.214,
1040904.302 165151.457, 1040902.518 165174.646, 1040995.273 165302.184, 1041005.084 165353.021, 1041000.151
165417.167, 1041023.572 165500.814, 1041028.273 165508.207, 1041022.044 165519.41, 1041016.423 165530.929, 1041010.785
165544.382, 1041005.976 165558.152, 1041007.082 165561.199, 1041008.552 165564.088, 1041010.992 165567.566, 1041013.95
5 165570.61, 1041017.366 165573.142, 1041021.138 165575.098, 1041034.496 165575.045, 1041047.698 165577.086,
1041060.417 165581.172, 1041072.338 165587.199, 1041083.168 165595.021, 1041092.638 165604.443, 1041100.515
165615.232, 1041104.974 165637.529, 1041117.544 165646.261, 1041128.77 165656.665, 1041138.43 165668.536, 1041146.334
165681.643, 1041152.328 165695.726, 1041156.293 165710.509, 1041158.151 165725.701, 1041158.101 165737.844, 1041156.70
3 165749.905, 1041150.206 165762.233, 1041141.938 165773.449, 1041132.085 165783.302, 1041120.869 165791.57,
1041108.541 165798.067, 1041103.014 165804.916, 1041098.582 165812.519, 1041095.346 165820.704, 1041093.379
165829.282, 1041112.261 165843.12, 1041132.544 165854.807, 1041153.984 165864.205, 1041176.324 165871.2, 1041190.594 1
65901.524, 1041194.392 165905.953, 1041198.847 165909.722, 1041203.843 165912.734, 1041209.256 165914.913, 1041214.946
165916.205, 1041220.769 165916.574, 1041226.576 165916.013, 1041232.221 165914.536, 1041237.559 165912.18, 1041242.454
165909.005, 1041246.782 165905.092, 1041247.789 165900.234, 1041249.598 165895.613, 1041252.155 165891.362, 1041255.39
1 165887.6, 1041259.211 165884.435, 1041263.509 165881.955, 1041268.161 165880.232, 1041273.037 165879.314,
1041277.998 165879.227, 1041284.153 165879.22, 1041289.649 165880.098, 1041294.949 165881.797, 1041299.932 165884.277,
1041304.483 165887.481, 1041308.498 165891.336, 1041311.885 165895.754, 1041314.565 165900.632, 1041316.168
165915.808, 1041319.004 165936.373, 1041322.591 165956.82, 1041319.999 165967.279, 1041315.713 165977.165, 1041309.852
165986.207, 1041302.577 165994.156, 1041294.088 166000.793, 1041284.619 166005.935, 1041274.43 166009.441, 1041267.271
166012.425, 1041260.723 166016.583, 1041254.978 166021.792, 1041250.201 166027.903, 1041246.532 166034.736, 1041244.07
8 166042.094, 1041242.91 166049.761, 1041242.891 166055.446, 1041243.855 166062.438, 1041245.89 166069.197,
1041245.554 166088.736, 1041247.63 166108.168, 1041252.088 166127.196, 1041258.859 166145.528, 1041267.839 166162.885,
1041278.893 166179.001, 1041291.849 166193.631, 1041306.512 166206.551, 1041322.656 166217.564, 1041340.035
166226.501, 1041377.496 166233.885, 1041415.337 166238.979, 1041453.417 166241.764, 1041491.596 166242.23, 1041529.733
166240.374, 1041567.686 166236.203, 1041605.316 166229.734, 1041746.068 166159.464, 1041824.717 166124.493, 1041918.36
4 166090.602, 1042025.389 166087.926, 1042096.89 166070.716)), ((1049019.806 163391.243, 1048591.005 163378.031,
1047000.759 163784.054, 1048451.484 165572.059, 1048805.889 165431.075, 1049016.653 163425.032, 1049019.806
163391.243)), ((1027824.508 164373.921, 1027883.124 163962.971, 1027557.251 164119.752, 1027410.562 163327.647,
1028395.301 162659.493, 1029156.254 162958.938, 1028457.108 159968.74, 1026694.156 159226.373, 1025402.046 160420.209,
1025555.113 160720.107, 1026512.316 161273.987, 1026405.4 162475.658, 1026642.667 162805.506, 1026895.278 163752.597,
1026962.82 164369.013, 1027824.508 164373.921)), ((1024442.238 163782.899, 1024423.885 162694.975, 1025239.093 162700.1
88, 1024849.435 163038.944, 1025164.356 163406.161, 1025967.258 163430.105, 1025867.973 161628.753, 1024469.023
160135.692, 1023932.784 160261.499, 1023944.427 160928.689, 1023394.793 161356.342, 102329.612 162004.402, 1023669.64
1 162510.996, 1024096.143 162710.993, 1023511.748 164067.606, 1024442.238 163782.899)), ((1035865.344 163198.026,
1035853.604 163161.339, 1035844.828 163133.915, 1035775.589 163064.676, 1035701.221 163018.516, 1035649.932
163031.338, 1035649.932 163090.32, 1035652.497 163136.48, 1035601.208 163139.044, 1035514.018 163090.32, 1035475.551 1
63041.596, 1035514.018 163000.565, 1035511.453 162964.663, 1035560.177 162913.375, 1035614.03 162874.908, 1035648.223
162797.874, 1035670.448 162733.865, 1035655.061 162710.785, 1035649.271 162704.416, 1035603.772 162654.368, 1035547.35
5 162656.932, 1035534.533 162687.705, 1035537.098 162736.429, 1035524.275 162754.38, 1035483.245 162787.718,
1035444.778 162782.589, 1035414.005 162762.074, 1035396.054 162697.963, 1035367.845 162669.754, 1035339.637 162667.19,
1035292.872 162641.29, 1035215.431 163129.406, 1035270.397 163110.835, 1035296.042 163115.964, 1035339.637 163131.351,
1035383.232 163149.302, 1035419.134 163151.866, 1035460.165 163139.044, 1035506.325 163164.688, 1035547.355
163185.204, 1035593.515 163203.155, 1035624.288 163195.462, 1035690.963 163208.283, 1035726.865 163257.008, 1035767.89
6 163300.603, 1035832.08 163284.127, 1035867.908 163231.363, 1035865.344 163198.026)), ((1035624.64 161723.353, 1035608.
905 161698.746, 1035595.173 161672.968, 1035583.532 161646.18, 1035574.055 161618.552, 1035566.802 161590.259,
1035561.819 161561.479, 1035559.137 161532.395, 1035558.773 161503.189, 1035449.448 161814.744, 1035503.79 161710.036,
1035542.353 161751.955, 1035604.85 161834.022, 1035658.016 161868.572, 1035722.717 161910.613, 1035753.449 161956.103
5754.766 161838.318, 1035729.49 161823.682, 1035705.456 161807.084, 1035682.816 161788.631, 1035661.713 161768.438, 10
35642.28 161746.633, 1035624.64 161723.353)), ((1020440.204 160282.239, 1020044.439 159609.126, 1020053.788 160188.751,
1019664.254 160880.562, 1019598.813 161242.049, 1019511.557 161743.767, 1019844.998 161644.047, 1020056.904 161441.49,
1020134.811 161070.654, 1020440.204 160282.239)), ((1041392.84 160756.189, 1041582.912 160245.455, 1041450.947 160154.9
96, 1041146.584 160588.162, 1041381.561 160089.337, 1040629.108 159438.157, 1039452.775 159360.536, 1039285.184
159469.174, 1038595.608 159579.781, 1038337.941 160388.106, 1038616.701 161119.079, 1039338.746 161662.372, 1040118.88
9 161701.427, 1040807.047 161340.424, 1040866.345 161154.191, 1041004.209 161117.668, 1041087.526 160937.347,
1041392.84 160756.189)), ((1030776.129 160077.329 1030193.506 159911.522 1029822.821 159968.944 1029759.015 160597.6

1030262.594 161262.684, 1030415.421 161299.535, 1030395.704 161395.883, 1030750.854 161700.554, 1031060.616 161350.215, 1031263.879 161120.325, 1031360.931 161010.559, 1031326.486 160347.9, 1030776.129 160077.329)), ((1032542.346 160994.631, 1032393.754 160906.317, 1032088.033 160979.476, 1031992.525 161579.564, 1032317.012 161416.395, 1032542.346 160994.631)), ((1021172.527 160840.051, 1020695.738 160462.983, 1020711.319 160902.376, 1020963.737 161011.445, 1021172.527 160840.051)), ((1032235.044 160805.293, 1032342.767 160732.882, 1032480.903 160768.361, 1032515.529 160672.626, 1032455.26 160600.852, 1032568.983 160528.418, 1032592.031 160276.325, 1032426.92 160188.025, 1032313.193 160260.459, 1031838.388 160046.243, 1031825.904 160198.389, 1031966.235 160815.881, 1032235.044 160805.293)), ((1035424.727 160278.891, 1035474.18 160265.36, 1035475.38 160271.28, 1035491 160268.49, 1035472.46 160171.88, 1035457 160175.43, 1035473.05 160260.89, 1035422.27 160272.41, 1035417.03 160241.42, 1035405.84 160247.71, 1035412.926 160281.845, 1035421.728 160279.453, 1035424.727 160278.891)), ((1036065.715 159517.714, 1036498.564 159070.734, 1036772.747 159103.767, 1036861.425 158838.83, 1037048.79 158620.567, 1037032.736 158580.932, 1037011.947 158547.858, 1037001.553 158516.675, 1037001.553 158481.711, 1037004.388 158463.757, 1037025.177 158434.463, 1037035.571 158410.839, 1037027.067 158390.05, 1037012.892 158367.371, 1037000.608 158326.738, 1037001.553 158305.004, 1036999.663 158277.6, 1037004.388 158243.582, 1037017.617 158214.288, 1037028.957 158178.38, 1037031.791 158157.591, 1037042.186 158123.572, 1037050.691 158103.728, 1037053.525 158063.095, 1037045.021 158039.471, 1037028.957 158022.462, 1037014.782 158013.012, 1037011.947 157996.948, 1037008.168 157969.544, 1037020.452 157954.425, 1037011.947 157940.251, 1036993.048 157950.645, 1036972.259 157949.7, 1036972.259 157932.691, 1036982.654 157919.462, 1036998.718 157900.562, 1037015.727 157887.333, 1037019.507 157870.324, 1036997.773 157865.599, 1036984.543 157873.158, 1036976.039 157879.773, 1036959.975 157878.828, 1036950.525 157863.709, 1036945.8 157846.7, 1036944.855 157828.746, 1036945.8 157817.406, 1036949.58 157807.011, 1036938.241 157807.957, 1036930.681 157816.461, 1036928.791 157804.176, 1036928.791 157785.277, 1036933.516 157768.268, 1036942.02 157749.369, 1036942.02 157728.58, 1036942.02 157697.397, 1036921.231 157682.277, 1036913.672 157660.543, 1036916.507 157634.084, 1036921.231 157611.405, 1036927.846 157584.946, 1036932.571 157562.267, 1036921.231 157547.148, 1036905.167 157537.699, 1036897.608 157522.579, 1036899.497 157506.515, 1036877.763 157484.781, 1036850.36 157473.441, 1036822.956 157455.487, 1036795.552 157446.983, 1036774.763 157433.753, 1036763.423 157413.909, 1036746.414 157403.514, 1036723.735 157404.459, 1036702.946 157412.964, 1036685.841 157413.535, 1036674.598 157413.909, 1036648.139 157400.68, 1036628.294 157383.67, 1036619.79 157362.881, 1036599.946 157335.478, 1036590.496 157320.358, 1036587.661 157298.624, 1036594.8 157279.008, 1036576.285 157251.649, 1036533.799 157300.514, 1036516.789 157318.468, 1036475.212 157343.983, 1036442.138 157366.661, 1036396.78 157401.625, 1036395.835 157412.964, 1036399.615 157424.304, 1036395.835 157436.588, 1036390.165 157450.762, 1036383.551 157465.882, 1036377.881 157479.111, 1036347.642 157526.359, 1036309.844 157573.607, 1036288.11 157610.46, 1036255.036 157653.928, 1036186.999 157729.525, 1036164.726 157747.167, 1036097.826 157861.015, 1035776.9 158125.044, 1035618.19 158675.979, 1035818.432 159550.559, 1036065.715 159517.714)), ((1021173.514 159264.418, 1020992.59 159019.09, 1020818.682 159043.746, 1020921.815 159343.368, 1021174.105 159420.421, 1021173.514 159264.418)), ((1025230.393 156935.587, 1025159.139 156876.21, 1025111.639 156902.929, 1024891.945 156754.488, 1024796.942 156698.081, 1024693.031 156701.05, 1024612.874 156659.485, 1024538.655 156558.544, 1024253.646 156516.982, 1024036.921 156475.417, 1023918.168 156463.54, 1023745.976 156490.263, 1023006.221 156618.078, 1022976.559 156833.389, 1022735.923 157084.638, 1022636.215 157283.214, 1022364.89 157564.057, 1022214.949 157833, 1022194.286 158019.561, 1022300.783 158273.324, 1022606.464 158691.341, 1022698.278 158770.193, 1022799.81 158809.077, 1022812.773 158832.299, 1022908.944 158853.608, 1022990.42 158755.714, 1023387.591 158655.098, 1024563.212 158586.253, 1024594.987 158506.82, 1024716.785 158427.385, 1024870.357 158416.795, 1025135.14 158252.632, 1025188.096 158114.945, 1025336.37 157982.556, 1025415.806 157797.209, 1025692.085 157485.061, 1025480.241 157336.964, 1025349.146 157086.997, 1025230.393 156935.587)), ((1020296.978 157533.653, 1020050.846 157498.585, 1020211.945 158848.03, 1020276.676 158511.772, 1020296.978 157533.653)), ((1033369.627 157951.134, 1033369.507 157942.898, 1033365.329 157943.017, 1033365.091 157937.526, 1033361.271 157937.048, 1033359.838 157913.651, 1033354.108 157914.129, 1033355.899 157939.675, 1033354.108 157940.033, 1033353.989 157945.046, 1033351.124 157945.166, 1033352.198 157952.686, 1033369.627 157951.134)), ((1033422.356 157944.66, 1033419.997 157936.996, 1033408.207 157938.175, 1033401.132 157895.138, 1033393.468 157896.907, 1033401.132 157939.943, 1033391.11 157940.533, 1033391.11 157947.018, 1033422.356 157944.66)), ((1032998.528 157599.303, 1033016.939 157592.882, 1033015.759 157589.149, 1032995.524 157596.247, 1032985.879 157560.231, 1032981.686 157544.61, 1033024.679 157539.095, 1033026.856 157538.747, 1033031.434 157538.014, 1033062.655 157533.018, 1033077.225 157530.2, 1033089.983 157526.761, 1033103.755 157524.232, 1033105.987 157529.074, 1033136.923 157516.52, 1033136.882 157512.528, 1033108.897 157517.035, 1033097.432 157519.892, 1033074.093 157525.491, 1033050.328 157530.22, 1033029.176 157533.239, 1032980.203 157539.308, 1032976.258 157525.074, 1032955.12 157456.383, 1032954.137 157453.187, 1032953.636 157451.56, 1032948.716 157435.569, 1032948.173 157433.807, 1032944.595 157422.177, 1032941.431 157411.896, 1032935.582 157394.24, 1032932.009 157382.34, 1032930.019 157375.713, 1032929.472 157373.891, 1032926.386 157363.613, 1032923.639 157354.466, 1032919.453 157337.978, 1032918.151 157338.4, 1032917.752 157334.501, 1032911.692 157335.359, 1032912.189 157336.975, 1032917.989 157355.84, 1032918.431 157357.278, 1032922.29 157369.803, 1032927.234 157385.914, 1032930.063 157395.116, 1032930.654 157397.039, 1032935.877 157413.83, 1032936.275 157415.192, 1032936.715 157416.69, 1032942.096 157435.059, 1032942.736 157437.243, 1032947.502 157453.509, 1032948.093 157455.528, 1032952.732 157471.361, 1032953.24 157473.097, 1032957.999 157489.34, 1032958.353 157490.55, 1032963.282 157507.372, 1032963.798 157509.133, 1032968.482 157525.121, 1032968.992 157526.861, 1032973.783 157543.214, 1032974.27 157544.876, 1032984.863 157581.035, 1032985.48 157583.139, 1032989.711 157597.582, 1032977.823 157600.595, 1032979.092 157604.069, 1032992.8 157600.812, 1032998.528 157599.303)), ((1020482.59 157430.95, 1020443.801 156693.064, 1020294.169 156789.632, 1020482.59 157430.95)), ((1029606.077 156073.814, 1029578.007 156052.222, 1029580.617 156031.343, 1029598.886 155988.281, 1029664.132 155947.829, 1029728.072 155909.986, 1029798.538 155885.193, 1029829.856 155923.035, 1029848.455 155923.811, 1029861.174 155924.34, 1029892.256 155903.339, 1029909.455 155891.717, 1029937.776 155895.207, 1029959.042 155929.56, 1029998.19 155902.157, 1030041.252 155863.009, 1030058.216 155774.275, 1030037.337 155776.885, 1030033.68 155765.304, 1030029.508 155752.092, 1030042.557 155718.164, 1030084.314 155688.151, 1030110.413 155692.066, 1030259.769 155606.544, 1030263.414 155590.608, 1030267.328 155569.077, 1030259.499 155554.397, 1030268.307 155536.78, 1030265.371 155517.207, 1030289.838 155492.74, 1030322.135 155492.74, 1030348.559 155486.868, 1030387.707 155469.251, 1030393.579 155484.91, 1030426.854 155484.91, 1030460.13 155467.294, 1030485.575 155443.805, 1030491.448 155413.466, 1030496.341 155389.978, 1030529.616 155348.873, 1030548.212 155326.363, 1030560.934 155326.363, 1030565.828 155339.086, 1030568.764 155355.723, 1030590.295 155362.574, 1030606.933 155361.596, 1030619.655 155331.256, 1030620.634 155272.535, 1030610.848 155239.26, 1030593.231 155194.24, 1030584.423 155159.986, 1030595.189 155127.69, 1030596.167 155098.329, 1030688.163 155070.926, 1030710.673 155070.926, 1030725.354 155068.968, 1030736.119 155065.054, 1030748.842 155045.48, 1030740.366 155032.475, 1030730.929 155017.995, 1030733.673 154996.546, 1030739.545 154976.972, 1030720.95 154964.249, 1030701.376 154959.356, 1030695.281 154963.3, 1030684.738 154970.121, 1030681.802 154993.61, 1030664.186 154998.503, 1030634.825 154995.567, 1030612.315 154978.929, 1030603.507 154954.462, 1030617.200 154931.952, 1030615.251 154901.613, 1030610.358 154861.487, 1030591.763 154840.743, 1030562.402

154870.295, 1030570.232 154897.699, 1030578.061 154925.102, 1030567.296 154956.42, 1030545.765 154986.759, 1030508.575
155015.141, 1030475.299 155032.757, 1030443.981 155041.565, 1030420.493 155038.629, 1030442.024 155007.311, 1030439.08
8 154982.844, 1030436.152 154960.334, 1030413.642 154946.633, 1030387.217 154949.569, 1030371.559 154968.164,
1030353.942 154993.61, 1030365.686 155022.97, 1030361.772 155040.587, 1030330.454 155031.778, 1030319.688 155008.29, 1
030290.328 154976.972, 1030313.816 154953.483, 1030323.603 154924.123, 1030332.411 154894.762, 1030354.921 154872.253,
1030374.495 154847.785, 1030356.878 154827.233, 1030328.496 154793.958, 1030290.328 154773.405, 1030296.2 154756.767,
1030311.859 154746.981, 1030335.347 154759.704, 1030345.134 154786.128, 1030385.26 154784.171, 1030419.514 154779.277,
1030464.534 154768.512, 1030493.894 154768.512, 1030511.511 154785.149, 1030532.063 154770.469, 1030552.616
154758.725, 1030574.147 154750.895, 1030599.592 154747.959, 1030617.209 154760.682, 1030632.868 154777.32, 1030652.441
154793.958, 1030666.143 154804.723, 1030676.909 154805.702, 1030681.802 154790.043, 1030693.546 154765.576, 1030719.97
1 154765.576, 1030728.666 154763.274, 1030753.246 154756.767, 1030752.267 154737.194, 1030742.481 154706.855,
1030740.523 154684.345, 1030742.481 154661.835, 1030762.054 154658.899, 1030764.991 154624.645, 1030773.799
154580.604, 1030778.692 154542.436, 1030777.713 154517.968, 1030771.841 154492.522, 1030770.863 154465.119, 1030775.75
6 154437.716, 1030782.607 154415.206, 1030781.628 154384.867, 1030761.076 154353.549, 1030735.63 154341.804,
1030703.333 154327.124, 1030673.973 154308.529, 1030636.783 154308.529, 1030588.489 154309.8, 1030553.594 154304.615,
1030513.468 154278.19, 1030482.15 154285.041, 1030468.449 154315.38, 1030443.981 154301.678, 1030417.557 154285.041, 1
030388.196 154254.702, 1030353.942 154244.915, 1030322.624 154242.957, 1030290.328 154249.808, 1030245.308 154255.68,
1030213.012 154283.083, 1030176.8 154291.892, 1030136.674 154281.126, 1030123.951 154253.723, 1030117.1 154228.277,
1030109.271 154196.959, 1030089.697 154173.47, 1030099.484 154151.939, 1030128.844 154154.876, 1030169.949 154178.364,
1030204.203 154193.044, 1030227.692 154203.81, 1030253.137 154191.087, 1030269.775 154174.449, 1030304.029 154173.47,
1030339.262 154179.343, 1030353.942 154199.895, 1030369.601 154228.277, 1030401.898 154245.893, 1030424.408
154228.277, 1030454.747 154222.405, 1030484.108 154223.384, 1030523.255 154220.447, 1030541.275 154213.833, 1030569.25
3 154198.916, 1030568.275 154171.513, 1030550.658 154141.174, 1030519.34 154108.877, 1030509.553 154072.666,
1030499.767 154043.305, 1030466.98 154028.625, 1030440.998 154029.131, 1030397.99 154029.97, 1030372.459 154030.467, 1
030356.878 154044.284, 1030327.518 154044.284, 1030279.633 154032.277, 1030219.975 154033.439, 1030191.48 154014.923,
1030159.184 154020.795, 1030118.999 154035.407, 1030096.548 154056.028, 1030073.059 154064.837, 1030044.677
154059.943, 1030042.72 154073.644, 1030015.317 154079.517, 1029975.191 154068.751, 1029943.873 154057.007, 1029906.682
154059.943, 1029883.194 154059.943, 1029863.62 154055.05, 1029844.937 154040.749, 1029814.197 154041.348, 1029798.048
154057.985, 1029779.453 154065.815, 1029764.773 154065.815, 1029743.242 154074.623, 1029725.626 154073.644, 1029709.96
7 154094.197, 1029686.478 154112.792, 1029670.819 154121.6, 1029652.224 154126.493, 1029634.608 154119.643,
1029620.906 154115.728, 1029610.141 154132.366, 1029588.609 154143.131, 1029564.142 154150.961, 1029549.462
154165.641, 1029539.675 154179.343, 1029508.357 154176.407, 1029499.549 154162.705, 1029467.252 154161.726, 1029452.57
2 154176.407, 1029451.593 154188.151, 1029451.593 154207.725, 1029453.551 154222.405, 1029461.38 154240.021,
1029451.593 154244.915, 1029433.977 154259.595, 1029432.999 154275.254, 1029423.212 154289.934, 1029406.574
154309.508, 1029397.766 154317.337, 1029367.426 154345.72, 1029337.087 154370.186, 1029297.94 154383.888, 1029213.773
154450.439, 1029216.709 15447

1022227.52 152020.140, 1022071.970 151943.907, 1021009.223 151932.2, 1021730.431 151017.970, 1021507.070 151717.403, 1021514.576 151653.498, 1021436.905 151575.827, 1021327.252 151452.468, 1021176.479 151374.797)))

2 POLYGON ((1026308.77 256767.698, 1026495.593 256638.616, 1026567.23 256589.859, 1026729.235 256481.356, 1026817.315 256421.566, 1026902.63 256362.665, 1027071.239 256248.925, 1027152.98 256192.892, 1027279.263 256106.327, 1027331.842 256069.276, 1027510.63 255948.374, 1027629.334 255866.839, 1027727.008 255800.518, 1027837.261 255725.658, 1027990.791 255622.893, 1028164.77 255501.6, 1028325.383 255391.142, 1028574.982 255221.726, 1028629.697 255184.957, 1028719.395 255124.679, 1028849.175 255034.538, 1029100.755 254862.455, 1029342.691 254696.746, 1029393.087 254661.415, 1029559.878 254544.48, 1029671.113 254468.873, 1030824.581 253612.447, 1030852.405 253590.019, 1031025.925 253450.148, 1031239.895 253390.968, 1031348.95 253356.008, 1031409.114 253315.489, 1031483.402 253239.361, 1031517.125 253195.017, 1031711.068 252984.795, 1031732.164 252970.294, 1031696.321 252911.684, 1031600.727 252808.666, 1031426.289 252644.904, 1031382.67 252603.959, 1031347.411 252575.222, 1031309.373 252549.894, 1031269.065 252528.352, 1031227.07 252510.871, 1030989.187 252404.491, 1030747.934 252304.622, 1030423.381 252161.479, 1030311.045 252106.788, 1030065.061 251982.94, 1029800.679 251851.444, 1029536.742 251721.835, 1029390.141 251648.652, 1029273.432 251677.77, 1029014.594 251732.087, 1028755.864 251786.326, 1028497.128 251839.908, 1028241.072 251891.473, 1027981.135 251942.904, 1027597.953 252014.662, 1027240.569 252070.656, 1026981.183 252103.963, 1026722.775 252139.173, 1026463.018 252155.541, 1026203.08 252162.122, 1025934.246 252167.362, 1025665.018 252144.954, 1025403.296 252114.843, 1025142.017 252086.46, 1024883.126 252053.468, 1024621.838 252021.673, 1024124.197 251965.482, 1024079.719 251960.46, 1023907.305 251945.965, 1023708.45 252525.826, 1023696.775 252625.088, 1023677.807 252786.354, 1023675.068 252811.439, 1023673.198 252828.569, 1023661.015 252940.139, 1023658.86 252959.878, 1023648.496 253054.791, 1023639.278 253147.561, 1023636.562 253174.901, 1023619.245 253349.185, 1023616.993 253371.845, 1023613.348 253408.526, 1023611.491 253427.216, 1023609.492 253447.326, 1023608.155 253460.784, 1023581.843 253736.906, 1023510.698 253739.029, 1023403.254 254000.826, 1023329.85 254179.657, 1023275.042 254319.611, 1023080.017 254638.384, 1022820.509 254644.154, 1022540.66 254650.913, 1022545.939 254871.041, 1022556.179 255380.334, 1022557.209 255464.419, 1022573.977 256128, 1022588.079 256759.439, 1022648.324 256753.366, 1023128.081 256747.251, 1023387.923 256741.608, 1023647.568 256735.564, 1023906.091 256730.102, 1023922.354 257378.749, 1023932.226 257811.369, 1024137.681 257726.422, 1024207.681 257697.479, 1024487.071 257580.254, 1024728.307 257478.516, 1024744.944 257469.589, 1024999.212 257366.603, 1025261.475 257259.665, 1025434.52 257185.389, 1025516.11 257150.37, 1025768.902 257043.536, 1026123.091 256887.08, 1026308.77 256767.698)))

3 POLYGON ((992073.467 203714.076, 992068.667 203711.502, 992061.716 203711.772, 992049.866 203627.244, 992045.713 203598.15, 992042.802 203569.472, 992035.529 203520.637, 992029.295 203479.072, 992027.632 203472.421, 992021.605 2034 29.407, 992009.551 203357.087, 992000.824 203299.731, 991992.721 203254.22, 991991.057 203245.91, 991988.564 203231.779 , 991987.73 203220.972, 991983.99 203204.761, 991967.783 203112.494, 991965.08 203098.57, 991950.949 203025.004, 991946. 379 202994.663, 991937.648 202946.451, 991921.025 202855.014, 991911.041 202799.837, 991900.444 202740.359, 991869.824 202575.697, 991764.785 201989.606, 991721.35 201752.025, 991698.404 201626.31, 991675.94 201514.696, 991643.912 201360. 3, 991601.475 201178.24, 991277.856 201171.193, 991266.86 201170.954, 991251.469 201170.618, 991219.09 201175.96, 991204.179 201178.492, 991186.587 201181.478, 991153.667 201185.146, 991054.845 201209.096, 991038.641 201213.483, 990 877.195 201257.19, 990804.354 201276.911, 990769.265 201286.645, 990696.319 201306.885, 990563.577 201341.294, 990339.7 25 201460.946, 990139.804 201568.505, 989888.122 201670.138, 989431.256 201850.729, 989193.428 201945.43, 988961.418 20 2037.861, 988733.885 202128.97, 988746.067 202151.955, 988839.246 202327.781, 988972.11 202565.648, 989093.931 202786.6 94, 989216.421 203006.799, 989339.937 203229.59, 989457.788 203440.679, 989580.849 203663.966, 989701.137 203880.675, 9 89819.296 204093.995, 989940.471 204313.315, 990070.825 204545.815, 990199.037 204779.141, 990332.127 205011.253, 9903 40.007 205029.463, 990515.505 204931.862, 990651.195 204856.399, 990827.05 204758.6, 990859.74 204742.56, 990865.046 204740.034, 990956.65 204696.415, 990986.013 204680.601, 991650.915 204299.324, 991996.832 204100.12, 992017.155 20408 1.695, 992035.011 204067.366, 992050.462 204055.594, 992067.289 204039.203, 992114.154 204008.167, 992102.332 203922.13 5, 992095.102 203867.302, 992088.722 203823.64, 992083.334 203779.071, 992075.282 203718.457, 992073.467 203714.076)))

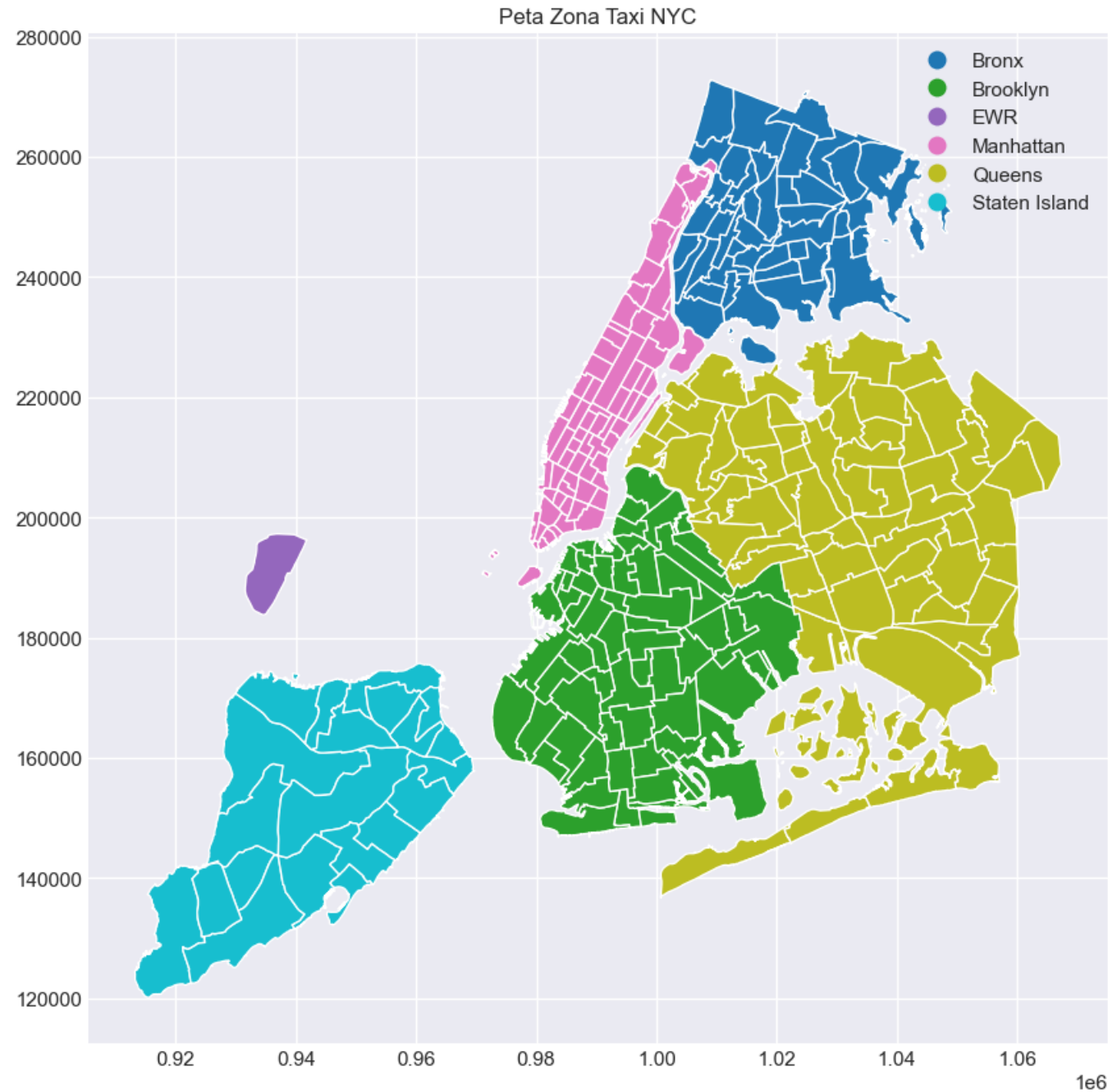
4 POLYGON ((935843.31 144283.336, 936046.565 144173.418, 936387.922 143967.756, 936481.134 143911.788, 936499.881 143900.531, 936394.5 143760.449, 936332.269 143670.365, 936119.914 143375.463, 936016.155 143237.504, 935797.149 14287 7.344, 935690.066 142680.573, 935604.82 142514.35, 935587.792 142481.148, 935490.424 142279.242, 935455.089 142201.129, 935399.391 142077.999, 935310.753 141868.728, 935170.195 141524.165, 934934.815 140949.058, 934741.354 140476.427, 934 576.741 140069.841, 934411.965 139666.682, 934250.777 139283.051, 934174.495 139117.496, 933867.698 138600.047, 933649. 654 138267.967, 933430.556 137940.891, 933196.99 137587.168, 932665.108 136780.949, 932392.464 136379.202, 932249.548 1 36158.667, 932168.045 136029.55, 932078.092 135896.657, 931920.425 135656.247, 931708.004 135399.2, 931574.635 135264.4 11, 931353.454 135067.38, 931061.466 134856.127, 930388.564 134554.848, 930298.791 134800.382, 930203.653 135037.167, 9 30136.688 135221.408, 930114.739 135283.216, 930027.938 135522.064, 929931.672 135774.393, 929838.014 136014.921, 9297 47.135 136255.445, 929661.837 136501.724, 929575.813 136747.39, 929490.653 136995.276, 929411.811 137263.304, 929107.50 1 138008.817, 929053.334 138145.002, 928910.815 138520.142, 928709.155 139039.295, 928632.528 139268.067, 928617.446 13 9321.574, 928400.094 140008.275, 928156.997 140700.794, 928071.302 140945.275, 928019.684 141090.642, 927942.359 14130 3.912, 927864.475 141523.109, 927766.539 141772.465, 927768.444 141799.37, 927771.789 141826.134, 927776.565 141852.681, 927782.757 141878.933, 927790.349 141904.816, 927799.317 141930.254, 927836.028 141918.021, 927873.106 141 906.952, 927910.514 141897.058, 927948.216 141888.347, 927986.174 141880.829, 928024.35 141874.512, 928060.367 141868.1 23, 928096.668 141863.619, 928133.153 141861.013, 928169.726 141860.311, 928206.285 141861.515, 928242.732 141864.623, 928278.967 141869.624, 928314.893 141876.507, 928350.411 141885.252, 928385.426 141895.836, 928419.842 141908.229, 928 453.565 141922.399, 928500.238 141954.314, 928544.437 141990.001, 928585.723 142029.157, 928623.699 142071.408, 928658. 036 142116.326, 928823.236 142335.725, 929317.037 142956.098, 930325.261 144116.558, 930384.542 144168.566, 930384.585 144168.602, 930384.627 144168.634, 930446.251 144217.837, 930446.32 144217.893, 930446.389 144217.945, 930510.277 1442 64.271, 930576.241 144307.591, 930602.554 144316.055, 930629.542 144322.024, 930656.97 144325.446, 930684.597 144326.29, 930712.183 144324.551, 930739.485 144320.241, 930799.567 144301.344, 930861.487 144287.243, 930924.662 1442 78.175, 930988.474 144274.284, 931052.283 144275.609, 931115.459 144282.082, 931177.392 144293.542, 931221.453 144311.4 92, 931263.549 144335.337, 931302.526 144364.697, 931337.342 144398.887, 931367.168 144436.981, 931391.453 144477.893, 931425.491 144522.167, 931464.3 144563.322, 931507.388 144600.589, 931507.42 144600.615, 931507.453 144600.638, 931554. 248 144633.4, 931604.044 144661.163, 931655.993 144683.548, 931709.241 144700.385, 932218.049 144827.688, 932355.263 14 4881.776, 932494.59 144930.506, 932635.656 144973.748, 932635.712 144973.764, 932635.768 144973.78, 932778.303 145011.4 51, 932843.211 145067.914, 932843.281 145067.97, 932843.353 145068.029, 932911.686 145120.883, 932983.152 145169.846, 9 32983.208 145169.882, 932983.26 145169.915, 933057.417 145214.653, 933057.502 145214.702, 933057.591 145214.751, 933134.107 145255.076, 933134.199 145255.122, 933134.294 145255.168, 933212.902 145290.962, 933293.201 145322.087, 933 353.949 145338.402, 933416.885 145349.239, 933481.258 145354.236, 933546.245 145353.17, 933568.246 145350.725, 933610.9 86 145345.875, 933674.624 145333.747, 933726.327 145313.727, 933705.245 145280.244, 933851.021 145260.880, 934000.500 1

80 145345.975, 933674.624 145352.747, 933756.527 145315.757, 933795.345 145289.344, 933851.051 145200.089, 934808.509 144801.038, 934909.776 144717.793, 935183.551 144529.936, 935515.148 144428.473, 935657.303 144377.824, 935843.31 144283.336))

Data Koordinat Berhasil Dibuat:

	LocationID	zone	borough	latitude \
0	1	Newark Airport	EWB	191376.749531
1	2	Jamaica Bay	Queens	164018.754403
2	3	Allerton/Pelham Gardens	Bronx	254265.478659
3	4	Alphabet City	Manhattan	202959.782391
4	5	Arden Heights	Staten Island	140681.351376

longitude
0 9.359968e+05
1 1.031086e+06
2 1.026453e+06
3 9.906340e+05
4 9.318714e+05



In [1226]:
df_koordinat.head()

Out[1226]:

	LocationID	zone	borough	latitude	longitude
0	1	Newark Airport	EWR	191376.749531	9.359968e+05
1	2	Jamaica Bay	Queens	164018.754403	1.031086e+06
2	3	Allerton/Pelham Gardens	Bronx	254265.478659	1.026453e+06
3	4	Alphabet City	Manhattan	202959.782391	9.906340e+05
4	5	Arden Heights	Staten Island	140681.351376	9.318714e+05

In [1227]:

```
# Daerah dengan waktu tersibuk
```

```
# Daerah tersibuk adalah daerah yang memiliki tarif dasar == 0.50
```

```
zona_sibuk = pd.pivot_table(df, values='lpep_pickup_datetime', index=['fare_amount', 'DO_Zone'], aggfunc='count')
```

Out[1227]:

lpep_pickup_datetime		
fare_amount	DO_Zone	
0.0	Astoria	5
	Baisley Park	3
	Bloomfield/Emerson Hill	2
	Central Park	1
	College Point	1
...	...	...
450.0	Flushing	1
	Queens Village	1
455.0	Flushing	1
459.9	Jamaica	1
490.0	Jamaica	1

10002 rows × 1 columns

• Jumlah perjalanan perhari

In [1228]:

```
# Kita buat kolom 'date' dan 'hour' untuk keperluan merging nanti
```

```
df['date'] = df['lpep_pickup_datetime'].dt.date
```

```
df['hour'] = df['lpep_pickup_datetime'].dt.hour
```

In [1229]:

```
df["date"].head()
```

Out[1229]:

```
0 2023-01-01
```

```
1 2023-01-01
```

```
2 2023-01-01
```

```
3 2023-01-01
```

```
4 2023-01-01
```

```
Name: date, dtype: object
```

In [1316]:

```
count_date = df.groupby(df["date"]).size().reset_index(name="Count")
```

```
count_date
```

Out[1316]:

	date	Count
0	2023-01-01	1266
1	2023-01-02	1442
2	2023-01-03	1963
3	2023-01-04	2236
4	2023-01-05	2220
5	2023-01-06	2365
6	2023-01-07	2003
7	2023-01-08	1553
8	2023-01-09	2083
9	2023-01-10	2024
10	2023-01-11	2318
11	2023-01-12	2345
12	2023-01-13	2343
13	2023-01-14	1993
14	2023-01-15	1658
15	2023-01-16	1460
16	2023-01-17	2094
17	2023-01-18	2175
18	2023-01-19	2416
19	2023-01-20	2266
20	2023-01-21	1977
21	2023-01-22	1637
22	2023-01-23	2208
23	2023-01-24	2252
24	2023-01-25	2475
25	2023-01-26	2476
26	2023-01-27	2241
27	2023-01-28	1891
28	2023-01-29	1553
29	2023-01-30	2078
30	2023-01-31	2164

In [1290]:

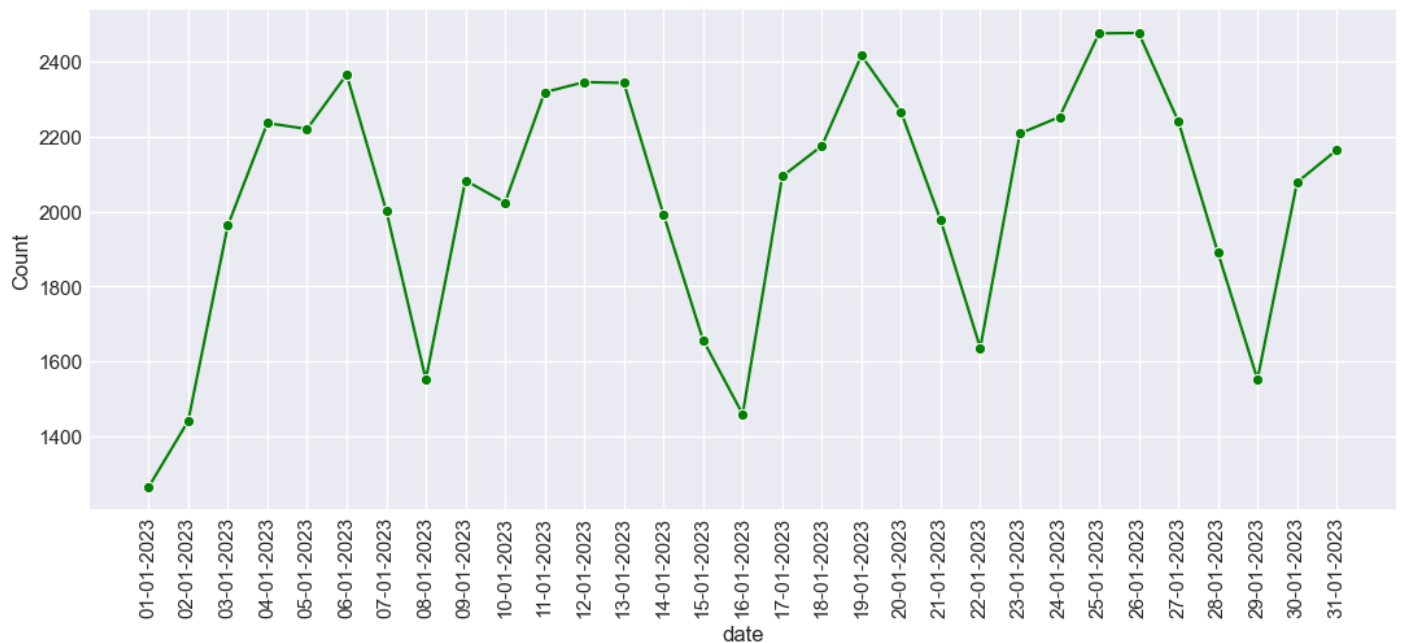
```
plt.figure(figsize=(13, 5))
sns.lineplot(data= count_date,
             x= "date",
             y = "Count",
             errorbar= None,
             marker = "o",
             ci = 0,
             color= "green")

plt.xticks(ticks = count_date["date"],
           rotation = 90,
           labels= [dt.strftime('%d-%m-%Y') for dt in count_date['date']] )
plt.show()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\2409766815.py:2: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=('ci', 0)` for the same effect.

```
sns.lineplot(data= count_date,
```



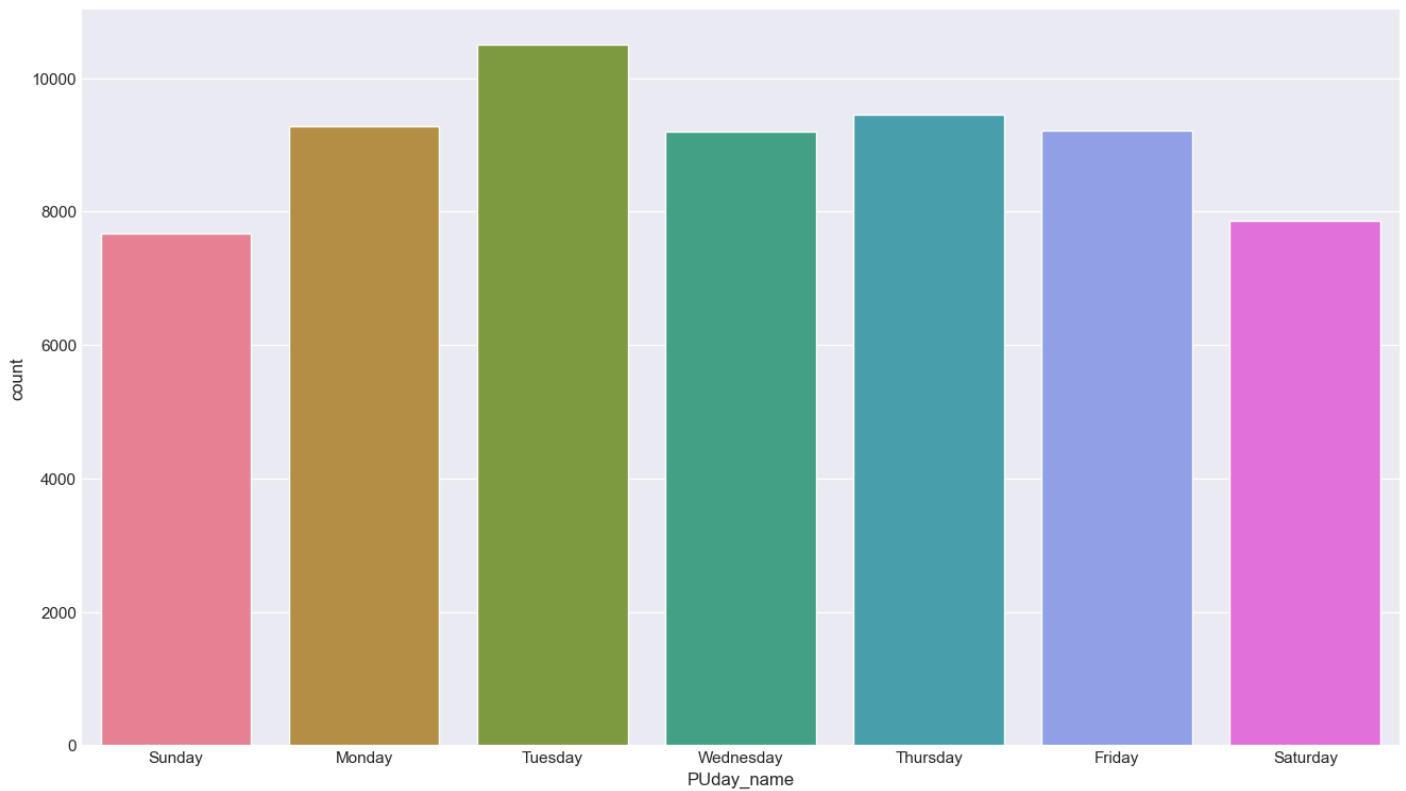
Terdapat tren yang cukup jelas dalam pemesanan taksi, pemesanan taksi akan naik di hari kerja dan turun pada hari libur.

In [1232]:

```
# Jumlah perjalanan perhari
```

```
df["PUday_name"] = df['lpep_pickup_datetime'].dt.day_name()
```

```
plt.figure(figsize=(16,9))
sns.countplot(data= df, x= "PUday_name", hue= "PUday_name")
plt.show()
```



Jumlah pemesanan taxi paling banyak ada pada hari selasa bukan di hari minggu

- **Jumlah perjalanan perjam**

```
In [1233]:
```

```
df["hour"].unique()
```

```
Out[1233]:
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  
       17, 18, 19, 20, 21, 22, 23], dtype=int32)
```

```
In [1318]:
```

```
count_hour = df.groupby(df["hour"]).size().reset_index(name="Count")
```

```
count_hour
```

Out[1318]:

	hour	Count
0	0	1021
1	1	791
2	2	568
3	3	528
4	4	389
5	5	375
6	6	876
7	7	2379
8	8	2870
9	9	3162
10	10	3297
11	11	3415
12	12	3499
13	13	3717
14	14	4104
15	15	4836
16	16	4968
17	17	4914
18	18	4928
19	19	4041
20	20	3085
21	21	2262
22	22	1779
23	23	1371

In [1235]:

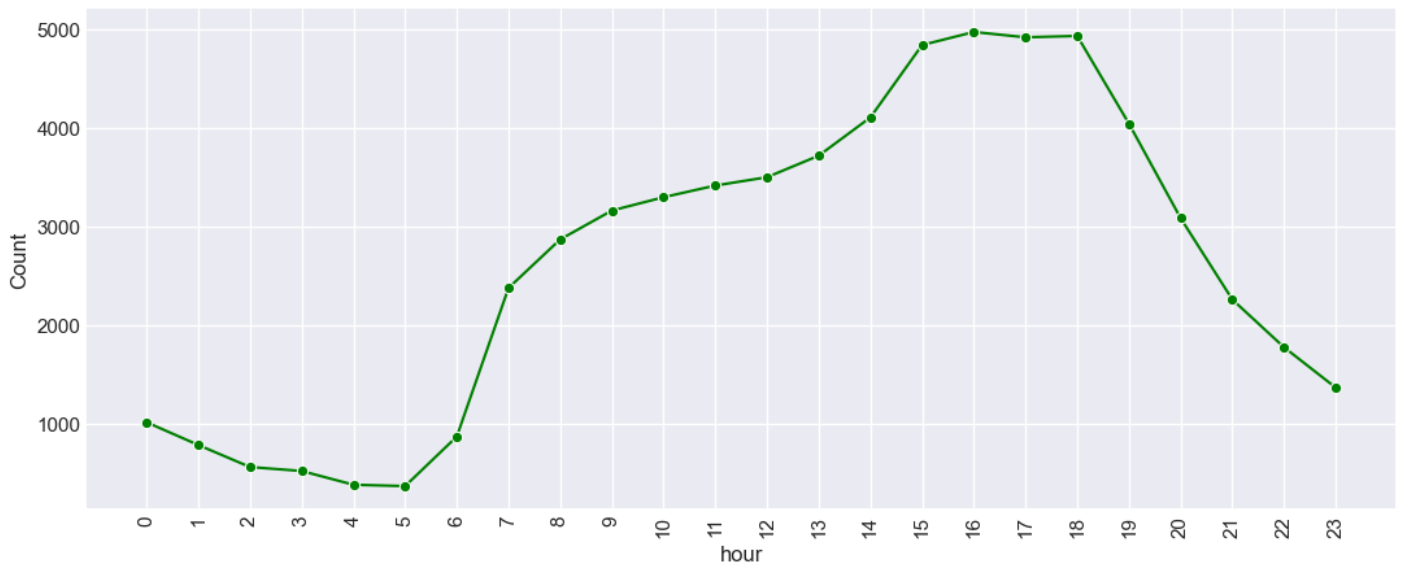
```
plt.figure(figsize=(13, 5))
sns.lineplot(data= count_hour,
              x= "hour",
              y = "Count",
              errorbar= None,
              marker = "o",
              ci = 0,
              color = "green")

plt.xticks(ticks = count_hour["hour"],
           rotation = 90)
plt.show()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\3801437325.py:2: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=('ci', 0)` for the same effect.

```
sns.lineplot(data= count_hour,
```



Jam 16.00 Adalah jam sibuk untuk pemesanan taksi

- **Jumlah penumpang perhari**

In [1317]:

```
count_passenger = df.groupby(df["date"])[["passenger_count"]].sum().reset_index()  
count_passenger
```

Out[1317]:

	date	passenger_count
0	2023-01-01	1769.0
1	2023-01-02	1895.0
2	2023-01-03	2701.0
3	2023-01-04	2838.0
4	2023-01-05	3019.0
5	2023-01-06	3237.0
6	2023-01-07	2573.0
7	2023-01-08	2073.0
8	2023-01-09	2753.0
9	2023-01-10	2580.0
10	2023-01-11	3063.0
11	2023-01-12	2994.0
12	2023-01-13	3159.0
13	2023-01-14	2623.0
14	2023-01-15	2215.0
15	2023-01-16	1850.0
16	2023-01-17	2783.0
17	2023-01-18	2810.0
18	2023-01-19	3151.0
19	2023-01-20	2961.0
20	2023-01-21	2614.0
21	2023-01-22	2146.0
22	2023-01-23	2939.0
23	2023-01-24	2910.0
24	2023-01-25	3207.0
25	2023-01-26	3251.0
26	2023-01-27	3013.0
27	2023-01-28	2494.0
28	2023-01-29	2094.0
29	2023-01-30	2675.0
30	2023-01-31	2816.0

In [1237]:

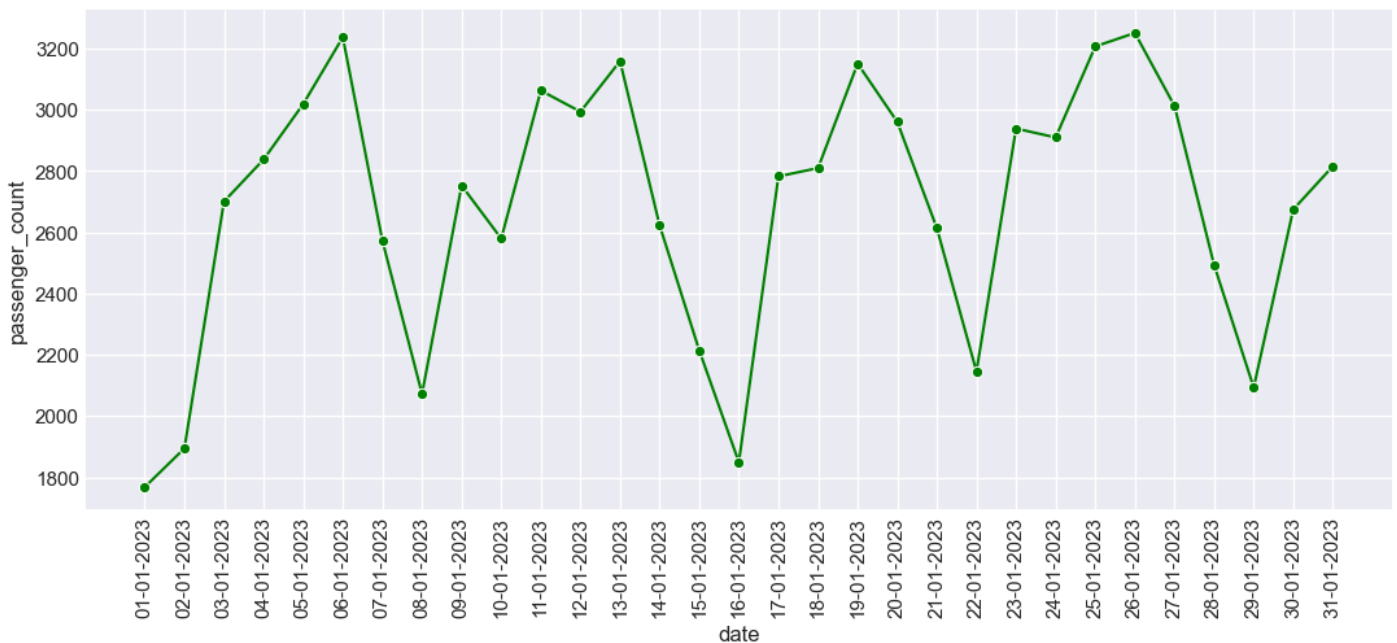
```
plt.figure(figsize=(13, 5))
sns.lineplot(data= count_passenger,
             x= "date",
             y = "passenger_count",
             errorbar= None,
             marker = "o",
             ci = 0,
             color = "green")

plt.xticks(ticks = count_passenger["date"],
           rotation = 90,
           labels= [dt.strftime('%d-%m-%Y') for dt in count_passenger['date']] )
plt.show()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\2011701612.py:2: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=('ci', 0)` for the same effect.

```
sns.lineplot(data= count_passenger,
```



Terdapat pola yang sama dengan total perjalanan perhari, jumlah penumpang perhari mengalami kenaikan pada hari kerja dan mengalami penurunan pada hari libur.

• Jumlah perjalanan by PUlocation

In [1238]:

```
top10_PUzone = df["PU_Zone"].value_counts().reset_index().sort_values(by= "count", ascending= False).head
```

In [1239]:

```

# 1. Menyiapkan Warna Baby Pink Gradasi
# Kita buat gradasi dari Baby Pink (#FFB6C1) ke Deep Pink/Ungu (#C71585)
# as_cmap=False agar jadi list warna, n_colors=10 sesuai jumlah data
colors = sns.blend_palette(["#98FA8D", "#F5FD08DD"], n_colors=10)

# Jika ingin yang Paling Atas (Terbanyak) berwarna Gelap, dan Bawah Baby Pink:
colors = colors[::-1]
# (Hapus baris "colors = colors[::-1]" jika ingin yang paling atas justru Baby Pink muda)

# 2. Setup Canvas
plt.figure(figsize=(13, 8))

# 3. Plotting
ax = sns.barplot(
    x='count',
    y='PU_Zone',
    data=top10_PUzone,
    palette=colors
)

# 4. CARA MANUAL (ANTI GAGAL) UNTUK MENAMPILKAN LABEL
# Kita iterasi setiap 'patch' (batang) yang ada di gambar
for p in ax.patches:
    # Ambil lebar batang (ini adalah nilai jumlah transaksinya)
    width = p.get_width()

    # Ambil posisi Y (tinggi batang) untuk menaruh teks di tengah-tengah batang
    y_position = p.get_y() + p.get_height() / 2

    # Tulis Teksnya
    # xy=(width, y_position) -> Posisi teks
    # xytext=(5, 0) -> Geser sedikit 5 pixel ke kanan agar tidak nempel batang
    if width > 0: # Hanya tulis jika nilainya ada
        ax.annotate(f'{int(width)}',
                    xy=(width, y_position),
                    xytext=(5, 0),
                    textcoords="offset points",
                    ha='left', va='center',
                    fontsize=11, fontweight='bold', color='black')

# 5. Memperlebar Batas Kanan
# Ini WAJIB agar angka tidak terpotong garis pinggir gambar
max_val = top10_PUzone['count'].max()
plt.xlim(0, max_val * 1.2)

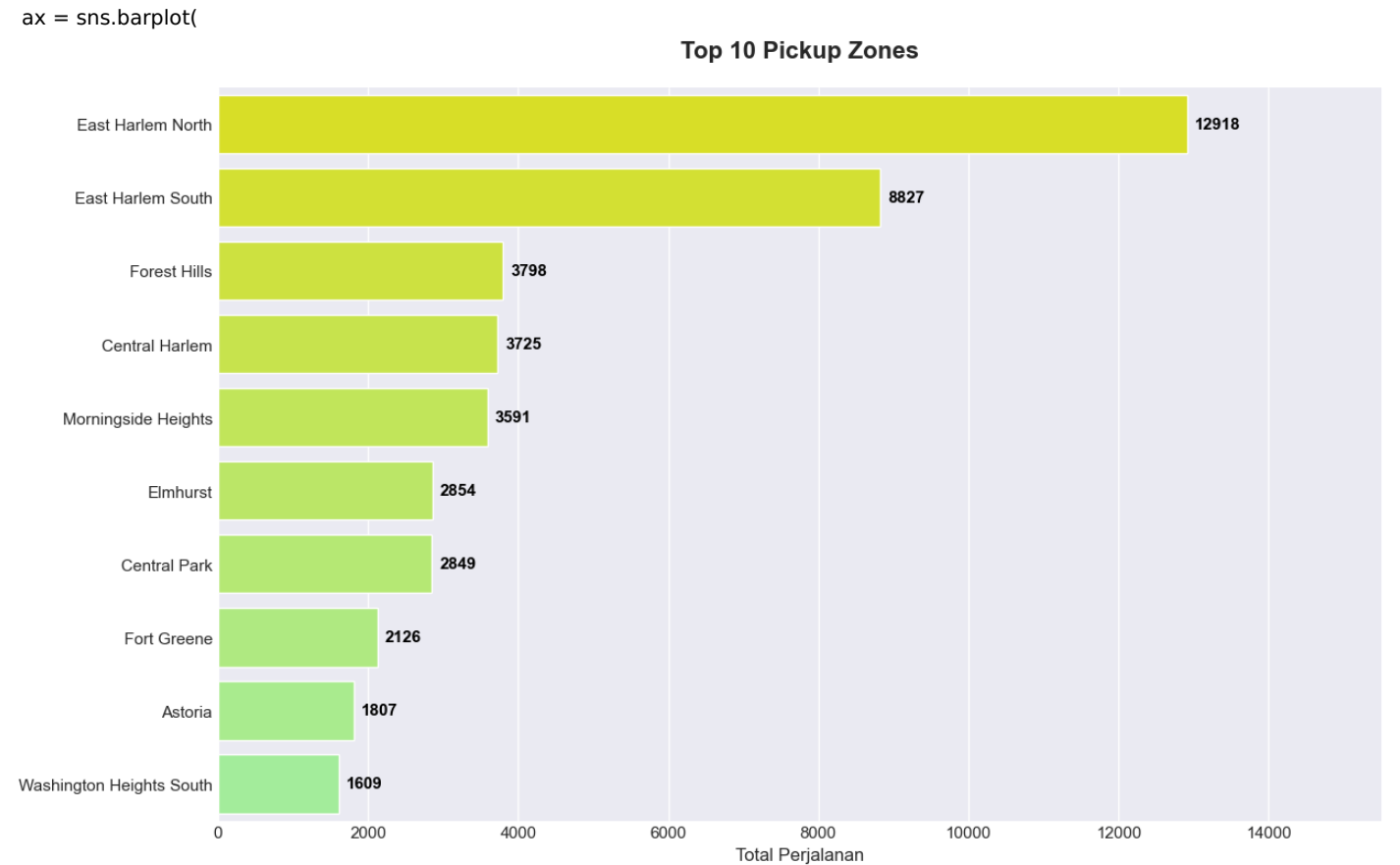
# 6. Estetika Akhir
plt.title('Top 10 Pickup Zones', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Total Perjalanan', fontsize=12)
plt.ylabel('') # Kosongkan label Y karena nama daerah sudah jelas
sns.despine(left=True, bottom=False) # Hapus garis kotak

plt.tight_layout()
plt.show()

```

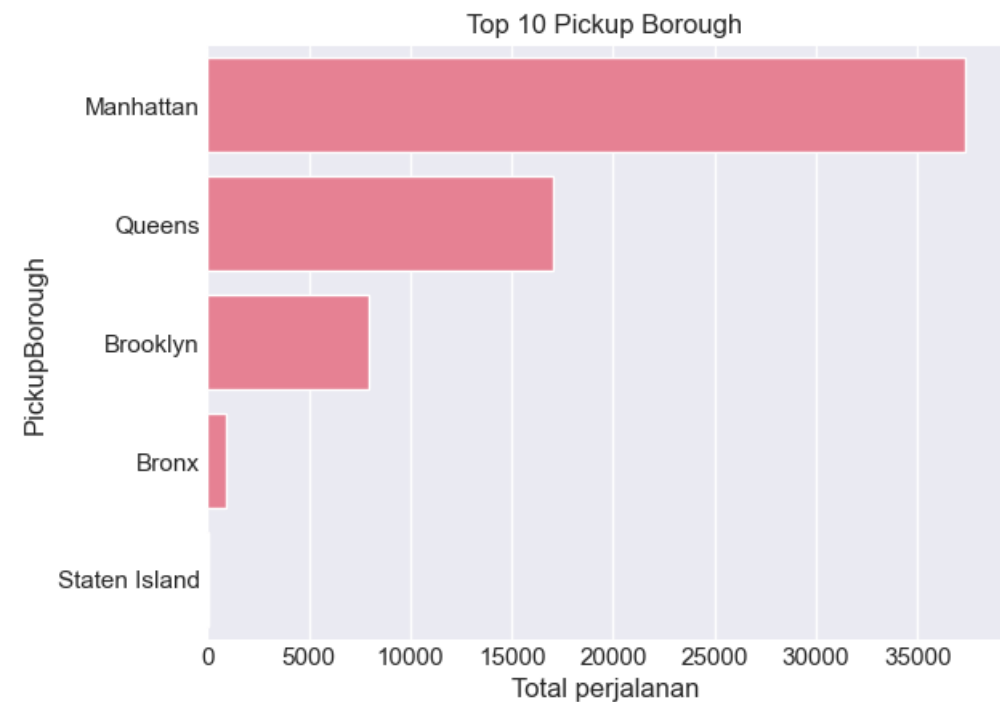
C:\Users\HP\AppData\Local\Temp\ipykernel_27448\1509856722.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.



```
In [1240]:  
top10_PUborough = df["PU_Borough"].value_counts().reset_index().sort_values(by= "count", ascending= False)  
In [1241]:  
sns.barplot(x='count', y='PU_Borough', data=top10_PUborough, errorbar=None )
```

```
plt.title('Top 10 Pickup Borough')  
plt.xlabel('Total perjalanan')  
plt.ylabel('PickupBorough')  
plt.show()
```



- **Jumlah perjalanan by DOlocation**

```
In [1242]:
top10_DOzone = df["DO_Zone"].value_counts().reset_index().sort_values(by= "count", ascending= False).he
In [ ]:
# 1. Menyiapkan Warna Baby Pink Gradasi
# Kita buat gradasi dari Baby Pink (#FFB6C1) ke Deep Pink/Ungu (#C71585)
# as_cmap=False agar jadi list warna, n_colors=10 sesuai jumlah data

# Jika ingin yang Paling Atas (Terbanyak) berwarna Gelap, dan Bawah Baby Pink:
colors = colors[::-1]
# (Hapus baris "colors = colors[::-1]" jika ingin yang paling atas justru Baby Pink muda)

# 2. Setup Canvas
plt.figure(figsize=(13, 8))

# 3. Plotting
ax = sns.barplot(
    x='count',
    y='DO_Zone',
    data= top10_DOzone,
    palette=colors
)

# 4. CARA MANUAL (ANTI GAGAL) UNTUK MENAMPILKAN LABEL
# Kita iterasi setiap 'patch' (batang) yang ada di gambar
for p in ax.patches:
    # Ambil lebar batang (ini adalah nilai jumlah transaksinya)
    width = p.get_width()

    # Ambil posisi Y (tinggi batang) untuk menaruh teks di tengah-tengah batang
    y_position = p.get_y() + p.get_height() / 2

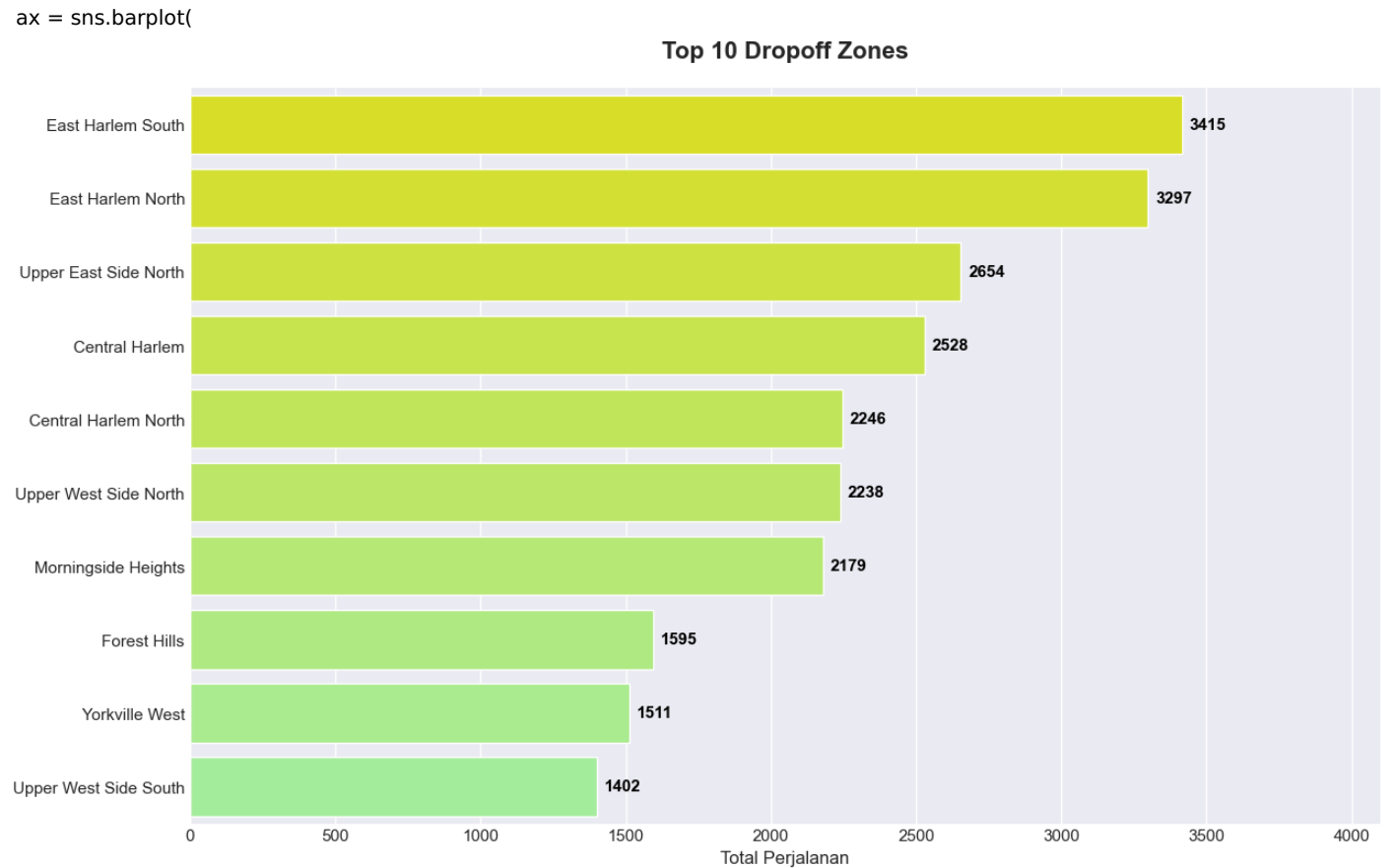
    # Tulis Teksnya
    # xy=(width, y_position) -> Posisi teks
    # xytext=(5, 0) -> Geser sedikit 5 pixel ke kanan agar tidak nempel batang
    if width > 0: # Hanya tulis jika nilainya ada
        ax.annotate(f'{int(width)}',
                    xy=(width, y_position),
                    xytext=(5, 0),
                    textcoords="offset points",
                    ha='left', va='center',
                    fontsize=11, fontweight='bold', color='black')

# 5. Memperlebar Batas Kanan
# Ini WAJIB agar angka tidak terpotong garis pinggir gambar
max_val = top10_DOzone['count'].max()
plt.xlim(0, max_val * 1.2)

# 6. Estetika Akhir
plt.title('Top 10 Dropoff Zones', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Total Perjalanan', fontsize=12)
plt.ylabel('') # Kosongkan label Y karena nama daerah sudah jelas
sns.despine(left=True, bottom=False) # Hapus garis kotak

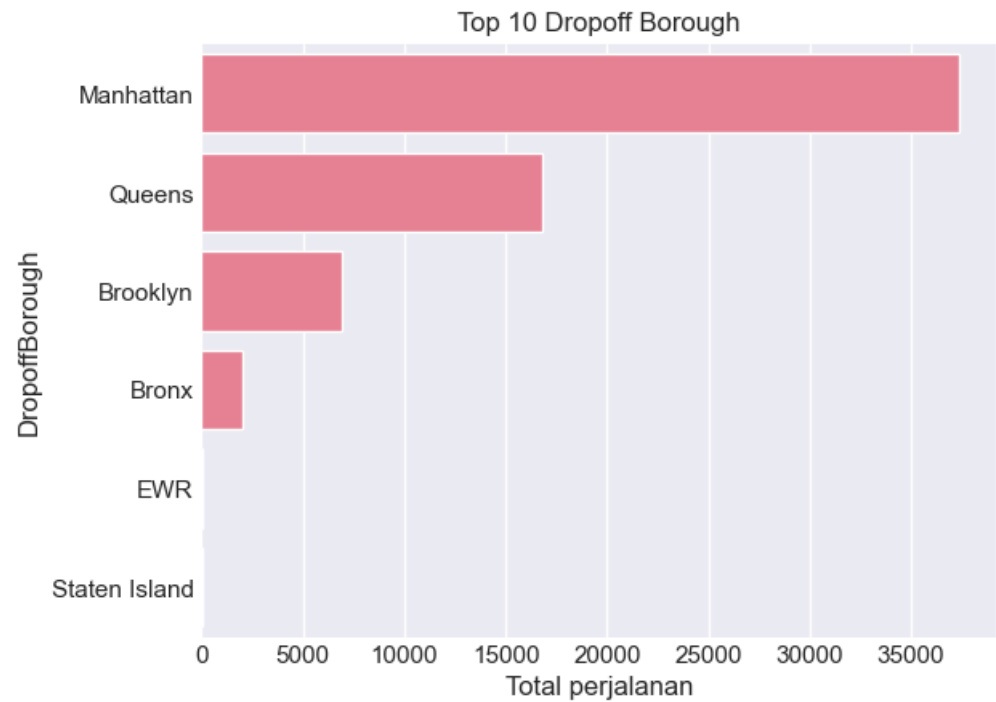
plt.tight_layout()
plt.show()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_27448\3154289206.py:14: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.



```
In [1244]:
top10_DOBorough = df["DO_Borough"].value_counts().reset_index().sort_values(by= "count", ascending= False)
In [1245]:
sns.barplot(x='count', y='DO_Borough', data=top10_DOBorough, errorbar=None )
```

```
plt.title('Top 10 Dropoff Borough')
plt.xlabel('Total perjalanan')
plt.ylabel('DropoffBorough')
plt.show()
```



c. Berdasarkan pendapatan

Analisis

1. Pendapaan dan transaksi terbanyak berdasarkan RatecodeID
2. Pendapaan dan transaksi terbanyak berdasarkan trip_type
3. Pendapaan dan transaksi terbanyak berdasarkan payment_type
4. Pendapaan dan transaksi terbanyak berdasarkan PU_Zone
5. Pendapaan dan transaksi terbanyak berdasarkan DO_Zone
6. Pendapatan perhari & perminggu --> hari yang paling banyak ada di PU_Zone dan DO_Zone mana

- **Pendapaan dan transaksi terbanyak berdasarkan RatecodeID**

In [1246]:

```
Total = df.groupby("RatecodeID")["total_amount"].sum().sort_values(by="total_amount", ascending= False)
```

Total

Out[1246]:

	RatecodeID	total_amount
0	Standard rate	1271163.38
1	Negotiated fare	43239.05
2	JFK	13154.20
3	Newark	1548.50
4	Nassau or Westchester	195.05

In [1247]:

```
Banyaknya = df.groupby("RatecodeID")["total_amount"].count().sort_values(by="total_amount", ascending=
```

Banyaknya

Out[1247]:

	RatecodeID	total_amount
0	Standard rate	61722
1	Negotiated fare	1271
2	JFK	155
3	Newark	24
4	Nassau or Westchester	3

In [1248]:

```
amount_RatecodeID = Total.merge(Banyaknya, left_on= "RatecodeID", right_on="RatecodeID", suffixes= ("_amount_RatecodeID", "_Banyaknya"))
```

Out[1248]:

	RatecodeID	total_amount_SUM	total_amount_COUNT
0	Standard rate	1271163.38	61722
1	Negotiated fare	43239.05	1271
2	JFK	13154.20	155
3	Newark	1548.50	24
4	Nassau or Westchester	195.05	3

In [1249]:

```
# Mengubah kategori 'JFK', 'Nassau or Westchester', 'Newark' menjadi 'Others'
```

```
amount_RatecodeID['RatecodeID'] = amount_RatecodeID['RatecodeID'].apply(lambda x: x if (x == 'Standard' or x == 'Nassau or Westchester') else 'Others')
amount_RatecodeID = amount_RatecodeID.groupby('RatecodeID').agg(total_amount_SUM = ('total_amount_SUM', 'sum'), total_amount_COUNT = ('total_amount_COUNT', 'count'))
```

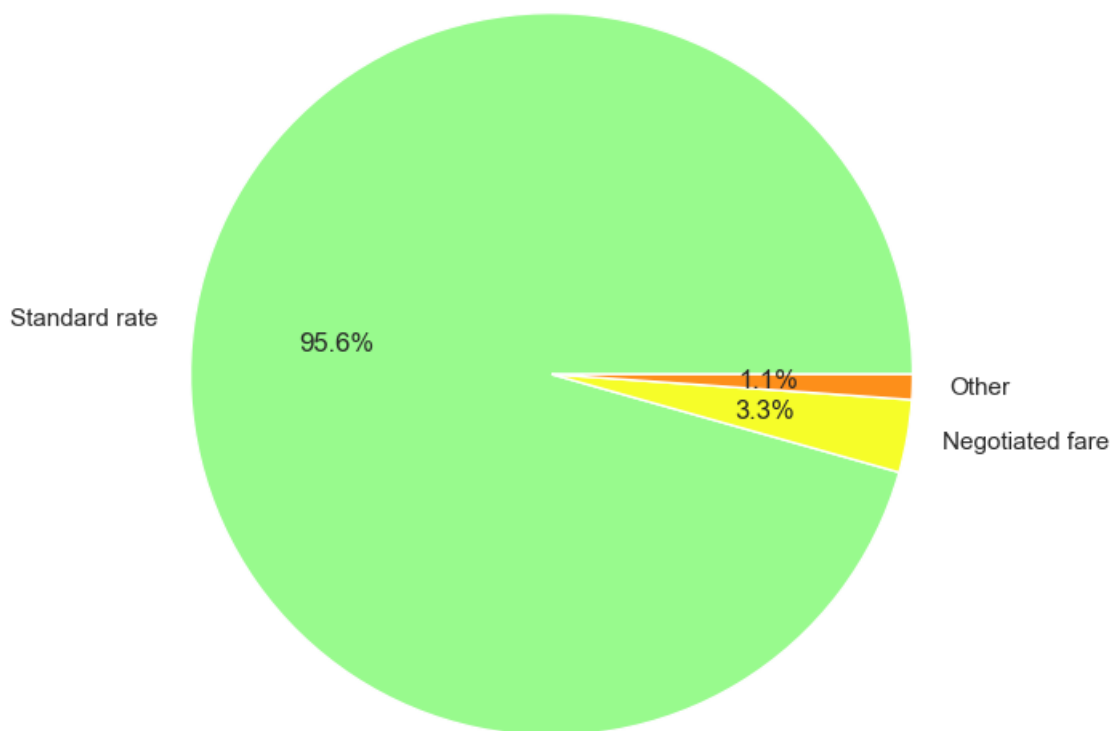
Out[1249]:

	RatecodeID	total_amount_SUM	total_amount_COUNT
0	Standard rate	1271163.38	61722
1	Negotiated fare	43239.05	1271
2	Other	14897.75	182

In [1292]:

```
plt.figure(figsize=(16,8))
plt.subplot(1,2,1)
plt.pie(
    x = amount_RatecodeID['total_amount_SUM'],
    labels= amount_RatecodeID['RatecodeID'],
    autopct= '%.1f%%',
    colors= ["#98FA8D", "#F5FD08DD", "#FD8708ED"]
)
plt.title('Pendapatan Total Berdasarkan Kode Tarif')
plt.show()
```

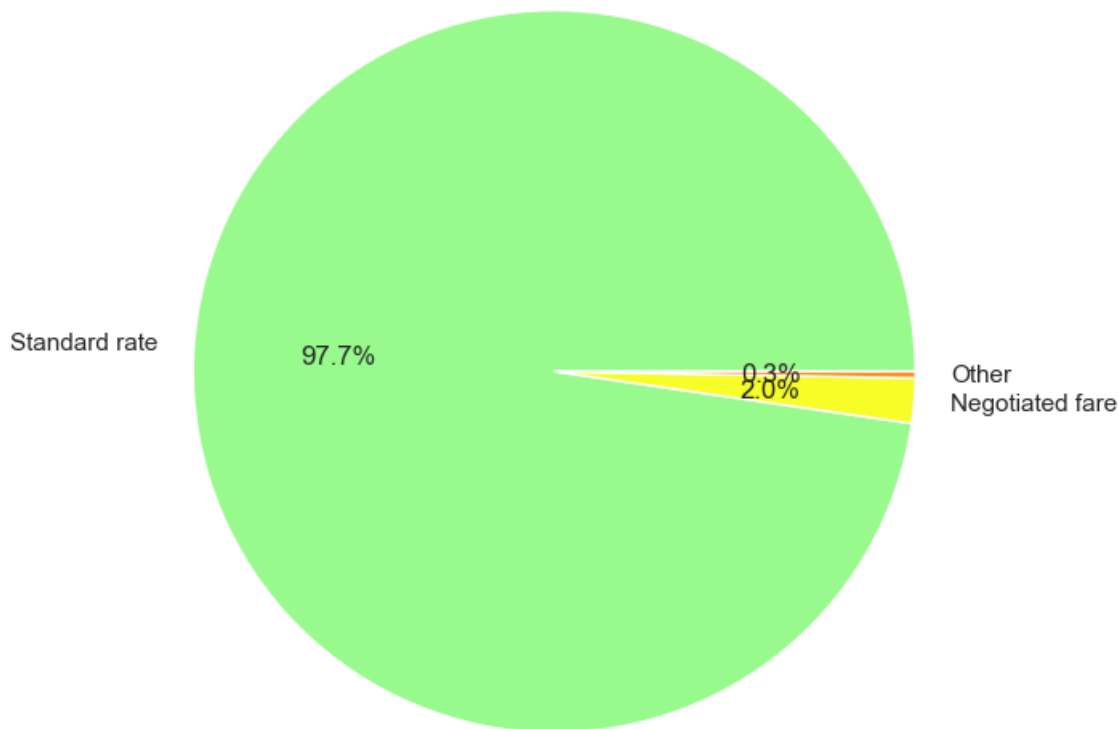
Pendapatan Total Berdasarkan Kode Tarif



In [1295]:

```
plt.figure(figsize=(16,8))
plt.subplot(1,2,2)
plt.pie(
    x = amount_RatecodeID['total_amount_COUNT'],
    labels= amount_RatecodeID['RatecodeID'],
    autopct= '%.1f%%',
    colors= ["#98FA8D", "#F5FD08DD", "#FD8708ED"])
plt.title('Banyaknya Transaksi Perjalanan Berdasarkan Kode Tarif')
plt.show()
```

Banyaknya Transaksi Perjalanan Berdasarkan Kode Tarif



• Pendapaan dan transaksi terbanyak berdasarkan trip_type

In [1252]:

```
Total = df.groupby("trip_type")["total_amount"].sum().sort_values(by="total_amount", ascending=False).r
Banyaknya = df.groupby("trip_type")["total_amount"].count().sort_values(by="total_amount", ascending=False)
amount_trip_type = Total.merge(Banyaknya, left_on="trip_type", right_on="trip_type", suffixes=("_SUM", "_COUNT"))
```

Out[1252]:

	trip_type	total_amount_SUM	total_amount_COUNT
0	Street-hail	1289237.90	62029
1	Dispatch	40062.28	1146

In [1296]:

```
plt.figure(figsize=(20,6))
```

```
plt.subplot(1,2,1)
```

```
plt.pie(
```

```
    x = amount_trip_type['total_amount_SUM'],
    labels= amount_trip_type['trip_type'],
    autopct= '%.1f%%',
    colors= ["#98FA8D", "#F5FD08DD", "#FD8708ED"]
)
```

```
plt.title('Pendapatan Total Berdasarkan Jenis Perjalanan')
```

```
plt.subplot(1,2,2)
```

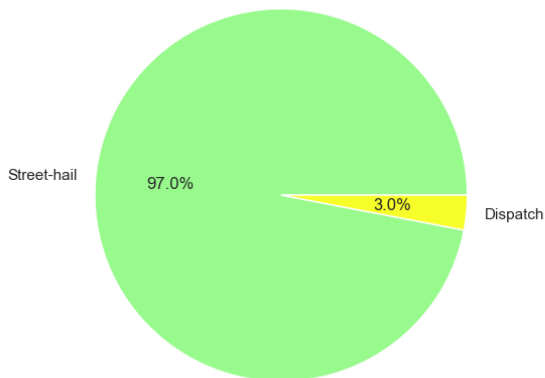
```
plt.pie(
```

```
    x = amount_trip_type['total_amount_COUNT'],
    labels= amount_trip_type['trip_type'],
    autopct= '%.1f%%',
    colors= ["#98FA8D", "#F5FD08DD", "#FD8708ED"]
)
```

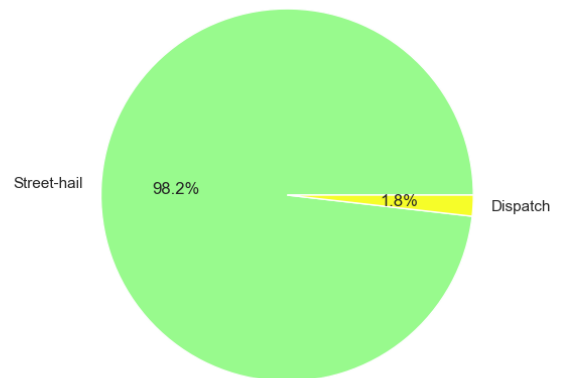
```
plt.title('Banyaknya Transaksi Perjalanan Berdasarkan Jenis Perjalanan')
```

```
plt.show()
```

Pendapatan Total Berdasarkan Jenis Perjalanan



Banyaknya Transaksi Perjalanan Berdasarkan Jenis Perjalanan



- **Pendapaan dan transaksi terbanyak berdasarkan payment_type**

In [1254]:

```
Total = df.groupby("payment_type")["total_amount"].sum().sort_values(by="total_amount", ascending=False)
Banyaknya = df.groupby("payment_type")["total_amount"].count().sort_values(by="total_amount", ascending=False)
amount_payment_type = Total.merge(Banyaknya, left_on="payment_type", right_on="payment_type", suffixes=("_SUM", "_COUNT"))
```

Out[1254]:

	payment_type	total_amount_SUM	total_amount_COUNT
0	Credit card	918444.26	40247
1	Cash	405401.97	22374
2	No charge	4614.10	460
3	Dispute	810.35	92
4	Unknown	29.50	2

In [1255]:

```
# Mengubah kategori sisanya menjadi 'Others'
```

```
amount_payment_type['payment_type'] = amount_payment_type['payment_type'].apply(lambda x: x if (x != 'Credit card' and x != 'Cash' and x != 'No charge' and x != 'Dispute' and x != 'Unknown') else 'Others')
amount_payment_type = amount_payment_type.groupby('payment_type').agg(total_amount_SUM = ('total_amount_SUM', 'sum'), total_amount_COUNT = ('total_amount_COUNT', 'count'))
```

Out[1255]:

	payment_type	total_amount_SUM	total_amount_COUNT
0	Credit card	918444.26	40247
1	Cash	405401.97	22374
2	Other	5453.95	554

In [1297]:

```
plt.figure(figsize=(20,6))
```

```
plt.subplot(1,2,1)
```

```
plt.pie(
```

```
    x = amount_payment_type['total_amount_SUM'],
```

```
    labels= amount_payment_type['payment_type'],
```

```
    autopct= '%.1f%%',
```

```
    colors= ["#98FA8D", "#F5FD08DD", "#FD8708ED"]
```

```
)
```

```
plt.title('Pendapatan Total Berdasarkan Cara Pembayaran')
```

```
plt.subplot(1,2,2)
```

```
plt.pie(
```

```
    x = amount_payment_type['total_amount_COUNT'],
```

```
    labels= amount_payment_type['payment_type'],
```

```
    autopct= '%.1f%%',
```

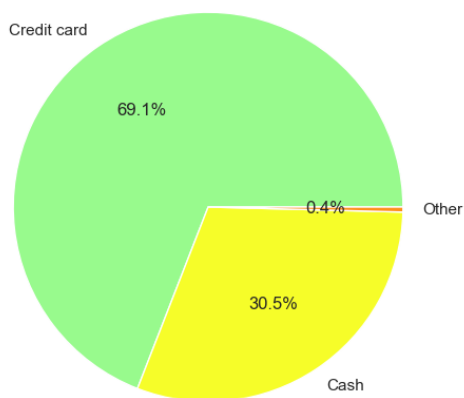
```
    colors= ["#98FA8D", "#F5FD08DD", "#FD8708ED"]
```

```
)
```

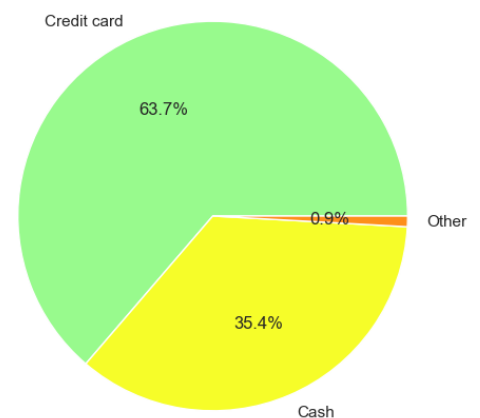
```
plt.title('Banyaknya Transaksi Perjalanan Berdasarkan Cara Pembayaran')
```

```
plt.show()
```

Pendapatan Total Berdasarkan Cara Pembayaran



Banyaknya Transaksi Perjalanan Berdasarkan Cara Pembayaran



- **Pendapaan dan transaksi terbanyak berdasarkan PU_Zone**

In [1257]:

```
Total = df.groupby("PU_Zone")["total_amount"].sum().sort_values(by="total_amount", ascending= False).r
```

```
Banyaknya = df.groupby("PU_Zone")["total_amount"].count().sort_values(by="total_amount", ascending= F
```

```
amount_PU_Zone = Total.merge(Banyaknya, left_on= "PU_Zone", right_on="PU_Zone", suffixes= ("_SUM", "_
```

```
amount_PU_Zone
```

Out[1257]:

	PU_Zone	total_amount_SUM	total_amount_COUNT
0	East Harlem North	247301.33	12918
1	East Harlem South	175151.94	8827
2	Morningside Heights	75269.09	3591
3	Forest Hills	71526.42	3798
4	Central Harlem	64908.75	3725
5	Central Park	63135.40	2849
6	Elmhurst	60155.25	2854
7	Washington Heights South	48969.20	1609
8	Fort Greene	45325.57	2126
9	Astoria	35672.52	1807

In [1305]:

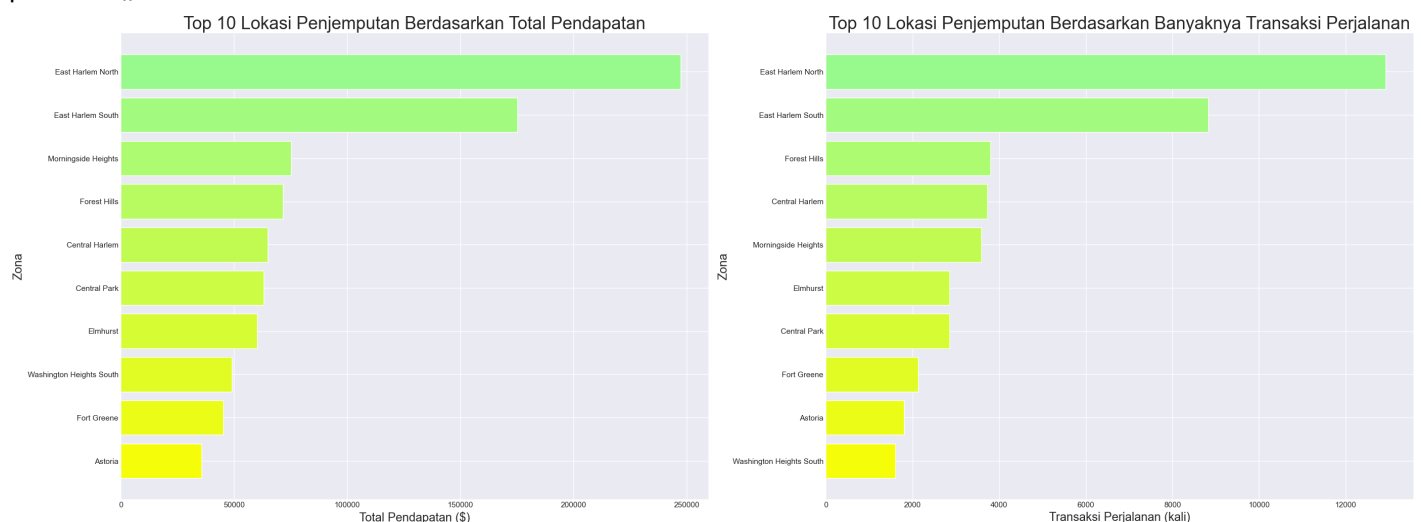
```
plt.figure(figsize=(33,12))
plt.subplot(1,2,1)
plt.barh(y=amount_PU_Zone.sort_values(by='total_amount_SUM')['PU_Zone'],
         width=amount_PU_Zone.sort_values(by='total_amount_SUM')['total_amount_SUM'],
         color = sns.blend_palette([ "#F5FD08DD", "#98FA8D"], n_colors=10))

plt.title('Top 10 Lokasi Penjemputan Berdasarkan Total Pendapatan', fontsize=25)
plt.ylabel('Zona', fontsize=17)
plt.xlabel('Total Pendapatan ($) ', fontsize=17)

plt.subplot(1,2,2)
plt.barh(y=amount_PU_Zone.sort_values(by='total_amount_COUNT')['PU_Zone'],
         width=amount_PU_Zone.sort_values(by='total_amount_COUNT')['total_amount_COUNT'],
         color=sns.blend_palette([ "#F5FD08DD", "#98FA8D"], n_colors=10))

plt.title('Top 10 Lokasi Penjemputan Berdasarkan Banyaknya Transaksi Perjalanan', fontsize=25)
plt.ylabel('Zona', fontsize=17)
plt.xlabel('Transaksi Perjalanan (kali)', fontsize=17)

plt.show()
```



Daerah penjemputan dengan total pendapatan tertinggi adalah East Herlem North sebesar 247301.33 dollar dengan 12918 transaksi perjalanan

- **Pendapaan dan transaksi terbanyak berdasarkan DO_Zone**

In [1259]:

```
Total = df.groupby("DO_Zone")[["total_amount"]].sum().sort_values(by="total_amount", ascending= False).r
Banyaknya = df.groupby("DO_Zone")[["total_amount"]].count().sort_values(by="total_amount", ascending= F
amount_DO_Zone = Total.merge(Banyaknya, left_on= "DO_Zone", right_on="DO_Zone", suffixes= ("_SUM", "
amount_DO_Zone
```

Out[1259]:

	DO_Zone	total_amount_SUM	total_amount_COUNT
0	Upper East Side North	51021.58	2654
1	East Harlem South	46202.96	3415
2	Upper West Side North	43209.06	2238
3	East Harlem North	41807.53	3297
4	Upper West Side South	34081.07	1402
5	Morningside Heights	32741.43	2179
6	Central Harlem	30612.31	2528
7	Yorkville West	30300.45	1511
8	Central Harlem North	30043.50	2246

In [1315]:

```
plt.figure(figsize=(33,12))
```

```
plt.subplot(1,2,1)
```

```
plt.barh(y=amount_DO_Zone.sort_values(by='total_amount_SUM')['DO_Zone'],
         width=amount_DO_Zone.sort_values(by='total_amount_SUM')['total_amount_SUM'],
         color=sns.blend_palette([ "#F5FD08DD", "#98FA8D"], n_colors=10))
```

```
plt.title('Top 10 Lokasi Pengantaran Berdasarkan Total Pendapatan', fontsize=25)
```

```
plt.ylabel('Zona', fontsize=17)
```

```
plt.xlabel('Total Pendapatan ($)') , fontsize=17)
```

```
plt.subplot(1,2,2)
```

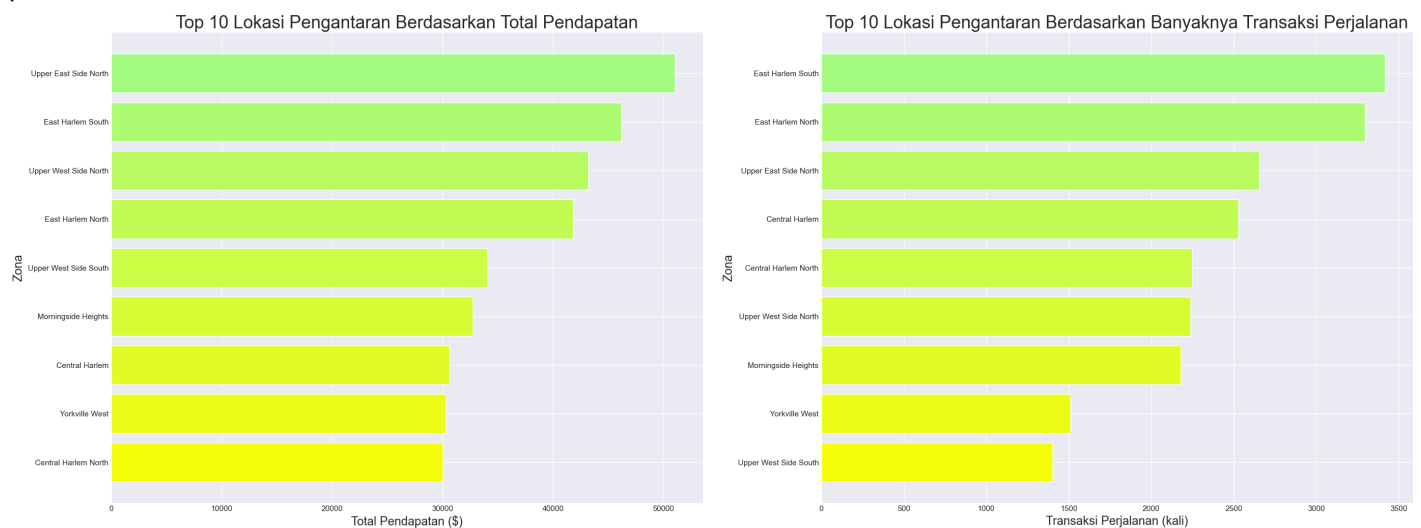
```
plt.barh(y=amount_DO_Zone.sort_values(by='total_amount_COUNT')['DO_Zone'],
         width=amount_DO_Zone.sort_values(by='total_amount_COUNT')['total_amount_COUNT'],
         color=sns.blend_palette([ "#F5FD08DD", "#98FA8D"], n_colors=10))
```

```
plt.title('Top 10 Lokasi Pengantaran Berdasarkan Banyaknya Transaksi Perjalanan', fontsize=25)
```

```
plt.ylabel('Zona', fontsize=17)
```

```
plt.xlabel('Transaksi Perjalanan (kali)', fontsize=17)
```

```
plt.show()
```



Daerah pengantaran dengan total pendapatan tertinggi adalah Upper East Side North sebesar 51021.58 dollar dengan 2654 transaksi perjalanan

In [1261]:

```
df["DOday_name"] = df['lpep_dropoff_datetime'].dt.day_name()
```

In [1262]:

Top 10 lokasi penjemputan

```
penjemputan = df.groupby(by=['PU_Zone'])[['total_amount']].count().reset_index().sort_values(by='total_amount', ascending=False)
```

Out[1262]:

	PU_Zone	total_amount
60	East Harlem North	12918
61	East Harlem South	8827
79	Forest Hills	3798
30	Central Harlem	3725
128	Morningside Heights	3591
67	Elmhurst	2854
32	Central Park	2849
80	Fort Greene	2126
1	Astoria	1807
197	Washington Heights South	1609

In [1263]:

Top 10 lokasi pengantaran

```
pengantaran = df.groupby(by=['DO_Zone'])[['total_amount']].count().reset_index().sort_values(by='total_amount', ascending=False)
```

Out[1263]:

	DO_Zone	total_amount
67	East Harlem South	3415
66	East Harlem North	3297
218	Upper East Side North	2654
35	Central Harlem	2528
36	Central Harlem North	2246
220	Upper West Side North	2238
150	Morningside Heights	2179
86	Forest Hills	1595
244	Yorkville West	1511
221	Upper West Side South	1402

In [1264]:

Dengan top 1 lokasi penjemputan kita lihat tujuan pengantarannya

```
trip = df.groupby(by=['PU_Zone', 'DO_Zone'])[['total_amount']].count().reset_index()  
trip[trip['PU_Zone'] == 'East Harlem North'].sort_values(by='total_amount', ascending=False).head(10)
```

Out[1264]:

	PU_Zone	DO_Zone	total_amount
1595	East Harlem North	East Harlem South	1970
1678	East Harlem North	Upper East Side North	1165
1578	East Harlem North	Central Harlem	953
1644	East Harlem North	Morningside Heights	923
1594	East Harlem North	East Harlem North	806
1579	East Harlem North	Central Harlem North	730
1680	East Harlem North	Upper West Side North	690
1695	East Harlem North	Yorkville West	674
1580	East Harlem North	Central Park	629
1635	East Harlem North	Manhattan Valley	485

Diketahui zona penjemputan terbanyak dan mendapatkan penghasilan terbanyak adalah daerah East Harlem North . Tabel diatas merupakan top 10 perjalanan terbanyak dengan titik jemput di daerah East Harlem North .

In [1265]:

```
trip1 = df.groupby(['PUday_name', 'PU_Zone', "DO_Zone"])[['total_amount']].sum().reset_index().sort_values(trip1
```

Out[1265]:

	PUday_name	PU_Zone	DO_Zone	total_amount
9368	Tuesday	East Harlem North	East Harlem South	5707.37
7636	Thursday	East Harlem North	East Harlem South	4538.43
9415	Tuesday	East Harlem North	Upper East Side North	4531.54
11088	Wednesday	East Harlem North	East Harlem South	4248.21
2350	Monday	East Harlem North	Upper East Side North	4152.68
...	...	...	...	...
2254	Monday	East Concourse/Concourse Village	Central Park	0.00
9342	Tuesday	East Flatbush/Farragut	Erasmus	0.00
10582	Wednesday	Baisley Park	Baisley Park	0.00
2119	Monday	Corona	Woodside	0.00
5870	Sunday	East Elmhurst	Astoria	0.00

12211 rows × 4 columns

In [1266]:

```
trip1[(trip1['PUday_name'] == 'Tuesday') & (df['PU_Zone'] == 'East Harlem North')].sort_values(by='total_ar
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_27448\282271162.py:1: UserWarning: Boolean Series key will be reindexed to match Data
Frame index.
trip1[(trip1['PUday_name'] == 'Tuesday') & (df['PU_Zone'] == 'East Harlem North')].sort_values(by='total_amount', ascending=False
).head(10)
Out[1266]:
```

	PUday_name	PU_Zone	DO_Zone	total_amount
9368	Tuesday	East Harlem North	East Harlem South	5707.37
9415	Tuesday	East Harlem North	Upper East Side North	4531.54
9417	Tuesday	East Harlem North	Upper West Side North	2526.36
9356	Tuesday	East Harlem North	Central Park	2286.58
9418	Tuesday	East Harlem North	Upper West Side South	2044.68
9455	Tuesday	East Harlem South	East Harlem South	1554.29
9425	Tuesday	East Harlem North	Yorkville East	1236.57
9564	Tuesday	Elmhurst	Jackson Heights	1008.54
9735	Tuesday	Fort Greene	Clinton Hill	883.52
9442	Tuesday	East Harlem South	Central Harlem North	838.80

Hari selasa menjadi hari yang paling banyak transaksi dan pendapatan tertinggi. Tabel di atas merupakan top 10 perjalanan di hari selasa dengan lokasi jemput di East Harlem North.

• Pendapatan perhari & perminggu

```
In [1267]:
# Melihat total seluruh pendapatan taksi hijau per minggu

PU_week_sum = df.groupby([df['lpep_pickup_datetime'].dt.to_period("W")])['total_amount'].sum().reset_index()
PU_week_sum['Week'] = ['Week 1', 'Week 2', 'Week 3', 'Week 4', 'Week 5','Week 6']
PU_week_sum
Out[1267]:
```

	lpep_pickup_datetime	total_amount	Week
0	2022-12-26/2023-01-01	28580.68	Week 1
1	2023-01-02/2023-01-08	283394.18	Week 2
2	2023-01-09/2023-01-15	311650.78	Week 3
3	2023-01-16/2023-01-22	295396.83	Week 4
4	2023-01-23/2023-01-29	319140.28	Week 5
5	2023-01-30/2023-02-05	91137.43	Week 6

```
In [1268]:
# Melihat banyaknya transaksi perjalanan taksi hijau per minggu
```

```
PU_week_ount = df.groupby([df['lpep_pickup_datetime'].dt.to_period("W")])['total_amount'].count().reset_index()
PU_week_ount['Week'] = ['Week 1', 'Week 2', 'Week 3', 'Week 4', 'Week 5','Week 6']
PU_week_ount
```

Out[1268]:

	lpep_pickup_datetime	total_amount	Week
0	2022-12-26/2023-01-01	1266	Week 1
1	2023-01-02/2023-01-08	13782	Week 2
2	2023-01-09/2023-01-15	14764	Week 3
3	2023-01-16/2023-01-22	14025	Week 4
4	2023-01-23/2023-01-29	15096	Week 5
5	2023-01-30/2023-02-05	4242	Week 6

In [1310]:

```
plt.figure(figsize=(20,6))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(PU_week_sum['Week'], PU_week_sum['total_amount'], color = "green")
```

```
plt.title('Total Pendapatan Seluruh Taksi Hijau per Minggu ($)')
```

```
plt.grid()
```

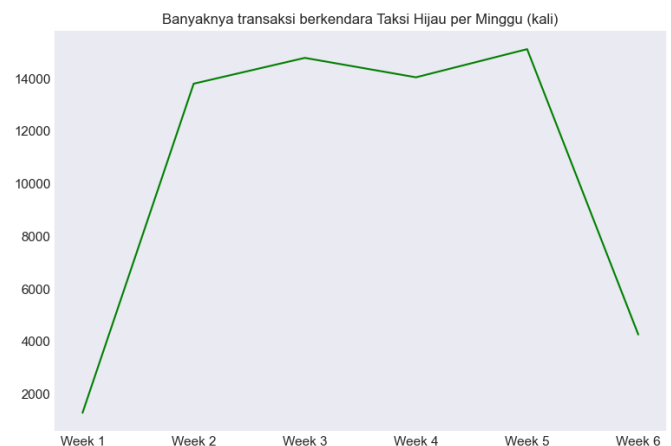
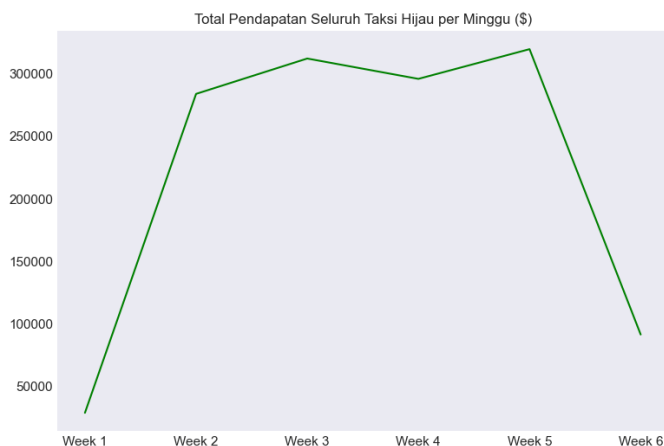
```
plt.subplot(1,2,2)
```

```
plt.plot(PU_week_ount['Week'], PU_week_ount['total_amount'],  
        color = "green")
```

```
plt.title('Banyaknya transaksi berkendara Taksi Hijau per Minggu (kali)')
```

```
plt.grid()
```

```
plt.show()
```



Total pendapatan perminggu paling tinggi ada di pekan ke 5 sebesar \$319,140

In [1270]:

```
# Melihat total seluruh pendapatan taksi hijau per hari
```

```
PU_day_sum = df.groupby([df['lpep_pickup_datetime'].dt.to_period("D")])['total_amount'].sum().reset_index()  
PU_day_sum
```

Out[1270]:

	lpep_pickup_datetime	total_amount
0	2023-01-01	28580.68
1	2023-01-02	29569.20
2	2023-01-03	40093.15
3	2023-01-04	45094.92
4	2023-01-05	47414.31
5	2023-01-06	48743.14
6	2023-01-07	41186.63
7	2023-01-08	31292.83
8	2023-01-09	44061.31
9	2023-01-10	42552.52
10	2023-01-11	49185.67
11	2023-01-12	49360.78
12	2023-01-13	50219.11
13	2023-01-14	41288.44
14	2023-01-15	34982.95
15	2023-01-16	30005.09
16	2023-01-17	44834.48
17	2023-01-18	46464.79
18	2023-01-19	51922.95
19	2023-01-20	47407.77
20	2023-01-21	41938.74
21	2023-01-22	32823.01
22	2023-01-23	45293.58
23	2023-01-24	47464.17
24	2023-01-25	52227.32
25	2023-01-26	53312.43
26	2023-01-27	47547.60
27	2023-01-28	40284.13
28	2023-01-29	33011.05
29	2023-01-30	44425.89
30	2023-01-31	46711.54

In [1271]:

```
# Mengurutkan pendapatan perhari taksi hijau dari yang terbesar  
PU_day_sum.sort_values(by='total_amount', ascending=False).head()
```

Out[1271]:

	lpep_pickup_datetime	total_amount
25	2023-01-26	53312.43
24	2023-01-25	52227.32
18	2023-01-19	51922.95
12	2023-01-13	50219.11
11	2023-01-12	49360.78

In [1272]:

```
PU_day_sum['lpep_pickup_datetime'] = PU_day_sum['lpep_pickup_datetime'].astype(str)
```

In [1273]:

```
# Melihat banyaknya transaksi taksi hijau per hari
```

```
PU_day_count = df.groupby([df['lpep_pickup_datetime'].dt.to_period("D")])['total_amount'].count().reset_index  
PU_day_count
```

Out[1273]:

	lpep_pickup_datetime	total_amount
0	2023-01-01	1266
1	2023-01-02	1442
2	2023-01-03	1963
3	2023-01-04	2236
4	2023-01-05	2220
5	2023-01-06	2365
6	2023-01-07	2003
7	2023-01-08	1553
8	2023-01-09	2083
9	2023-01-10	2024
10	2023-01-11	2318
11	2023-01-12	2345
12	2023-01-13	2343
13	2023-01-14	1993
14	2023-01-15	1658
15	2023-01-16	1460
16	2023-01-17	2094
17	2023-01-18	2175
18	2023-01-19	2416
19	2023-01-20	2266
20	2023-01-21	1977
21	2023-01-22	1637
22	2023-01-23	2208
23	2023-01-24	2252
24	2023-01-25	2475
25	2023-01-26	2476
26	2023-01-27	2241
27	2023-01-28	1891
28	2023-01-29	1553
29	2023-01-30	2078
30	2023-01-31	2164

In [1274]:

```
PU_day_count['lpep_pickup_datetime'] = PU_day_count['lpep_pickup_datetime'].astype(str)
```

In []:

```
# Mengurutkan banyaknya transaksi perhari taksi hijau dari yang terbesar
```

```
PU_day_count.sort_values(by='total_amount', ascending=False).head()
```

Out[]:

	lpep_pickup_datetime	total_amount
25	2023-01-26	2476
24	2023-01-25	2475
18	2023-01-19	2416
5	2023-01-06	2365
11	2023-01-12	2345

Total pendapatan taksi akan naik di hari kerja, dan pendapatan tertinggi perhari ada di tanggal 26 sebesar \$2476

In [1276]:

```
# Rata-rata pendapatan per hari semua taksi hijau
```

```
PU_day_count['total_amount'].mean()
```

Out[1276]:

```
np.float64(2037.9032258064517)
```

In [1277]:

```
# Rata-rata pendapatan per hari taksi hijau
```

```
PU_day_sum['total_amount'].mean()/960
```

Out[1277]:

```
np.float64(44.66734475806451)
```

In [1278]:

```
# Rata-rata perjalanan per armada (911)
```

```
PU_day_count['total_amount'].mean() / 911
```

Out[1278]:

```
np.float64(2.236995857087214)
```

In [1312]:

```
# Grafik pendapatan dan banyaknya transaksi per hari
```

```
plt.figure(figsize=(15,20))
```

```
plt.subplot(2,1,1)
```

```
plt.plot(PU_day_sum['lpep_pickup_datetime'], PU_day_sum['total_amount'], color="green")
```

```
plt.title('Total Pendapatan Seluruh Taksi Hijau per Hari ($)')
```

```
plt.xticks(PU_day_sum['lpep_pickup_datetime'], rotation=90)
```

```
plt.grid()
```

```
plt.subplot(2,1,2)
```

```
plt.plot(PU_day_count['lpep_pickup_datetime'], PU_day_count['total_amount'], color= "green")
```

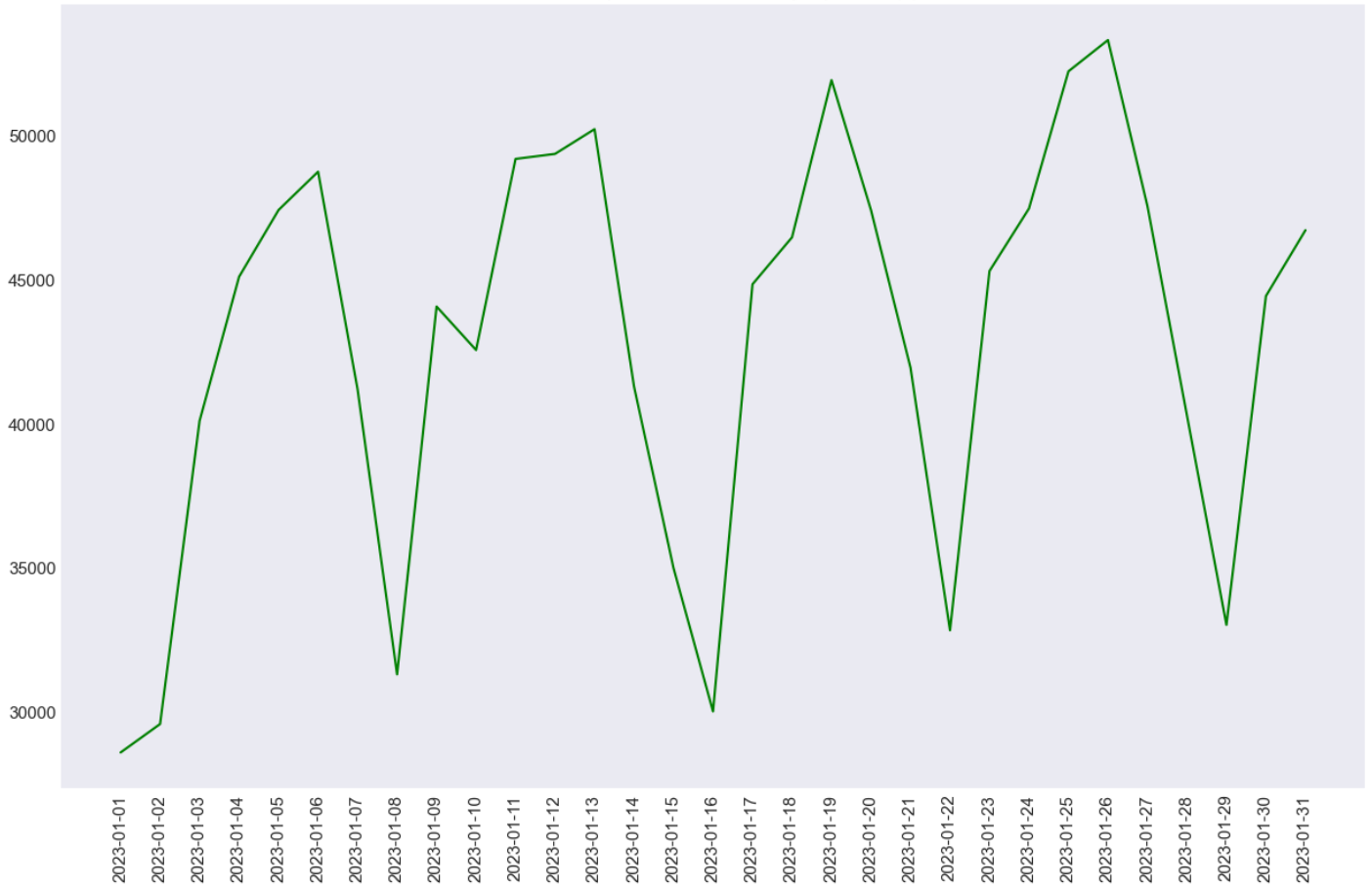
```
plt.title('Banyaknya Transaksi Perjalanan Taksi Hijau per Hari (kali)')
```

```
plt.xticks(PU_day_count['lpep_pickup_datetime'], rotation=90)
```

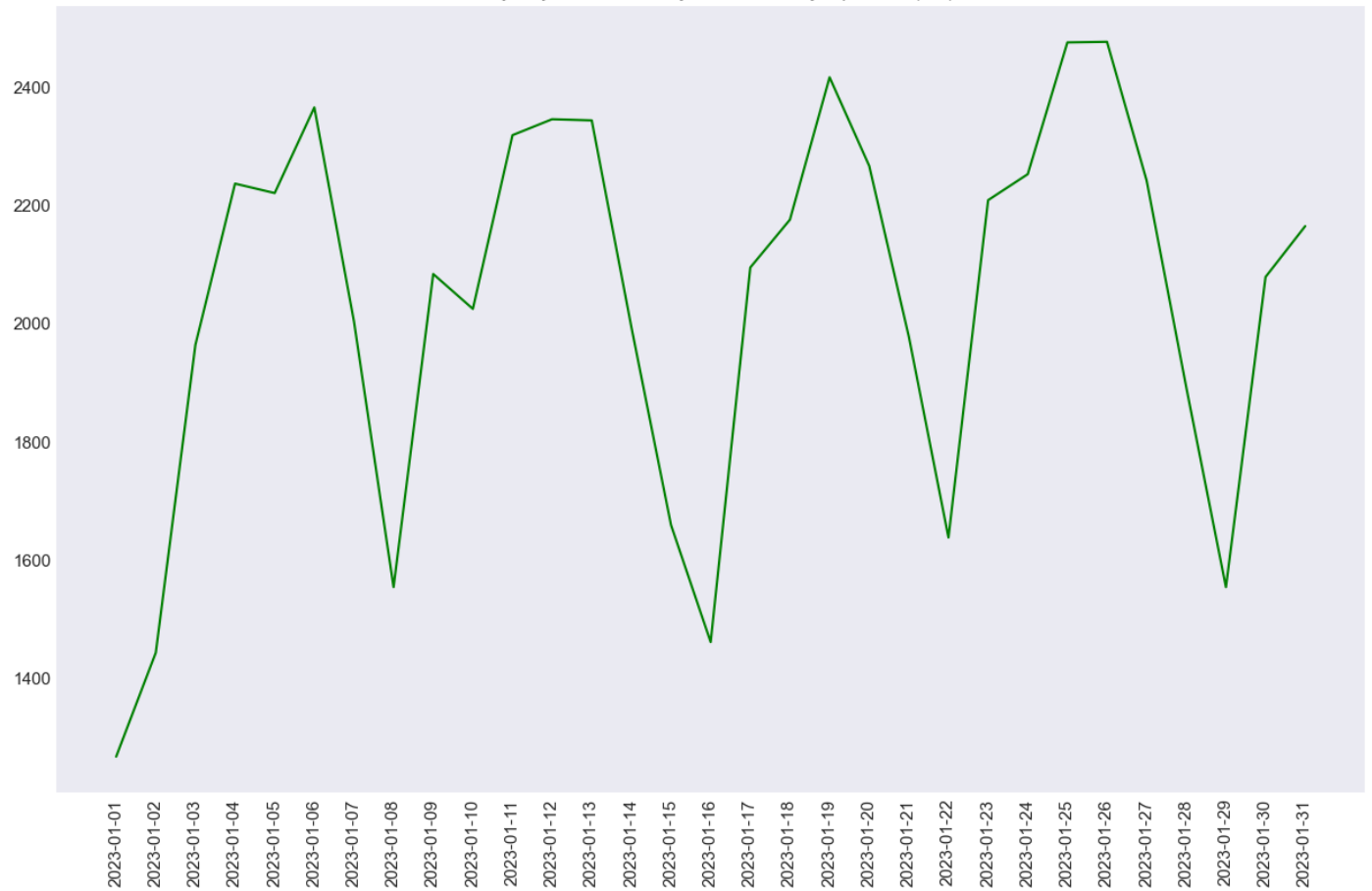
```
plt.grid()
```

```
plt.show()
```

Total Pendapatan Seluruh Taksi Hijau per Hari (\$)



Banyaknya Transaksi Perjalanan Taksi Hijau per Hari (kali)



Total pendapatan taksi akan naik di hari kerja, dan pendapatan tertinggi perhari ada di tanggal 26 sebesar \$2476

In [1280]:

```
# Melihat total pendapatan dengan periode harian
```

```
PU_name_sum = df.groupby([df['lpep_pickup_datetime'].dt.day_name()])['total_amount'].sum().reset_index()
```

```
PU_name_sum["urutan"] = [5,1,6,7,4,2,3]
```

```
PU_name_sum = PU_name_sum.sort_values(by="urutan")
```

```
PU_name_sum
```

```
Out[1280]:
```

	lpep_pickup_datetime	total_amount	urutan
1	Monday	193355.07	1
5	Tuesday	221655.86	2
6	Wednesday	192972.70	3
4	Thursday	202010.47	4
0	Friday	193917.62	5
2	Saturday	164697.94	6
3	Sunday	160690.52	7

```
In [1281]:
```

```
# Melihat banyaknya transaksi dengan periode harian
```

```
PU_name_count = df.groupby([df['lpep_pickup_datetime'].dt.day_name()])['total_amount'].count().reset_index()
```

```
PU_name_count["urutan"] = [5,1,6,7,4,2,3]
```

```
PU_name_count = PU_name_count.sort_values(by="urutan")
```

```
PU_name_count
```

```
Out[1281]:
```

	lpep_pickup_datetime	total_amount	urutan
1	Monday	9271	1
5	Tuesday	10497	2
6	Wednesday	9204	3
4	Thursday	9457	4
0	Friday	9215	5
2	Saturday	7864	6
3	Sunday	7667	7

```
In [1314]:
```

```
# Membuat grafik dengan total pendapatan dan transaksi perjalanan periode harian
```

```
plt.figure(figsize=(20,6))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(PU_name_sum['lpep_pickup_datetime'],PU_name_sum['total_amount'], color = "green")
```

```
plt.title('Total Pendapatan Seluruh Taksi Hijau Harian ($)')
```

```
plt.xticks(PU_name_sum['lpep_pickup_datetime'], rotation=90)
```

```
plt.grid()
```

```
plt.subplot(1,2,2)
```

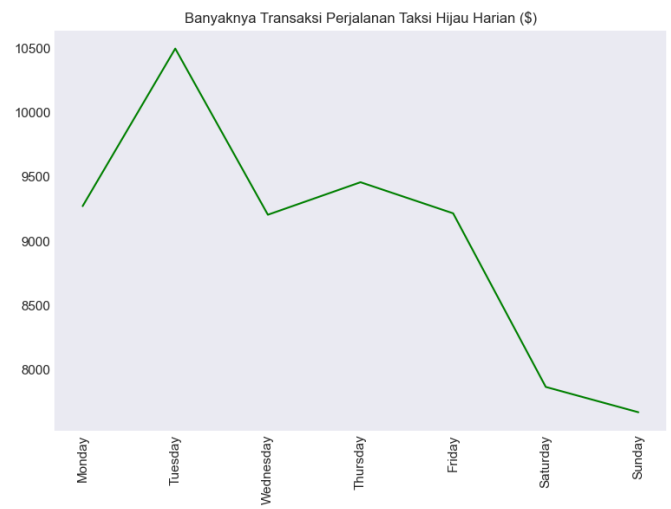
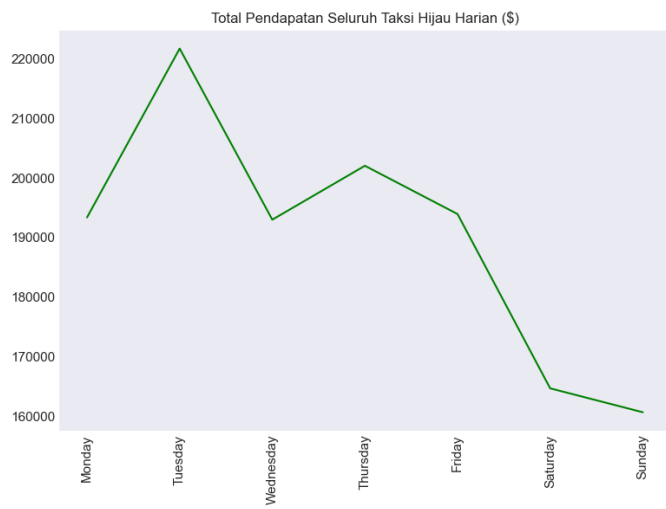
```
plt.plot(PU_name_count['lpep_pickup_datetime'],PU_name_count['total_amount'], color = "green")
```

```
plt.title('Banyaknya Transaksi Perjalanan Taksi Hijau Harian ($)')
```

```
plt.xticks(PU_name_count['lpep_pickup_datetime'], rotation=90)
```

```
plt.grid()
```

```
plt.show()
```



Total pendapatan tertinggi perbulan ada di hari selasa \$221,655

In [1283]:

Simpan file to csv

df.to_csv('C:/Users/HP/Downloads/Projek Modul 2/NYC_databersih.csv', index = **None**, header=**True**)

In []: